



Universidade do Minho
Escola de Engenharia
Licenciatura em Engenharia Informática

Unidade Curricular de Laboratórios de Informática IV

Ano Letivo de 2025/2026

Mini-Formiga

Luís Miguel Silva Coelho – A106843
Gonçalo Duarte Coutinho Martins – A106914
Matilde Rodrigues Teixeira – A106876
Tiago José Pereira Martins – A106927

27 de maio de 2026

LI4

Data de Receção	
Responsável	
Avaliação	
Observações	

Mini-Formiga

Luís Miguel Silva Coelho – A106843
Gonçalo Duarte Coutinho Martins – A106914
Matilde Rodrigues Teixeira – A106876
Tiago José Pereira Martins – A106927

27 de maio de 2026

Resumo

O presente documento constitui a Especificação de Requisitos de Software do sistema Mini-Formiga, desenvolvido no âmbito da Unidade Curricular de Laboratórios de Informática IV da Licenciatura em Engenharia Informática da Universidade do Minho, ano letivo 2025/2026.

Este documento foi produzido com o suporte de um agente, *Large Language Model* (LLM), ao longo de todas as atividades de Engenharia de Requisitos, nomeadamente na elicitação, refinamento, estruturação e validação dos requisitos. O agente foi utilizado como ferramenta de apoio cognitivo, simulando entrevistas com *stakeholders*, identificando ambiguidades e propondo melhorias à especificação. O agente principal utilizado foi o modelo *ChatGPT 5.2* da *OpenAI*.

De forma a mitigar possíveis imprecisões ou alucinações geradas pelo modelo principal, os resultados produzidos foram submetidos a um processo de revisão e validação cruzada com um segundo agente de Inteligência Artificial independente, assegurando um maior grau de fiabilidade e consistência do conteúdo gerado, sendo este último o modelo *Claude Sonnet 4.6* da *Anthropic*.

O documento segue as normas IEEE 830 e ISO/IEC/IEEE 29148:2018 e serve de base para as etapas subsequentes do projeto: Definição da Arquitetura, Implementação e Verificação/Validação.

Área de Aplicação: Engenharia de Requisitos assistida por LLM; Sistemas de Informação para Gestão de Retalho; Ponto de Venda e Controlo de Inventário.

Palavras-Chave: Engenharia de Requisitos, LLM, SRS, IEEE 830, Gestão de Stocks, Ponto de Venda, Lojas de Conveniência, User Stories, Casos de Uso, UML.

Índice

1	Introdução	1
2	Conceção e Engenharia de Requisitos Assistida por LLM	2
2.1	Contextualização	2
2.2	Fundamentação	3
2.3	Apresentação do Caso de Estudo	4
2.4	Motivação e Objetivos	5
2.5	Análise da Viabilidade	6
2.6	Equipa de Trabalho	9
2.7	Plano de execução do trabalho	10
2.8	Estrutura do Relatório	11
2.9	Definição do Domínio do Sistema	12
2.10	Análise de Contexto e Restrições	13
2.11	Identificação de <i>Stakeholders</i>	13
2.12	Método de Levantamento de Requisitos	14
2.13	Descrição Geral dos Requisitos Levantados	15
2.14	User Stories	16
2.15	Especificação dos Requisitos Levantados	25
2.16	Validação dos Requisitos Levantados	30
2.17	Apresentação geral da especificação	33
2.17.1	Identificação e Caracterização das Entidades	34
2.18	Casos de Uso	38
2.18.1	Diagrama de Use Cases	38
2.18.2	Especificação de Use Cases	39
2.19	Utilização de Agentes LLM na Engenharia de Requisitos	53
3	Arquitetura e Design do Software utilizando LLM	58
3.1	Arquitetura Global do Sistema	58
3.2	Diagrama de Classes	61
3.2.1	Organização Modular e Responsabilidades	62
3.2.2	Padrões de Projeto Utilizados	64
3.2.3	Decisões de Design Orientado a Objetos	65
3.2.4	Exemplos Concretos Derivados de Casos de Utilização	67
3.3	Diagramas de Sequência	67
3.4	Diagrama de Componentes	73
3.5	Definição da Stack Tecnológica	74

3.6	Especificação de API	79
3.7	Esboço das Interfaces do Sistema	83
3.7.1	Autenticação e seleção de perfil	83
3.7.2	Interfaces do Gestor da Cadeia	85
3.8	Conceção do Modelo da Persistência de Dados	86
3.9	Decisões Arquiteturais	90
3.10	Exploração de LLM no Design Arquitetural	94
4	Implementação e Desenvolvimento Assistido por LLM	100
4.1	Configuração do Ambiente	100
4.2	Implementação por Módulos	101
4.3	Geração de Código com LLM	102
4.4	Decisões e Adaptações da Fase de Implementação	103
4.5	Revisão de código gerado	104
4.6	Deteção de <i>smells</i> e sugestão de modificações	106
5	Verificação, Validação e Avaliação da Qualidade do Software	112
5.1	Estratégia de Testes	112
5.2	Testes Unitários	113
5.3	Testes de Integração	113
5.4	Testes de Sistema	114
5.5	Testes de Aceitação	115
5.6	Uso de LLM para Criação de Testes	116
5.7	Cobertura de Código	116
5.8	Avaliação segundo ISO/IEC 25010	117
5.9	Verificação da satisfação dos SRS	120
6	Conclusões e Trabalho Futuro	126
6.1	Apreciação Crítica do Trabalho Realizado	126
6.2	Trabalho Futuro	128
6.3	Reflexão Final	128
	Lista de Siglas e Acrónimos	131
	Anexos	132
	Anexo 1 — Entrevistas com os <i>Stakeholders</i>	132
	Anexo 2 — Ata da reunião com o Sr. Formiga para Validação de requisitos	141
	Anexo 3 — Relacionamentos do Modelo de Domínio do Mini-Formiga	143
	Anexo 4 — Mockups das Interfaces do Sistema	150
	Anexo 5 — Ata da reunião com o Sr. Formiga para Validação e Refinamento de requisitos	159
	Anexo 6 — Guia de Utilização e Manutenção do Trabalho	161

Lista de Figuras

2.1	Diagrama de Gantt do projeto Mini-Formiga	11
2.2	Modelo de Domínio do sistema Mini-Formiga	34
2.3	Diagrama de Casos de Uso do sistema Mini-Formiga	39
3.1	Arquitetura global do sistema Mini-Formiga	58
3.2	Diagrama de Classes do sistema Mini-Formiga	62
3.3	Diagrama de sequência do caso de uso UC-02 – Registrar Venda no PDV	68
3.4	Diagrama de sequência do caso de uso UC-05 – Registrar Fecho de Caixa Diário	70
3.5	Diagrama de sequência do método iniciarSincronizacao(lojaId)	71
3.6	Diagrama de Componentes do sistema Mini-Formiga	73
3.8	Mockup do <i>dashboard</i> do Gestor da Cadeia	86
3.9	Modelo lógico da base de dados local do sistema Mini-Formiga	90
3.10	Modelo lógico da base de dados global do sistema Mini-Formiga	90
5.1	Relatórios de cobertura e testes de mutação	117
6.1	Ata da reunião com o Sr. Formiga	142
6.2	Mockup da interface de relatórios do Gestor da Cadeia	150
6.3	Mockup da interface de gestão de fornecedores	151
6.4	Mockup da interface de encomendas	151
6.5	Mockup da interface de gestão de utilizadores	152
6.6	Mockup da interface de sincronização	152
6.7	Mockup da interface de consulta de stock e alertas	153
6.8	Mockup da interface de fecho de caixa	153
6.9	Mockup da interface de relatórios do Gerente de Loja	154
6.10	Mockup da interface de ajuste manual de inventário	154
6.11	Mockup da interface de gestão de funcionários	155
6.12	Mockup da interface de registo de venda no PDV	155
6.13	Mockup da interface de processamento de devolução	156
6.14	Mockup da interface de consulta de stock do armazém	156
6.15	Mockup da interface de registo de entrada de mercadoria	157
6.16	Mockup da interface de inventário físico	157
6.17	Ata da reunião com o Sr. Formiga (página 1)	159

Lista de Tabelas

2.1	Identificação dos <i>Stakeholders</i> do Mini-Formiga	14
2.2	<i>User Stories</i> do Gestor da Cadeia	17
2.3	CrITÉrios de Verificação das <i>User Stories</i> do Gestor da Cadeia	18
2.4	<i>User Stories</i> do Gerente de Loja	19
2.5	CrITÉrios de Verificação das <i>User Stories</i> do Gerente de Loja	20
2.6	<i>User Stories</i> do Funcionário	21
2.7	CrITÉrios de Verificação das <i>User Stories</i> do Funcionário	22
2.8	<i>User Stories</i> do Responsável de Armazém	23
2.9	CrITÉrios de Verificação das <i>User Stories</i> do Responsável de Armazém	24
2.10	Requisitos Funcionais do Sistema Mini-Formiga	27
2.11	Requisitos Não Funcionais do Sistema Mini-Formiga	29
2.12	Requisitos de Domínio do Sistema Mini-Formiga	30
2.13	Refinação dos requisitos após validação com o Sr. Formiga e confirmação pela equipa	33
2.14	Entidades do Modelo de Domínio do Mini-Formiga	37
2.15	UC-01 – Autenticar no Sistema	40
2.16	UC-02 – Registar Venda no PDV	41
2.17	UC-03 – Emitir Fatura	42
2.18	UC-04 – Processar Devolução de Produto	43
2.19	UC-05 – Registar Fecho de Caixa Diário	44
2.20	UC-06 – Consultar Indicadores e Gerar Relatórios	45
2.21	UC-07 – Consultar Stock e Receber Alertas de Reposição	46
2.22	UC-08 – Registar Entrada de Mercadoria	47
2.23	UC-09 – Criar Encomenda a Fornecedor	48
2.24	UC-10 – Registar Ajuste Manual de Inventário	49
2.25	UC-11 – Realizar Inventário Físico	50
2.26	UC-12 – Gerir Utilizadores e Acessos	51
2.27	UC-13 – Sincronizar Dados com Servidor Central	52
2.28	UC-14 – Gerir Fornecedores e Condições Comerciais	53
3.1	Comparação de alternativas para o <i>backend</i>	75
3.2	Comparação de alternativas para o <i>frontend</i>	76
3.3	Comparação de alternativas para persistência de dados	77
3.4	Comparação entre arquitetura monolítica modular e micro-serviços	78
3.5	Endpoints – Autenticação	80
3.6	Endpoints – Utilizadores	80

3.7	Endpoints – Produtos, Categorias e Meios de Pagamento	81
3.8	Endpoints – Vendas, Faturas, Devoluções e Fechos de Caixa	83
3.9	Principais interações com o agente de LLM e impacto nas decisões de design .	99
4.1	Taxonomia de <i>code smells</i> analisados pelo agente de IA, por categoria e severidade típica	108
4.2	<i>Code smells</i> identificados no código do projeto	110
5.1	Avaliação da qualidade segundo ISO/IEC 25010	119
5.2	Verificação dos requisitos funcionais	122
5.3	Verificação dos requisitos não funcionais	124
5.4	Verificação dos requisitos de domínio	125
6.1	Relacionamentos do Modelo de Domínio do Mini-Formiga	150

1 Introdução

No contexto atual, caracterizado pela crescente digitalização dos processos de negócio, a adoção de sistemas de informação tornou-se essencial para garantir a eficiência, a organização e a escalabilidade das operações empresariais. Em particular, no setor do retalho, onde a gestão de vendas, inventário e fornecedores exige precisão e rapidez, a utilização de soluções tecnológicas integradas assume um papel fundamental.

O presente trabalho insere-se no âmbito da unidade curricular de Laboratórios de Informática IV da Licenciatura em Engenharia Informática da Universidade do Minho e tem como objetivo o desenvolvimento do sistema Mini-Formiga, uma plataforma de gestão integrada direcionada para cadeias de lojas de conveniência.

Este projeto surge da necessidade de modernizar os processos de gestão de uma loja de conveniência em fase de expansão, onde métodos manuais e ferramentas dispersas se revelam insuficientes para suportar o aumento da complexidade operacional. Assim, propõe-se a conceção de um sistema capaz de centralizar informação, automatizar tarefas e fornecer suporte à tomada de decisão.

Ao longo deste relatório, são abordadas as diferentes fases do ciclo de vida do software, incluindo a engenharia de requisitos, a definição da arquitetura, a implementação e a validação do sistema. Destaca-se ainda a utilização de modelos de linguagem (LLM) como ferramentas de apoio ao desenvolvimento, contribuindo para a elicitação, análise e validação de requisitos.

Desta forma, o sistema Mini-Formiga pretende não só melhorar a eficiência operacional e reduzir erros associados à gestão manual, como também preparar o negócio para uma expansão sustentável, garantindo consistência, fiabilidade e controlo centralizado.

2 Conceção e Engenharia de Requisitos Assistida por LLM

2.1 Contextualização

O Sr. Formiga sempre soube que uma loja de conveniência vive de ritmo. Não o ritmo apressado dos grandes centros comerciais, mas o cadenciado pulsar de uma comunidade que sabe onde encontrar o que precisa a qualquer hora. Durante anos, a sua única loja foi exatamente isso: um ponto de referência no bairro, gerido com a intuição de quem conhece cada cliente pelo nome e cada produto pela prateleira onde repousa.

O seu estabelecimento, uma simples loja de conveniência, tornou-se rapidamente um espaço onde o essencial e o imediato se cruzam. Nas suas prateleiras, o *stock* era uma mistura estratégica de comidas prontas para consumo, como sanduíches frescas e *snacks*, uma parede de bebidas e refrigerantes sempre gelados, mercearia de emergência para colmatar faltas de última hora na despensa e artigos de higiene pessoal.

A loja abre as portas de segunda a sábado, às 7h30, quando o bairro ainda desperta com o cheiro do pão quente e o primeiro café do dia. Mantém-se em funcionamento até às 20h00, acompanhando o regresso apressado de quem sai do trabalho e precisa de fazer o jantar ou repor um produto esquecido. Ao domingo, as luzes permanecem apagadas, num raro momento de pausa que permite reorganizar prateleiras e organizar encomendas.

As encomendas aos fornecedores seguem também um ritmo próprio. O armazém central do principal fornecedor funciona de segunda a sexta-feira, entre as 8h00 e as 18h00, sendo este o intervalo durante o qual o Sr. Formiga pode formalizar pedidos de reposição e assegurar a sua preparação no próprio dia. Fora desse horário, as encomendas ficam registadas, mas apenas são processadas no início do dia útil seguinte. Assim, a gestão do *stock* exige uma atenção redobrada, pois uma decisão tardia pode significar prateleiras vazias no dia seguinte.

O Sr. Formiga, de nome completo Manuel Joaquim Formiga, tem 52 anos e cresceu no mesmo bairro onde hoje trabalha, filho de uma costureira e de um padeiro, tendo aprendido desde cedo o valor da proximidade com as pessoas e da confiança no comércio local.

Mas a intuição tem limites, e o Sr. Formiga começou a senti-los. Com a oportunidade de expansão à porta, novos espaços comerciais disponíveis em localidades próximas, o que antes era gerível pela memória e por folhas de cálculo dispersas tornou-se subitamente insuficiente.

Uma loja conhece-se de cor. Uma cadeia de lojas exige um sistema.

Foi então que o Sr. Formiga se sentou, pela primeira vez, com verdadeira atenção, diante do computador que sempre estivera ali, encostado ao balcão como um funcionário esquecido. O que encontrou não o surpreendeu, mas perturbou-o: registos de vendas em ficheiros .x/sx sem nomenclatura consistente, listas de *stock* atualizadas à mão em documentos separados por mês, contactos de fornecedores espalhados entre e-mails e uma agenda de papel, e um caderno de capa azul onde o fecho de caixa era registado todas as noites com letra miúda e tinta que já começava a desvanecer.

Aquele caderno azul era, afinal, o coração do sistema. E o Sr. Formiga percebeu que um coração assim não sobrevive a uma cadeia de lojas.

2.2 Fundamentação

Com o intuito de modernizar a gestão da sua cadeia de lojas, eliminar a dispersão de informação e preparar o negócio para uma expansão sustentada, o Sr. Formiga pretende um sistema de gestão integrado, funcional e de utilização prática para os seus colaboradores.

Com este sistema, deve ser possível que cada loja opere de forma autónoma durante o dia, registando vendas, controlando stock e gerindo as operações de caixa localmente, com sincronização automática para o servidor central no final de cada dia. Desta forma, o Sr. Formiga poderá acompanhar o desempenho de todas as lojas a partir de um único ponto de acesso, sem depender de compilações manuais de informação.

O sistema deverá disponibilizar um terminal de Ponto de Venda (PDV) intuitivo, permitindo que os operadores de caixa registem transações de forma rápida e sem erros. O PDV deverá ainda suportar múltiplos meios de pagamento e emitir automaticamente recibos e faturas sempre que solicitados.

Para além disso, o sistema deverá incluir um módulo de controlo de stock que monitorize os níveis de inventário em tempo real e emita alertas automáticos quando um produto atingir o seu nível mínimo. Com base nesses alertas, deverá ser possível gerar sugestões de encomenda a fornecedores, agilizando o processo de reabastecimento e evitando ruturas de stock.

O Sr. Formiga deverá poder consultar, a qualquer momento, relatórios detalhados de vendas, rentabilidade e desempenho, filtráveis por loja, por período e por categoria de produto, exportáveis em PDF ou CSV, de modo a facilitar a comunicação com o seu contabilista.

Por fim, o sistema deverá garantir a segurança e a integridade dos dados através da autenticação por perfil de utilizador, do registo de auditoria de todas as operações financeiras e da comunicação cifrada entre os terminais da loja e o servidor central, assegurando a conformidade com o Regulamento Geral sobre a Proteção de Dados e com as normas de faturação da Autoridade Tributária.

2.3 Apresentação do Caso de Estudo

O caso de estudo incide sobre a **cadeia de lojas de conveniência do Sr. Formiga**. Atualmente, o Sr. Formiga opera uma única loja física e encontra-se numa fase de expansão planeada para múltiplas localizações, pretendendo manter uma gestão centralizada e eficiente de toda a cadeia.

As lojas comercializam produtos de consumo frequente, incluindo alimentos prontos a consumir, bebidas, produtos de higiene e artigos de conveniência doméstica, procurando responder às necessidades quotidianas da comunidade envolvente.

O modelo de funcionamento assenta na operação autónoma de cada estabelecimento ao longo do dia, com a posterior consolidação centralizada dos dados no final de cada dia, permitindo uma supervisão global, consistente e atempada do desempenho do negócio.

Atualmente, a cadeia é composta por 1 loja em operação. No âmbito deste projeto, o sistema Mini-Formiga foi concebido para suportar uma cadeia de até 5 lojas, correspondendo à expansão planeada pelo Sr. Formiga a curto e médio prazo. A arquitetura adotada garante, no entanto, escalabilidade para além deste número sem necessidade de alterações estruturais ao sistema.

Neste contexto, o sistema a desenvolver, denominado **Mini-Formiga**, surge como uma plataforma integrada de gestão destinada a suportar todos os processos operacionais e administrativos da cadeia.

Este software terá como objetivo assegurar a organização, monitorização e controlo das atividades comerciais, desde o registo de vendas no ponto de venda até à análise de rentabilidade por loja, incluindo ainda funcionalidades de controlo de *stock*, gestão de fornecedores e emissão de faturação.

Em termos de utilizadores do sistema, cada loja conta com 1 gerente de loja, 1 responsável de armazém e 2 a 3 funcionários operadores de PDV. O Sr. Formiga, enquanto gestor da cadeia, acede ao sistema de forma centralizada a partir de qualquer loja ou dispositivo. O contabilista e os fornecedores interagem com o sistema de forma pontual e indireta, não constituindo utilizadores regulares da plataforma.

A implementação do **Mini-Formiga** visa, assim, proporcionar uma solução tecnológica fiável, intuitiva e centralizada, capaz de apoiar a tomada de decisão, reduzir a dispersão de informação e sustentar o crescimento estruturado da cadeia de lojas, garantindo simultaneamente a eficiência operacional e a consistência na gestão.

2.4 Motivação e Objetivos

Motivação

Desde a abertura da sua primeira loja, o Sr. Formiga acompanhou pessoalmente cada detalhe da operação, garantindo que o funcionamento diário decorria sem falhas e que cada cliente era atendido com a proximidade que sempre caracterizou o seu negócio. Durante anos, este modelo artesanal revelou-se eficaz, permitindo-lhe manter controlo direto sobre vendas, inventário e fornecedores. Contudo, a perspetiva de expansão para múltiplas lojas trouxe consigo um aumento significativo da complexidade operacional e da quantidade de informação a gerir.

A intenção de abrir novos estabelecimentos levou o Sr. Formiga a analisar com maior atenção os métodos utilizados no registo e organização dos dados da loja. O que encontrou revelou um conjunto de práticas acumuladas ao longo do tempo, resultantes mais da necessidade imediata do que de um planeamento estruturado. Os registos de vendas encontravam-se dispersos por vários ficheiros .xlsx, sem convenções consistentes de nomenclatura. Os níveis de stock eram anotados manualmente em documentos mensais independentes, os contactos de fornecedores estavam distribuídos entre mensagens de correio eletrónico, anotações em papel e listas avulsas, e o fecho diário de caixa continuava a ser registado no antigo caderno azul que acompanhava o balcão desde os primeiros anos da loja.

Embora cada um destes métodos funcionasse isoladamente, a coexistência de formatos distintos e não interligados tornava impossível obter uma visão global e fiável do negócio. A ausência de integração entre dados dificultava a análise de desempenho, impedia a automatização de tarefas rotineiras e aumentava o risco de erros humanos, inconsistências e perda de informação. Além disso, a dependência do conhecimento informal do próprio Sr. Formiga representava um obstáculo à escalabilidade, uma vez que grande parte dos processos não estava documentada nem sistematizada.

Perante este cenário, tornou-se evidente que o modelo de gestão existente não seria sustentável num contexto de crescimento. A abertura de novas lojas multiplicaria a quantidade de dados e operações, tornando impraticável a consolidação manual de informação e comprometendo a capacidade de supervisão centralizada. A necessidade de um sistema integrado deixou de ser apenas uma melhoria desejável e passou a constituir um requisito essencial para assegurar a continuidade e a evolução do negócio.

Assim, o Sr. Formiga decidiu avançar com a implementação de uma solução tecnológica que permitisse centralizar a gestão da cadeia, uniformizar os registos, automatizar processos e garantir a fiabilidade da informação. A criação deste sistema permitirá reduzir erros operacionais, melhorar a eficiência do atendimento, simplificar tarefas administrativas e libertar tempo para que o proprietário possa concentrar-se na definição de estratégias de crescimento e expansão.

Para além dos benefícios operacionais, esta modernização representa também um passo natural na evolução da loja que sempre foi um ponto de referência local. Ao integrar tecnologia com

a proximidade que sempre caracterizou o seu serviço, o Sr. Formiga procura preservar a identidade do negócio enquanto o prepara para enfrentar os desafios de uma cadeia de lojas, garantindo solidez, consistência e capacidade de adaptação a longo prazo.

Objetivos

Após analisar as limitações do modelo atual e considerar as necessidades associadas à expansão da cadeia, o Sr. Formiga definiu um conjunto de objetivos a alcançar com a implementação do sistema de gestão integrado, nomeadamente:

- Centralização da gestão da cadeia, através de uma plataforma única que agregue os dados de todas as lojas, eliminando a dispersão de informação e permitindo uma visão consolidada e atualizada do negócio;
- Garantia de autonomia operacional de cada loja, possibilitando o funcionamento independente durante o dia, com sincronização automática dos dados com o servidor central no final de cada dia;
- Automatização do controlo de *stock*, assegurando a monitorização contínua dos níveis de inventário e a emissão de alertas de reposição antes da ocorrência de ruturas de stock;
- Modernização do ponto de venda, substituindo registos manuais por um terminal PDV intuitivo, reduzindo tempos de atendimento e erros operacionais;
- Profissionalização do processo de faturação, garantindo conformidade com as normas legais e fiscais e mantendo um histórico organizado e acessível de todas as transações;
- Disponibilização de relatórios de desempenho e rentabilidade, filtráveis por loja, período e categoria de produto, que apoiem a tomada de decisão informada;
- Automatização de tarefas administrativas, reduzindo a carga de trabalho manual e aumentando a eficiência operacional;
- Preparação do sistema para expansão futura, garantindo escalabilidade e capacidade de integração de novas lojas sem comprometer a organização e o controlo da cadeia.

2.5 Análise da Viabilidade

Com o crescimento sustentado do negócio, o Sr. Formiga percebeu que a expansão da sua loja para uma cadeia de estabelecimentos implicava desafios que já não poderiam ser resolvidos apenas com experiência e boa gestão. A multiplicação de pontos de venda traria consigo um aumento significativo no volume de dados de vendas, movimentos de *stock*, encomendas

a fornecedores e faturação, tornando evidente a necessidade de uma solução estruturada e centralizada.

O projeto **Mini-Formiga** surge, assim, como resposta direta a essa necessidade de controlo e escalabilidade. Num contexto onde a margem de lucro das lojas de conveniência depende fortemente da rotatividade de produtos, da gestão eficiente de inventário e da reposição atempada, a ausência de um sistema integrado representa um risco operacional elevado. Ruturas frequentes, excesso de *stock* parado ou erros de faturação traduzem-se em perdas financeiras que poderiam ser evitadas com o apoio tecnológico adequado.

Quanto à utilidade do sistema, considerou-se fundamental desenvolver uma solução que permita:

- Centralizar a informação de todas as lojas num único sistema de dados;
- Garantir autonomia operacional diária de cada estabelecimento;
- Automatizar o controlo de inventário e a geração de alertas de reposição;
- Disponibilizar relatórios de apoio à decisão, baseados em dados consolidados.

Desta forma, o Sr. Formiga poderá gerir a cadeia com base em indicadores concretos como: vendas por loja, margens de lucro por categoria de produto, desempenho por período, reduzindo a dependência da intuição e aumentando a previsibilidade do negócio.

Viabilidade Técnica

Do ponto de vista técnico, o sistema Mini-Formiga é exequível com recurso a tecnologias maduras e amplamente adotadas no desenvolvimento de sistemas de gestão empresarial. A arquitetura prevista assenta num modelo cliente-servidor, com terminais locais em cada loja e um servidor central responsável pela consolidação de dados, uma abordagem tecnicamente bem documentada e suportada por um vasto ecossistema de ferramentas.

Os principais riscos técnicos identificados, e as respetivas estratégias de mitigação, são os seguintes:

- **Conformidade fiscal:** a emissão de documentos fiscais em conformidade com as normas da Autoridade Tributária exige o cumprimento de requisitos de numeração sequencial e de formato legalmente definidos. Este aspeto não representa um obstáculo técnico intransponível, mas implica uma análise cuidada da legislação aplicável durante a fase de implementação.
- **Segurança das comunicações:** a transmissão de dados financeiros entre os terminais de loja e o servidor central deve ser protegida por protocolos de cifra modernos, requisito suportado de forma nativa pelas principais *frameworks* de desenvolvimento web e de

comunicação em rede.

Viabilidade Económica

As ferramentas e tecnologias a selecionar serão de utilização gratuita ou de código aberto, não implicando licenciamento adicional. Os custos de infraestrutura para efeitos de desenvolvimento e teste são igualmente reduzidos, podendo ser suportados por serviços de *cloud* com planos gratuitos ou de baixo custo.

Em termos do retorno esperado para o negócio do Sr. Formiga, a implementação do sistema permitirá, de forma estimada:

- Reduzir o tempo dedicado a tarefas administrativas manuais, tais como consolidação de dados, fecho de caixa e geração de encomendas, libertando recursos humanos para atividades de maior valor acrescentado;
- Diminuir as perdas associadas a ruturas de *stock* e a erros de faturação, cujo impacto financeiro é reconhecido pelo próprio Sr. Formiga como extremamente relevante;
- Suportar a expansão para novas lojas sem aumento proporcional dos custos administrativos, dado que o sistema será concebido para ser escalável desde a sua base.

Viabilidade Operacional

A adoção de um novo sistema de gestão implica sempre um período de transição que pode introduzir resistência por parte dos utilizadores, em particular dos funcionários com menor familiaridade tecnológica. Para mitigar este risco, o sistema será especificado com uma interface de ponto de venda simples e intuitiva, minimizando a curva de aprendizagem e reduzindo a probabilidade de erros operacionais durante o atendimento.

A transição do modelo atual, baseado em ficheiros dispersos e registos manuais, para o sistema Mini-Formiga deverá ser faseada, começando por uma loja piloto, de forma a validar os processos e corrigir eventuais problemas antes da expansão à cadeia completa.

Viabilidade Temporal

O projeto está estruturado em quatro fases principais, alinhadas com as etapas definidas no enunciado da unidade curricular: Engenharia de Requisitos, Arquitetura e Modelação, Implementação, e Verificação e Validação. O calendário detalhado encontra-se no diagrama de Gantt apresentado na Secção 2.7.

A viabilidade temporal foi avaliada tendo em conta a dimensão da equipa, o volume de funcionalidades a desenvolver e as dependências entre fases. As funcionalidades nucleares do negócio: ponto de venda, controlo de *stock* e fecho de caixa, foram priorizadas para implementação obrigatória dentro do prazo, estando as funcionalidades de menor criticidade operacional sujeitas à disponibilidade de tempo residual. Esta estratégia de priorização, a formalizar durante a fase de levantamento de requisitos, garante que o sistema entregue no final do projeto é funcional e responde às necessidades essenciais do negócio.

Em síntese, o projeto Mini-Formiga apresenta viabilidade em todas as dimensões analisadas, constituindo um passo essencial para a modernização e sustentabilidade futura da cadeia de lojas do Sr.Formiga.

2.6 Equipa de Trabalho

Com o objetivo de estruturar o crescimento da sua cadeia de lojas e modernizar os seus processos internos, o Sr. Formiga decidiu apostar no desenvolvimento de um sistema de gestão integrada adaptado às suas necessidades específicas.

Para concretizar o desenvolvimento do Mini-Formiga, foi constituída uma equipa de desenvolvimento, assumindo diferentes responsabilidades no processo de desenvolvimento do software.

A equipa organiza-se da seguinte forma:

- **Engenheiro Informático / Desenvolvedor Principal:** responsável pela implementação do sistema, desenvolvimento dos módulos de Ponto de Venda, gestão de *stock*, faturação e sincronização de dados, bem como pela integração entre componentes.
- **Analista de Sistemas:** encarregado pela recolha e análise de requisitos, modelação do sistema (casos de uso, diagramas UML, modelo de dados) e validação da conformidade entre requisitos e implementação.
- **Responsável pela Qualidade e Testes:** responsável pela definição da estratégia de testes, elaboração de casos de teste, execução de testes funcionais e de integração, bem como verificação dos requisitos não funcionais.
- **Gestor de Projeto:** encarregado pelo planeamento das atividades, acompanhamento do cronograma, coordenação das tarefas da equipa e garantia do cumprimento dos prazos estabelecidos.

Para além dos elementos humanos, o projeto integra ainda a utilização de LLM como agentes artificiais de apoio ao longo das diferentes fases do ciclo de vida do software, nomeadamente na geração de documentação, refinamento de requisitos e apoio à implementação. Ainda assim, todas as decisões técnicas e validações finais são da responsabilidade da equipa.

Esta organização permite uma divisão clara de responsabilidades, promovendo simultaneamente

o trabalho colaborativo e a revisão cruzada dos componentes produzidos.

2.7 Plano de execução do trabalho

A execução do projeto **Mini-Formiga** foi estruturada em diferentes etapas, seguindo uma abordagem sequencial e incremental ao longo de aproximadamente quatro meses.

O planeamento das atividades teve como referência direta os períodos oficialmente estabelecidos para cada etapa, garantindo o cumprimento rigoroso das datas indicadas no cronograma. A distribuição temporal das tarefas dentro de cada etapa foi realizada tendo em conta:

- A complexidade e volume de trabalho associado a cada fase;
- A dependência entre tarefas, por exemplo, a implementação depende da conclusão do design;
- A necessidade de reservar tempo para revisões, validações intermediárias e eventuais correções;
- A preparação das entregas parciais e da apresentação final.

As datas de entrega foram definidas de acordo com os marcos estabelecidos no enunciado, assegurando que cada etapa produza os resultados necessários para a fase seguinte. A representação detalhada da calendarização das tarefas, respetivas durações e interdependências encontra-se ilustrada no diagrama de Gantt apresentado de seguida.

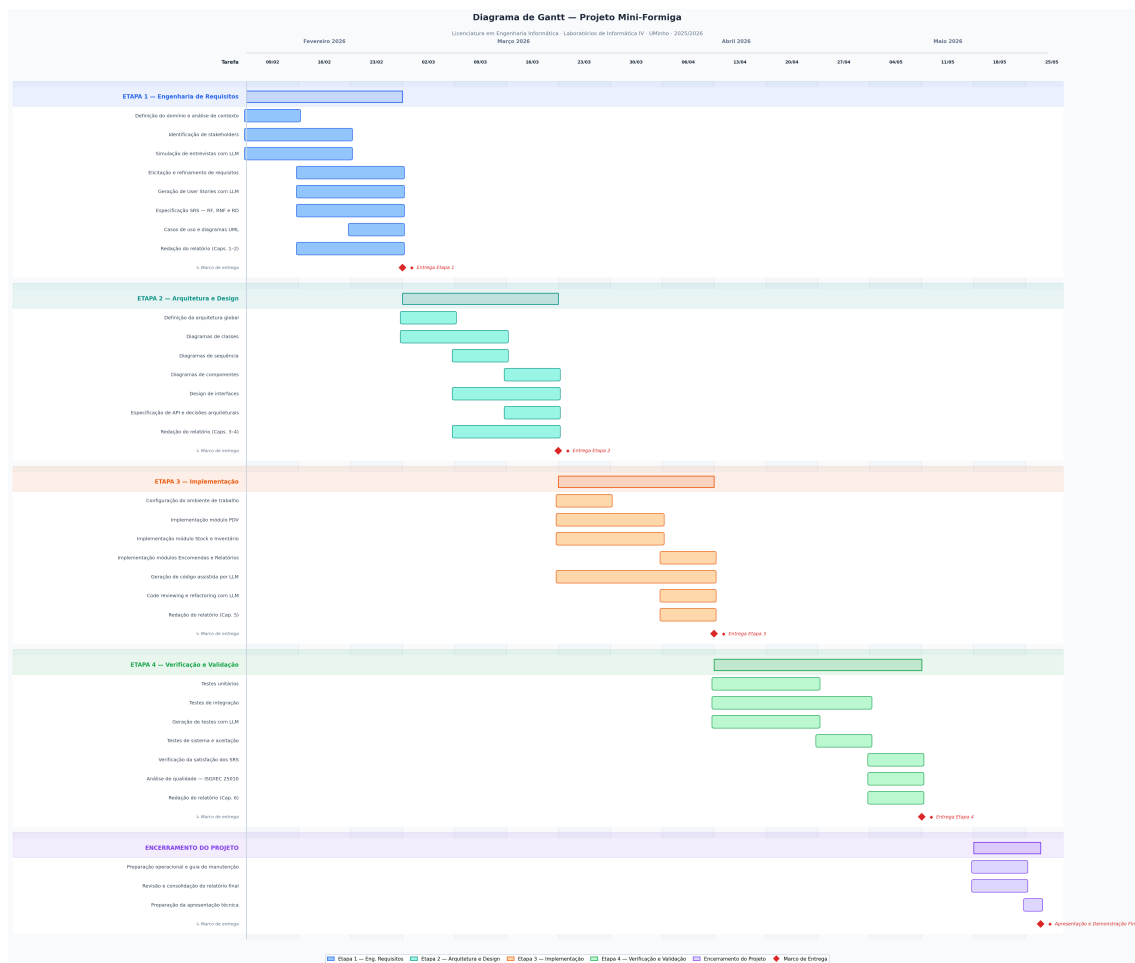


Figura 2.1: Diagrama de Gantt do projeto Mini-Formiga

2.8 Estrutura do Relatório

O presente relatório encontra-se organizado de forma a acompanhar, de modo estruturado e progressivo, as diferentes etapas do desenvolvimento do sistema de gestão integrada **Mini-Formiga**, desde a contextualização do problema até à implementação, verificação e avaliação da qualidade do sistema.

O **Capítulo 1 — Introdução** apresenta o enquadramento geral do projeto, descrevendo o contexto de digitalização dos processos de retalho, a necessidade de modernização da gestão da cadeia de lojas do Sr. Formiga e os objetivos globais do sistema Mini-Formiga.

No **Capítulo 2 — Conceção e Engenharia de Requisitos Assistida por LLM**, é apresentado o caso de estudo em detalhe, incluindo a contextualização do negócio, a fundamentação da necessidade do sistema, a motivação, os objetivos, a análise de viabilidade, a equipa de trabalho e o plano de execução. Este capítulo inclui ainda a definição do domínio do sistema, a análise de contexto e restrições, a identificação dos *stakeholders*, o método de levantamento

de requisitos, as *user stories*, os requisitos funcionais, não funcionais e de domínio, bem como a respetiva validação. Na parte final do capítulo, são também apresentados o modelo de domínio, a identificação das entidades e relacionamentos e a especificação dos casos de uso, estabelecendo a base conceptual e funcional do sistema.

O **Capítulo 3 — Arquitetura e Design do Software utilizando LLM** descreve a arquitetura global do sistema, com operação local por loja e consolidação centralizada. São apresentados os principais artefactos de design, nomeadamente o diagrama de classes, os diagramas de sequência, o diagrama de componentes, a definição da *stack* tecnológica, a especificação da API REST, os esboços das interfaces, o modelo de persistência de dados e as decisões arquiteturais tomadas. O capítulo inclui ainda a análise da utilização de LLM no apoio ao design arquitetural e à revisão do modelo de domínio.

O **Capítulo 4 — Implementação e Desenvolvimento Assistido por LLM** descreve a concretização técnica do sistema, incluindo a configuração do ambiente de desenvolvimento, a implementação modular do *backend* e do *frontend*, a utilização de LLM na geração e revisão de código, as adaptações realizadas durante a implementação, a revisão do código gerado e a deteção de *code smells*. São também discutidas as refatorizações efetuadas e a dívida técnica identificada.

O **Capítulo 5 — Verificação, Validação e Avaliação da Qualidade do Software** apresenta a estratégia de testes adotada, incluindo testes unitários, de integração, de sistema e de aceitação. São descritos o uso de LLM no apoio à criação de testes, a análise de cobertura de código com JaCoCo, os testes de mutação com PIT, a avaliação da qualidade segundo a norma ISO/IEC 25010 e a verificação da satisfação dos requisitos definidos no SRS.

Por fim, o **Capítulo 6 — Conclusões e Trabalho Futuro** sintetiza a apreciação crítica do trabalho realizado, identificando os principais pontos fortes, limitações e aprendizagens do projeto. São ainda apresentadas possíveis evoluções futuras para o sistema **Mini-Formiga**, incluindo melhorias de usabilidade, integração com fornecedores, expansão móvel e aprofundamento da conformidade fiscal.

2.9 Definição do Domínio do Sistema

O sistema a desenvolver insere-se no domínio da gestão operacional de cadeias de lojas de conveniência no setor do retalho. Neste contexto, as organizações operam múltiplos estabelecimentos físicos onde são realizadas atividades diárias como o registo de vendas, o controlo de inventário, a receção de mercadoria, a gestão de fornecedores e a análise do desempenho comercial.

O caso de estudo considerado corresponde à cadeia de lojas **Mini-Formiga**, propriedade do Sr. Formiga, que pretende modernizar e integrar os processos de gestão atualmente realizados de forma parcial ou com recurso a ferramentas dispersas. A operação da cadeia envolve diferentes intervenientes tais como, gestores, gerentes de loja, operadores de caixa e responsáveis

de armazém, que necessitam de aceder a informação consistente e atualizada sobre vendas, produtos, inventário e fornecedores.

Neste domínio, a correta gestão da informação assume um papel central para o funcionamento eficiente do negócio. Processos como o registo de transações no ponto de venda (PDV), o controlo de níveis de *stock*, a emissão de documentos fiscais e a reposição de produtos dependem da disponibilidade de um sistema fiável, rápido e capaz de operar em ambientes distribuídos entre múltiplas lojas.

Assim, o sistema Mini-Formiga tem como objetivo apoiar a gestão integrada da cadeia de lojas, proporcionando funcionalidades para registo de vendas, controlo de inventário, gestão de fornecedores, geração de relatórios e monitorização do desempenho do negócio. O sistema deverá ainda garantir a consistência dos dados entre as diferentes lojas e suportar mecanismos de sincronização entre os terminais locais e o servidor central.

2.10 Análise de Contexto e Restrições

O sistema Mini-Formiga será utilizado num contexto organizacional composto por uma cadeia de lojas de conveniência distribuídas geograficamente, operando de forma autónoma mas sob gestão centralizada pelo proprietário da cadeia.

Cada loja dispõe de um ou mais terminais de Ponto de Venda (PDV), utilizados para o registo diário de transações, bem como de estações de trabalho destinadas à gestão de inventário e operações administrativas. Estes sistemas comunicam com um servidor central responsável pela consolidação de dados e geração de relatórios globais do negócio.

Uma característica relevante do contexto operacional é a possibilidade de falhas temporárias de conectividade à internet nas lojas. Por esse motivo, o sistema deverá suportar a operação local de forma autónoma, mantendo os registos de transações localmente e sincronizando-os posteriormente com o servidor central quando a ligação for restabelecida.

Para além das restrições tecnológicas, o sistema está igualmente sujeito a um conjunto de restrições legais e regulamentares aplicáveis ao setor do retalho em Portugal, nomeadamente no que respeita às normas de faturação da Autoridade Tributária e ao cumprimento do Regulamento Geral sobre a Proteção de Dados (RGPD). Estas restrições influenciam diretamente a forma como determinadas funcionalidades do sistema devem ser implementadas.

2.11 Identificação de *Stakeholders*

A identificação dos *stakeholders* do sistema Mini-Formiga foi realizada numa fase preliminar ao levantamento de requisitos, com o objetivo de mapear todos os intervenientes com interesse direto ou indireto no sistema a desenvolver. Para tal, a equipa partiu da análise do contexto

de negócio descrito pelo Sr. Formiga, identificando os diferentes perfis de utilizadores que interagem com os processos operacionais e administrativos da cadeia de lojas.

Para cada ator identificado, foram considerados três aspetos fundamentais: o papel que desempenha na organização, as responsabilidades que lhe estão associadas e as necessidades que o sistema deverá satisfazer do seu ponto de vista. Este mapeamento permitiu delimitar o âmbito do sistema e assegurar que nenhum grupo de utilizadores relevante fica por representar no processo de levantamento de requisitos. A Tabela 2.1 apresenta os *stakeholders* identificados e a sua caracterização.

Ator	Papel	Necessidades
Sr. Formiga	Proprietário e gestor da cadeia	Necessita de uma visão global do negócio, com acesso a relatórios consolidados e análise de rentabilidade por loja.
Gerente de Loja	Responsável pela gestão operacional da loja	Necessita de funcionalidades para a gestão diária de <i>stocks</i> , vendas e fecho de caixa, bem como acesso a relatórios específicos da sua loja.
Funcionário	Operador de atendimento e registo de vendas	Necessita de uma interface de Ponto de Venda (PDV) simples, rápida e fiável para o registo eficiente de transações.
Resp. Armazém	Responsável pelo controlo de inventário e receção de mercadoria	Necessita de mecanismos para registo de entregas, gestão de inventário e emissão de alertas de reposição.
Fornecedores	Entidades externas de abastecimento	Necessitam de receber encomendas e comunicar catálogos e condições comerciais atualizadas.
Contabilista	Responsável financeiro externo	Necessita de acesso a dados financeiros exportáveis para efeitos de processamento contabilístico.

Tabela 2.1: Identificação dos *Stakeholders* do Mini-Formiga

2.12 Método de Levantamento de Requisitos

O levantamento de requisitos do sistema Mini-Formiga foi realizado recorrendo exclusivamente ao método de entrevistas aos *stakeholders* relevantes. Esta opção metodológica foi tomada de forma deliberada, tendo em conta a natureza e o contexto de utilização do sistema.

De facto, a plataforma destina-se maioritariamente a ser utilizada por funcionários internos da organização. Neste sentido, os principais utilizadores do sistema são bem identificáveis, acessíveis e possuem conhecimento direto sobre os processos operacionais que o sistema deverá suportar.

Assim, as entrevistas revelaram-se o método mais adequado, uma vez que permitem:

- obter informação detalhada e contextualizada sobre os processos de negócio;
- esclarecer dúvidas em tempo real e explorar requisitos implícitos;
- adaptar dinamicamente as questões com base nas respostas obtidas;
- garantir maior precisão na identificação de necessidades específicas dos utilizadores internos.

Por outro lado, métodos como questionários foram considerados menos adequados neste contexto. Estes são tipicamente mais eficazes quando existe um grande número de utilizadores heterogéneos ou geograficamente dispersos, o que não se verifica neste caso. Adicionalmente, os questionários tendem a limitar a profundidade das respostas e dificultam a clarificação de requisitos ambíguos, o que poderia comprometer a qualidade do levantamento.

Desta forma, a utilização exclusiva de entrevistas permitiu assegurar um levantamento de requisitos mais rico, preciso e alinhado com as necessidades reais dos utilizadores do sistema.

2.13 Descrição Geral dos Requisitos Levantados

A partir da aplicação do método de levantamento descrito anteriormente, foi possível identificar e consolidar um conjunto de requisitos funcionais e não funcionais que refletem as necessidades operacionais do sistema Mini-Formiga.

A análise das entrevistas permitiu estruturar os requisitos de forma organizada, agrupando-os em módulos funcionais coerentes, cada um correspondente a uma área específica de responsabilidade dentro do sistema. Esta abordagem modular facilita não só a compreensão global do sistema, mas também o seu desenvolvimento e futura manutenção.

Os requisitos identificados evidenciam uma forte orientação para o suporte às operações internas das lojas, com especial foco na eficiência, fiabilidade e controlo dos processos.

Deste processo emergiram sete módulos funcionais principais que estruturam o sistema:

- **Autenticação e Gestão de Utilizadores:** controlo de acessos por perfil, autenticação segura e registo de auditoria.
- **Gestão de Produtos e Catálogo:** registo e categorização de produtos e associação a fornecedores.
- **Ponto de Venda (PDV):** registo de transações, múltiplos meios de pagamento, emissão de recibos e faturas, operação *offline*.

- **Gestão de Stock e Inventário:** controlo em tempo real, alertas de stock mínimo, ajustes manuais e inventários físicos.
- **Gestão de Fornecedores e Encomendas:** catálogo de fornecedores, sugestões automáticas de encomenda e histórico de encomendas.
- **Relatórios e *Dashboard*:** indicadores consolidados, relatórios de vendas, stock e rentabilidade, exportação em PDF e CSV.
- **Sincronização e Consolidação:** sincronização automática diária entre as lojas e o servidor central, com deteção de conflitos.

2.14 User Stories

As *User Stories* foram elaboradas com base nas entrevistas simuladas realizadas aos *stakeholders*, cujas transcrições se encontram no Anexo 1. Estas encontram-se organizadas por perfil de utilizador e seguem o formato padrão: “Como [papel], quero [funcionalidade], para que [benefício].”

A cada *User Story* foi atribuído um identificador único, de forma a assegurar a sua rastreabilidade, bem como uma prioridade definida de acordo com o método *MoSCoW*. Esta classificação permitiu distinguir funcionalidades essenciais para o funcionamento inicial do sistema de funcionalidades importantes, mas passíveis de implementação posterior.

Para além da formulação das *User Stories*, foram definidos critérios de verificação para cada uma delas. Estes critérios descrevem, sob a forma de teste ou comportamento observável, a condição mínima que deve ser satisfeita para considerar a respetiva *User Story* cumprida. A sua utilização permite reduzir ambiguidades, facilitar a validação posterior do sistema e estabelecer uma ligação direta entre as necessidades expressas pelos *stakeholders*, os requisitos especificados e os testes realizados na fase de verificação.

As *User Stories* do Gestor da Cadeia encontram-se na Tabela 2.2, com os respetivos critérios de verificação na Tabela 2.3; as do Gerente de Loja encontram-se na Tabela 2.4, com os critérios na Tabela 2.5; as do Funcionário encontram-se na Tabela 2.6, com os critérios na Tabela 2.7; e as do Responsável de Armazém encontram-se na Tabela 2.8, com os critérios na Tabela 2.9.

Gestor da Cadeia (Sr. Formiga)

Para gerar as *User Stories* relativas à gestão da cadeia, a equipa baseou-se na entrevista realizada ao Sr. Formiga (Entrevista 1, Anexo 1).

A *User Story* **US-01** foi definida com base na passagem:

“Quero uma página com as vendas de cada loja, a margem de lucro por produto, e perceber quais os estabelecimentos mais rentáveis. Idealmente com gráficos e filtros por período.”

estipulando assim a *User Story*, sendo posteriormente atribuída uma prioridade de *Must Have* pela equipa, segundo o *standard* MoSCoW.

Já a *User Story* **US-02** foi definida com base na passagem:

“E que me avise automaticamente quando o *stock* de alguma loja estiver a baixar — já perdi vendas por falta de produto e só me apercebi tarde demais.”

dando origem à respetiva *User Story*, à qual a equipa atribuiu igualmente uma prioridade de *Must Have* no *standard* MoSCoW.

As restantes *User Stories* referentes ao Gestor da Cadeia foram definidas do mesmo modo, com base nesta mesma entrevista, e encontram-se na Tabela 2.2.

ID	User Story	Prioridade
US-01	Como gestor da cadeia, quero uma página com as vendas de cada loja, a margem de lucro por produto e os estabelecimentos mais rentáveis, com gráficos e filtros por período, para monitorizar o desempenho global da cadeia.	Must Have
US-02	Como gestor da cadeia, quero receber alertas automáticos quando o <i>stock</i> de qualquer loja estiver a baixar para níveis críticos, para evitar perdas de vendas por rutura de produto.	Must Have
US-03	Como gestor da cadeia, quero gerar relatórios de margens de lucro por produto e por loja para um intervalo temporal selecionado, com opção de exportação, para apoiar decisões estratégicas.	Should Have
US-04	Como gestor da cadeia, quero registar fornecedores e criar encomendas consolidadas para múltiplas lojas, para centralizar e otimizar o processo de aprovisionamento.	Should Have
US-05	Como gestor da cadeia, quero aceder aos principais indicadores de desempenho a partir de um dispositivo móvel, de forma responsiva, para consultar informação em qualquer lugar.	Could Have
US-06	Como gestor da cadeia, quero exportar dados financeiros mensais em formato CSV ou XLSX, para facilitar a integração com ferramentas externas de análise.	Could Have

Tabela 2.2: *User Stories* do Gestor da Cadeia

CrITÉRIOS de Verificação das *User Stories*

ID	CrITÉrio de Verificação
US-01	Teste: aceder ao <i>dashboard</i> com dados registados de pelo menos 3 lojas e verificar que os dados de cada loja aparecem individualizados, que existem gráficos visíveis e que o filtro por período altera corretamente os resultados apresentados.
US-02	Teste: reduzir manualmente o <i>stock</i> de um produto de uma loja para valor igual ou inferior ao mínimo configurado e verificar que é gerado automaticamente um alerta que identifica a loja e o produto afetados.
US-03	Teste: selecionar uma loja e um intervalo temporal e verificar que o relatório gerado apresenta margens de lucro por produto, bem como opção de exportação funcional.
US-04	Teste: registar um novo fornecedor e criar uma encomenda consolidada para duas lojas distintas, verificando que a encomenda fica corretamente associada ao fornecedor e ao histórico.
US-05	Teste: aceder ao sistema através de um dispositivo móvel e verificar que os principais indicadores são apresentados de forma responsiva, sem necessidade de deslocamento horizontal.
US-06	Teste: solicitar a exportação dos dados financeiros de um mês e verificar que o ficheiro gerado abre corretamente em CSV/XLSX e contém os campos esperados.

Tabela 2.3: CrITÉRIOS de Verificação das *User Stories* do Gestor da Cadeia

Gerente de Loja

Para gerar as *User Stories* relativas à gestão da loja, a equipa baseou-se na entrevista realizada ao Gerente de Loja (Entrevista 2, Anexo 1).

A *User Story* **US-07** foi definida com base na passagem:

“O ideal era que, quando a mercadoria chegasse, se registasse diretamente no sistema e o *stock* atualizasse de forma automática.”

sendo-lhe atribuída uma prioridade de *Must Have*, dado que a correta atualização do inventário é uma condição essencial para o funcionamento do sistema.

A *User Story* **US-09** foi definida com base na passagem:

“Se houvesse uma totalização automática das vendas do dia, poupava-me tempo e havia mais segurança nos números.”

à qual a equipa atribuiu também uma prioridade de *Must Have*, uma vez que o fecho de caixa diário é uma operação crítica para o controlo financeiro da loja.

As restantes *User Stories* referentes ao Gerente de Loja foram definidas do mesmo modo, com base nesta mesma entrevista, e encontram-se na Tabela 2.4.

ID	User Story	Prioridade
US-07	Como gerente de loja, quero registar a receção de mercadoria e atualizar automaticamente o stock, para manter o inventário correto.	Must Have
US-08	Como gerente de loja, quero ver o stock atual de todos os produtos, para saber o que precisa de reposição.	Must Have
US-09	Como gerente de loja, quero registar o fecho de caixa diário com totalização automática, para garantir o controlo financeiro.	Must Have
US-10	Como gerente de loja, quero consultar relatórios de vendas por produto e período, para identificar padrões operacionais.	Should Have
US-11	Como gerente de loja, quero gerir os dados dos funcionários e os seus acessos ao sistema, para controlar quem opera o PDV.	Should Have
US-12	Como gerente de loja, quero criar ajustes manuais de inventário (quebras, desperdício), para manter o stock fidedigno.	Should Have

Tabela 2.4: *User Stories* do Gerente de Loja

Critérios de Verificação das *User Stories*

ID	Critério de Verificação
US-07	Teste: registrar a recepção de uma encomenda com vários produtos e verificar que o <i>stock</i> de cada produto é atualizado automaticamente após a confirmação, refletindo as quantidades recebidas.
US-08	Teste: aceder à vista de inventário e verificar que todos os produtos da loja são listados com as respetivas quantidades em <i>stock</i> , com indicação visual dos artigos abaixo do nível mínimo.
US-09	Teste: realizar o fecho de caixa no final de um dia com vendas registadas e verificar que o sistema totaliza automaticamente os valores, gerando um resumo financeiro sem necessidade de cálculo manual.
US-10	Teste: selecionar um produto, um turno e um intervalo de datas e verificar que o relatório gerado apresenta os dados de vendas correspondentes, permitindo identificar variações por período e por turno.
US-11	Teste: criar um novo funcionário, atribuir-lhe um perfil de acesso restrito ao PDV e verificar que o utilizador criado consegue autenticar-se apenas nas funcionalidades permitidas, sendo bloqueado nas restantes.
US-12	Teste: registrar um ajuste manual de inventário para um produto (quebra ou desperdício), indicando a quantidade e o motivo, e verificar que o <i>stock</i> é atualizado de forma correta e que o ajuste fica registado no histórico.

Tabela 2.5: Critérios de Verificação das *User Stories* do Gerente de Loja

Funcionário

Para gerar as *User Stories* relativas ao operador de PDV, a equipa baseou-se na entrevista realizada ao Funcionário (Entrevista 3, Anexo 1).

A *User Story* **US-13** foi definida com base na passagem:

“Que seja rápida e fácil de usar. Não tenho muito tempo para andar à procura de botões. Se conseguir registar um produto com um código de barras ou com uma pesquisa rápida, é o ideal. E que não bloqueie — nada pior do que o sistema travar quando há clientes à espera.”

sendo-lhe atribuída uma prioridade de *Must Have*, pois a eficiência do atendimento no ponto de venda é central para a operação diária da loja.

A *User Story* **US-18** foi definida com base na passagem do Gerente de Loja (Entrevista 2, Anexo 1), corroborada pelo próprio funcionário:

“O sistema tem mesmo de funcionar sem ligação à internet — isso é indispensável.”

à qual a equipa atribuiu uma prioridade de *Must Have*, dado que uma falha de rede não pode comprometer o registo de vendas.

As restantes *User Stories* referentes ao Funcionário foram definidas do mesmo modo, com base nesta mesma entrevista, e encontram-se na Tabela 2.6.

ID	User Story	Prioridade
US-13	Como funcionário, quero registar vendas de forma rápida e intuitiva, para agilizar o atendimento aos clientes.	Must Have
US-14	Como funcionário, quero processar pagamentos em numerário, cartão e MB Way, para atender clientes com diferentes meios de pagamento.	Must Have
US-15	Como funcionário, quero emitir recibos e faturas simplificadas, para satisfazer a necessidade dos clientes.	Must Have
US-16	Como funcionário, quero anular linhas de venda antes de finalizar a transação, para corrigir erros de registo.	Must Have
US-17	Como funcionário, quero processar devoluções de produtos, para resolver reclamações de clientes.	Should Have
US-18	Como funcionário, quero que o sistema funcione sem internet, para não interromper vendas em caso de falha de rede.	Must Have

Tabela 2.6: *User Stories* do Funcionário

Critérios de Verificação das *User Stories*

ID	Critério de Verificação
US-13	Teste: registrar uma venda com pelo menos três produtos, utilizando leitura de código de barras e pesquisa manual, e verificar que todos os artigos são adicionados corretamente sem atrasos perceptíveis ou bloqueios do sistema.
US-14	Teste: processar três transações distintas — uma em numerário, uma por cartão e uma por MB Way — e verificar que cada método é aceite corretamente e que o sistema regista o meio de pagamento utilizado em cada venda.
US-15	Teste: finalizar uma venda e solicitar a emissão de recibo e de fatura simplificada, verificando que ambos os documentos são gerados com os dados corretos da transação e em conformidade com os requisitos legais aplicáveis.
US-16	Teste: iniciar uma venda, adicionar vários produtos e anular uma linha antes de finalizar a transação, verificando que o artigo removido deixa de constar no total e que a operação não afeta os restantes itens registados.
US-17	Teste: registrar a devolução de um produto de uma venda anterior e verificar que o valor é corretamente creditado, que o <i>stock</i> é atualizado e que a devolução fica registada no histórico de transações.
US-18	Teste: desativar a ligação à internet durante uma sessão ativa e verificar que o sistema continua a permitir o registo de vendas e o processamento de pagamentos sem interrupções, sincronizando os dados quando a ligação for restabelecida.

Tabela 2.7: Critérios de Verificação das *User Stories* do Funcionário

Responsável de Armazém

Para gerar as *User Stories* relativas ao controlo de armazém, a equipa baseou-se na entrevista realizada ao Responsável de Armazém (Entrevista 4, Anexo 1).

A *User Story* **US-19** foi definida com base na passagem:

“Um sistema onde eu pudesse registar as entradas diretamente, associando à guia de remessa, seria uma enorme ajuda para todos.”

sendo-lhe atribuída uma prioridade de *Must Have*, uma vez que o registo de entradas de mercadoria é o ponto de partida para a fiabilidade de todo o inventário.

A *User Story* **US-20** foi definida com base na passagem:

“Precisava de poder aceder ao inventário onde pudesse registar as contagens e o sistema identificasse automaticamente as discrepâncias face ao *stock* registado.”

à qual a equipa atribuiu uma prioridade de *Should Have*, por se tratar de uma funcionalidade importante para a integridade do *stock*, mas não bloqueante para o arranque do sistema.

As restantes *User Stories* referentes ao Responsável de Armazém foram definidas do mesmo modo, com base nesta mesma entrevista, e encontram-se na Tabela 2.8.

ID	User Story	Prioridade
US-19	Como responsável de armazém, quero registar a entrada de mercadoria com referência a guias de remessa, para controlar o recebido vs. encomendado.	Must Have
US-20	Como responsável de armazém, quero realizar contagens de inventário físico e registar discrepâncias, para manter o stock real.	Should Have
US-21	Como responsável de armazém, quero consultar a localização de produtos no armazém, para agilizar a reposição nas lojas.	Could Have

Tabela 2.8: *User Stories* do Responsável de Armazém

Critérios de Verificação das *User Stories*

ID	Critério de Verificação
US-19	Teste: registar a entrada de uma encomenda associando-a à respetiva guia de remessa e verificar que o sistema regista corretamente as quantidades recebidas, as associa ao documento de origem e atualiza o <i>stock</i> de forma automática.
US-20	Teste: realizar uma contagem física de um conjunto de produtos e introduzir os valores no sistema, verificando que são identificadas automaticamente as discrepâncias face ao <i>stock</i> registado e que estas ficam disponíveis para revisão ou correção.
US-21	Teste: pesquisar um produto no sistema e verificar que é apresentada a sua localização no armazém, permitindo ao responsável identificar rapidamente onde se encontra o artigo sem necessidade de consulta manual.

Tabela 2.9: Critérios de Verificação das *User Stories* do Responsável de Armazém

2.15 Especificação dos Requisitos Levantados

Com base nas *User Stories* identificadas na secção anterior, a equipa procedeu à especificação dos requisitos do sistema, organizados em três categorias: requisitos funcionais, requisitos não funcionais e requisitos de domínio. Esta distinção segue o *standard* IEEE 830/29148 para Especificação de Requisitos de Software (SRS), segundo o qual os requisitos funcionais descrevem comportamentos, ações e funcionalidades que o sistema deve executar, enquanto os requisitos não funcionais definem restrições e atributos de qualidade que condicionam o modo como o sistema deve operar. Por sua vez, os requisitos de domínio decorrem de regras de negócio específicas do setor de atividade e de obrigações legais ou regulamentares que o sistema deve respeitar, influenciando diretamente o seu comportamento ou a forma como determinadas funcionalidades devem ser implementadas. A rastreabilidade de cada requisito às *User Stories* que lhe deram origem encontra-se indicada na coluna “Origem”.

Requisitos Funcionais

Os requisitos funcionais especificam as capacidades operacionais que o sistema Mini-Formiga deve providenciar. Para ilustrar o processo de classificação utilizado pela equipa, apresentam-se, de seguida dois exemplos.

O requisito **RF-01** foi classificado como funcional porque descreve uma ação concreta que o sistema deve executar: apresentar ao utilizador um conjunto de indicadores de desempenho de forma visual e interativa. Trata-se de um comportamento observável e testável, diretamente derivado das *User Stories* **US-01** e **US-03**.

O requisito **RF-05** foi igualmente classificado como funcional porque especifica uma operação que o sistema deve realizar em resposta a um evento: detetar que o nível de *stock* de um produto atingiu o mínimo definido e emitir um alerta automático ao utilizador. Este comportamento é verificável através de critérios de aceitação mensuráveis, derivado das *User Stories* **US-02** e **US-08**.

Os restantes requisitos funcionais foram classificados e especificados segundo o mesmo método, encontrando-se na Tabela 2.10.

ID	Descrição	Origem	Prioridade
RF-01	O sistema deve disponibilizar um <i>dashboard</i> que apresente indicadores consolidados de vendas, rentabilidade e <i>stock</i> por loja e por período, com suporte a filtros e representação gráfica.	US-01, US-03	Must Have

(continua na página seguinte)

ID	Descrição	Origem	Prioridade
RF-02	O sistema deve permitir ao gestor gerar e exportar relatórios de rentabilidade por loja e por período, nos formatos PDF e CSV.	US-03, US-06	Must Have
RF-03	O sistema deve permitir ao gestor registrar e gerir fornecedores, incluindo o catálogo de produtos e condições associadas, e criar encomendas consolidadas para múltiplas lojas.	US-04	Should Have
RF-04	O sistema deve disponibilizar uma interface acessível a partir de dispositivos móveis, que permita a consulta de indicadores de desempenho e alertas.	US-05	Could Have
RF-05	O sistema deve monitorizar os níveis de <i>stock</i> em tempo real e detetar que o nível de <i>stock</i> de um produto atingiu ou ficou abaixo do mínimo definido, emitindo um alerta automático ao utilizador.	US-02, US-08	Must Have
RF-06	O sistema deve permitir ao gerente registrar a receção de mercadoria, associando-a a uma guia de remessa, e atualizar automaticamente o <i>stock</i> em resultado dessa receção.	US-07, US-19	Must Have
RF-07	O sistema deve apresentar ao gerente a listagem atualizada do <i>stock</i> de todos os produtos, com indicação dos que se encontram abaixo do nível mínimo.	US-08	Must Have
RF-08	O sistema deve registar automaticamente o fecho de caixa diário, totalizando as transações do dia e apresentando o resumo ao gerente para conferência.	US-09	Must Have
RF-09	O sistema deve permitir ao gerente consultar relatórios de vendas filtráveis por produto, turno e período temporal.	US-10	Should Have
RF-10	O sistema deve permitir ao gerente criar, editar e desativar contas de utilizadores, definindo o perfil de acesso de cada funcionário.	US-11	Should Have
RF-11	O sistema deve permitir ao gerente registar ajustes manuais de inventário, indicando a quantidade, o motivo (quebra, desperdício, correção) e o utilizador responsável.	US-12	Should Have

(continua na página seguinte)

ID	Descrição	Origem	Prioridade
RF-12	O sistema deve disponibilizar um terminal de Ponto de Venda (PDV) que permita registrar produtos por leitura de código de barras ou por pesquisa, e finalizar transações de forma rápida.	US-13	Must Have
RF-13	O sistema deve suportar o processamento de pagamentos em numerário, cartão bancário e MB Way, registrando o meio de pagamento utilizado em cada transação.	US-14	Must Have
RF-14	O sistema deve emitir automaticamente recibos e faturas simplificadas no momento do fecho de cada transação, aplicando as regras de faturação definidas nos requisitos de domínio.	US-15	Must Have
RF-15	O sistema deve permitir ao operador remover ou corrigir linhas de artigo durante o processo de venda, antes da finalização da transação.	US-16	Must Have
RF-16	O sistema deve permitir o registo de devoluções de produtos, revertendo o respetivo impacto no <i>stock</i> e no histórico financeiro.	US-17	Should Have
RF-17	O sistema deve suportar o modo de operação <i>offline</i> , mantendo o registo local de transações e sincronizando automaticamente com o servidor central quando a ligação for restabelecida, aplicando uma estratégia de resolução de conflitos do tipo <i>last-write-wins</i> .	US-18	Must Have
RF-18	O sistema deve permitir ao responsável de armazém registar a localização física de cada produto, por corredor e prateleira.	US-21	Could Have
RF-19	O sistema deve permitir ao responsável de armazém realizar contagens de inventário físico, registar as quantidades apuradas e apresentar automaticamente as discrepâncias face ao <i>stock</i> registado no sistema.	US-20	Should Have

Tabela 2.10: Requisitos Funcionais do Sistema Mini-Formiga

Requisitos Não Funcionais

Os requisitos não funcionais definem os atributos de qualidade e as restrições que o sistema Mini-Formiga deve satisfazer, independentemente das funcionalidades que oferece. Para ilustrar o processo de classificação utilizado, apresentam-se dois exemplos.

O requisito **RNF-01** foi classificado como não funcional porque não descreve o que o sistema faz, mas sim uma restrição sobre como deve fazê-lo: o tempo de resposta máximo admissível nas operações de PDV. Trata-se de um atributo de desempenho que condiciona a qualidade da experiência do utilizador, derivado da necessidade expressa nas *User Stories* **US-13** e **US-18** de que o sistema seja rápido e fiável.

O requisito **RNF-04** foi igualmente classificado como não funcional porque especifica uma propriedade de segurança do sistema: a obrigatoriedade de autenticação por perfil antes do acesso a qualquer módulo. Não descreve uma funcionalidade em si, mas uma restrição transversal a toda a operação do sistema, derivada da *User Story* **US-11** e da necessidade de conformidade com o RGPD.

Os restantes requisitos não funcionais foram classificados e especificados segundo o mesmo método, encontrando-se na Tabela 2.11.

ID	Descrição	Origem	Prioridade
RNF-01	O sistema deve responder a qualquer operação de registo de venda no PDV em menos de 2 segundos, em condições normais de operação.	US-13, US-18	Must Have
RNF-02	O sistema deve estar disponível para operação local em cada loja durante pelo menos 99,5% do tempo, medido mensalmente, excluindo períodos de manutenção planeada previamente comunicados, independentemente do estado da ligação à rede.	US-18	Must Have
RNF-03	A sincronização automática entre os terminais de loja e o servidor central deve ocorrer no final de cada dia, com deteção e registo de eventuais conflitos de dados.	US-01, US-03	Must Have
RNF-04	O acesso a qualquer módulo do sistema deve exigir autenticação prévia por credenciais individuais, com controlo de permissões definido por perfil de utilizador.	US-11	Must Have
RNF-05	Todas as operações financeiras realizadas no sistema devem ser registadas num <i>log</i> de auditoria imutável, com identificação do utilizador, data, hora e tipo de operação.	US-09, US-11	Must Have
RNF-06	A comunicação entre os terminais de loja e o servidor central deve ser cifrada mediante protocolos seguros (TLS 1.2 ou superior), assegurando a confidencialidade e integridade dos dados transmitidos.	US-11	Must Have

(continua na página seguinte)

ID	Descrição	Origem	Prioridade
RNF-07	O sistema deve garantir conformidade com o Regulamento Geral sobre a Proteção de Dados (RGPD), nomeadamente no que respeita ao tratamento, armazenamento e eliminação de dados pessoais de clientes e colaboradores.	US-11, US-15	Must Have
RNF-08	O sistema deve garantir que os documentos fiscais emitidos cumprem integralmente as regras de faturação definidas nos requisitos de domínio e na legislação aplicável.	US-15	Must Have
RNF-09	O sistema deve ser escalável, permitindo a integração de novas lojas sem alterações à arquitetura base nem degradação de desempenho.	US-01, US-03	Should Have
RNF-10	A interface do PDV deve ser operável por um funcionário sem formação técnica prévia, devendo este conseguir executar, sem assistência, as operações de registo de venda, processamento de pagamento e emissão de recibo após uma sessão de treino de 30 minutos.	US-13	Must Have
RNF-11	Os ficheiros exportados para contabilidade devem seguir uma estrutura de campos normalizada (data, descrição, valor, IVA, loja), compatível com as ferramentas de contabilidade habitualmente utilizadas pelo contabilista externo (CSV e XLSX).	US-06	Should Have

Tabela 2.11: Requisitos Não Funcionais do Sistema Mini-Formiga

Requisitos de Domínio

Os requisitos de domínio decorrem de obrigações legais e regras de negócio específicas do setor do retalho em Portugal que o sistema Mini-Formiga deve respeitar.

ID	Descrição	Fonte
RD-01	Cada produto deve ter obrigatoriamente uma taxa de IVA associada, de acordo com as taxas em vigor em Portugal: 6% (taxa reduzida), 13% (taxa intermédia) ou 23% (taxa normal).	Código do IVA
RD-02	O sistema deve emitir fatura simplificada para transações até 1 000€ com consumidores finais, e fatura completa quando o cliente solicite NIF ou o valor exceda esse limiar.	Art.º 40.º CIVA
RD-03	Todos os documentos fiscais emitidos devem obedecer a numeração sequencial, única e ininterrupta por série.	AT
RD-04	O sistema não deve permitir o registo de uma venda que resulte em <i>stock</i> negativo, bloqueando a transação e alertando o operador.	Regra de negócio
RD-05	As lojas operam de segunda-feira a sábado, entre as 07h30 e as 20h00. Transações registadas fora deste horário devem ser sinalizadas para auditoria.	Regra de negócio
RD-06	Encomendas submetidas fora do horário de funcionamento do armazém do fornecedor (segunda a sexta-feira, 08h00–18h00) apenas são processadas no início do próximo dia útil. O sistema deve informar o utilizador da data esperada de processamento no momento da submissão.	Regra de negócio
RD-07	O sistema deve garantir conformidade com o RGPD, recolhendo apenas os dados pessoais de clientes estritamente necessários para a emissão de fatura.	Art.º 5.º RGPD

Tabela 2.12: Requisitos de Domínio do Sistema Mini-Formiga

2.16 Validação dos Requisitos Levantados

A validação dos requisitos foi realizada em três fases complementares. Numa primeira fase, os requisitos foram revistos internamente pela equipa, verificando a sua completude, consistência e ausência de ambiguidades, com base nas *User Stories* e nas entrevistas simuladas com os *stakeholders* (Anexo 1).

Numa segunda fase, os resultados produzidos, incluindo as *User Stories*, os requisitos funcionais (Tabela 2.10), os requisitos não funcionais (Tabela 2.11) e os requisitos de domínio (Tabela 2.12), foram submetidos a um processo de validação cruzada com um segundo agente LLM independente, com o objetivo de identificar eventuais inconsistências, lacunas ou imprecisões que pudessem ter escapado à análise inicial.

Numa terceira fase, foi realizada uma sessão de validação com o cliente, o Sr. Formiga, na qual

foram apresentados e discutidos os principais resultados obtidos. Esta sessão permitiu validar a adequação dos requisitos às necessidades do negócio, bem como clarificar aspetos ambíguos e alinhar o comportamento esperado do sistema com a perspectiva do utilizador final. A ata desta reunião encontra-se disponível no Anexo 6.3.

Tendo em conta o *standard* IEEE 830/29148 para SRS, os principais critérios de validação aplicados foram os seguintes:

- **Compleitude:** todos os objetivos definidos pelo Sr. Formiga têm correspondência em pelo menos um requisito funcional.
- **Consistência:** não existem requisitos contraditórios entre si ou com as restrições identificadas.
- **Rastreabilidade:** todos os requisitos funcionais estão associados a pelo menos uma *User Story*, conforme indicado na coluna “Origem” das Tabelas 2.10 e 2.11.
- **Verificabilidade:** cada requisito possui um critério de aceitação mensurável ou um comportamento observável que permite a sua validação na fase de testes.
- **Viabilidade:** os requisitos são exequíveis dentro das restrições tecnológicas, orçamentais e temporais identificadas.

Os requisitos foram ainda classificados segundo o modelo de priorização **MoSCoW** (*Must Have, Should Have, Could Have, Won't Have*), garantindo que o âmbito do sistema está alinhado com as prioridades reais do negócio e que a implementação pode ser faseada de forma controlada. Os requisitos de domínio não estão sujeitos a priorização MoSCoW, uma vez que decorrem de obrigações legais ou regras de negócio incontornáveis, sendo todos de cumprimento obrigatório.

Refinação dos Requisitos Decorrente da Validação com o Cliente

A análise das observações produzidas pelo agente de validação, em conjunto com a revisão interna da equipa, serviu como base para a preparação da reunião de validação com o Sr. Formiga. Nessa reunião, os requisitos identificados foram discutidos com o cliente, permitindo confirmar necessidades, esclarecer ambiguidades e ajustar formulações que ainda podiam dar origem a interpretações diferentes.

As alterações registadas nesta secção refletem, por isso, não apenas a revisão técnica realizada pela equipa e pelo agente LLM independente, mas também o alinhamento obtido na reunião com o Sr. Formiga. Após a incorporação dos ajustes, os requisitos refinados foram novamente revistos e confirmados pela equipa, de forma a assegurar que mantinham consistência com as *User Stories*, os requisitos de domínio e os casos de uso especificados.

Este passo não resultou na criação de novos requisitos, mas sim na refinação dos requisitos já

identificados. O objetivo foi melhorar atributos de qualidade da especificação, nomeadamente a clareza, a verificabilidade, a consistência interna e a rastreabilidade entre requisitos, regras de domínio e casos de uso. A Tabela 2.13 apresenta os principais ajustes realizados.

ID	Aspeto Refinado	Formulação/Problema Inicial	Refinação Efetuada	Atributo Melhorado
RD-01, RD-02, RD-03 vs. RF-14 e RNF-08	Relação entre requisitos de domínio, funcionais e não funcionais	Existia sobreposição parcial entre requisitos de faturação, sem distinção clara entre obrigação legal e comportamento esperado do sistema.	Foi clarificado que os requisitos de domínio estabelecem a obrigação legal, enquanto RF-14 e RNF-08 especificam como o sistema deve satisfazer essa obrigação. Nenhum requisito foi eliminado, por terem natureza complementar.	Consistência e rastreabilidade
RF-05	Frequência e momento de emissão de alertas de <i>stock</i>	A expressão “monitorizar continuamente” era ambígua quanto à frequência real de verificação dos níveis de <i>stock</i> .	Foi clarificado que a monitorização ocorre em tempo real no contexto das operações que alteram o <i>stock</i> , sendo o alerta emitido quando uma venda ou ajuste de inventário coloca o produto num nível igual ou inferior ao mínimo configurado.	Clareza e verificabilidade
RNF-02	Período de referência da disponibilidade local	A disponibilidade mínima de 99,5% não especificava o intervalo temporal usado para o cálculo.	Foi acrescentado que a disponibilidade é medida mensalmente, excluindo períodos de manutenção planeada previamente comunicados.	Verificabilidade

(continua na página seguinte)

ID	Aspeto Refinado	Formulação/Problema Inicial	Refinação Efetuada	Atributo Melhorado
RF-17	Comportamento em caso de conflito na sincronização	O requisito previa sincronização com o servidor central quando a ligação fosse restabelecida, mas não definia a estratégia de resolução de conflitos.	Foi adicionada a referência explícita à estratégia <i>last-write-wins</i> , alinhando o requisito com o comportamento descrito no UC-13.	Consistência e rastreabilidade
RNF-10	Critério de aprendizagem da interface PDV	A formulação anterior referia uma curva de aprendizagem máxima de 30 minutos, mas não definia que operações o utilizador deveria conseguir executar após esse período.	Foi clarificado que o critério se aplica às operações essenciais do PDV: registo de venda, processamento de pagamento e emissão de recibo, executadas sem assistência por um funcionário sem formação técnica prévia após uma sessão de treino de 30 minutos.	Clareza e verificabilidade

Tabela 2.13: Refinação dos requisitos após validação com o Sr. Formiga e confirmação pela equipa

As restantes observações produzidas pelo agente foram analisadas e consideradas não aplicáveis ao âmbito atual do sistema, ou já se encontravam implicitamente resolvidas pela estrutura de rastreabilidade adotada.

2.17 Apresentação geral da especificação

O modelo de domínio do sistema Mini-Formiga, apresentado na Figura 2.2, representa a estrutura conceptual do sistema, identificando as entidades do domínio e os relacionamentos entre elas. A sua construção partiu da análise dos objetivos do negócio descritos acima e antecede formalmente o levantamento de requisitos, servindo de base à especificação detalhada das entidades e relacionamentos nas secções seguintes.

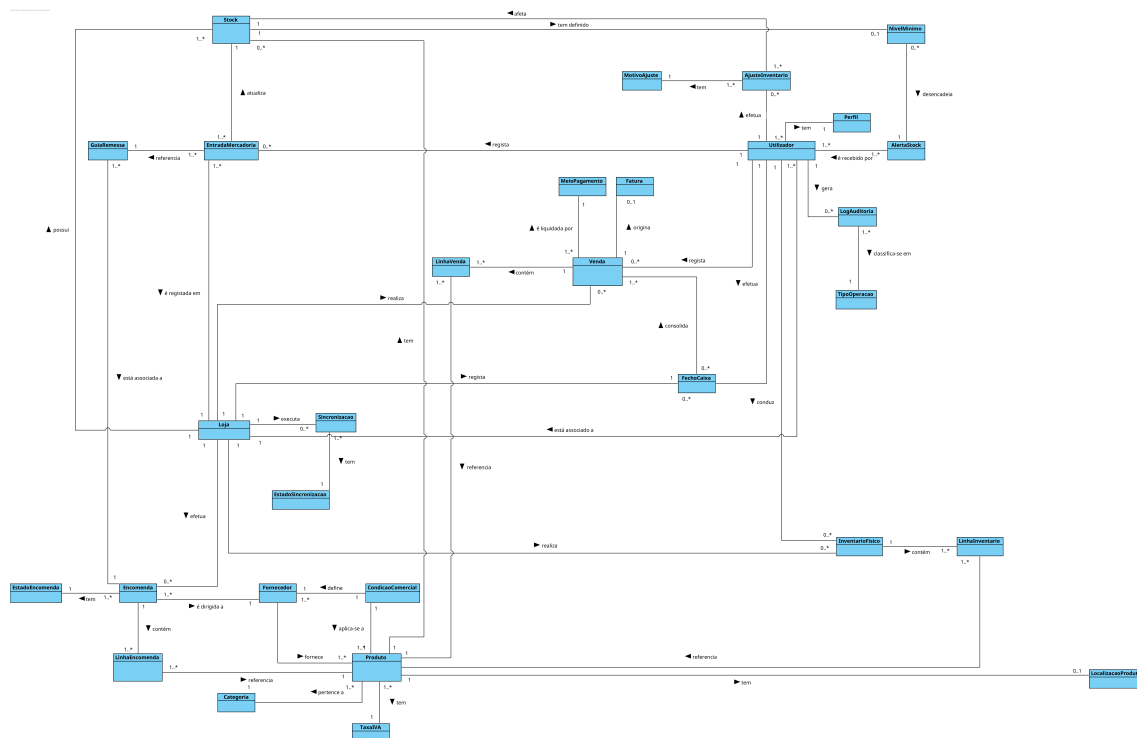


Figura 2.2: Modelo de Domínio do sistema Mini-Formiga

O modelo é composto por trinta entidades, organizadas em torno de quatro conceitos centrais: **Loja**, **Produto**, **Venda** e **Utilizador**. A **Loja** é o nó de maior centralidade estrutural, dado que as operações de venda, controlo de inventário, encomendas, sincronização e fecho de caixa convergem todas nela. O **Produto** é o nó de maior conectividade, sendo referenciado por praticamente todos os fluxos operacionais do sistema. A **Venda** agrega o núcleo transaccional do ponto de venda, articulando meios de pagamento, faturação e fecho de caixa. O **Utilizador** atravessa transversalmente todo o modelo, estando associado a todas as operações que requerem identificação do responsável para efeitos de controlo de acessos e auditoria.

A filosofia de modelação adotada privilegiou a granularidade e a rastreabilidade mantendo na medida do possível a simplicidade. Sempre que um conceito apresentava ciclo de vida próprio, podia variar de forma independente em diferentes contextos, ou era necessário para fins de auditoria, foi elevado a entidade autónoma em vez de ser reduzido a um atributo de uma entidade existente. Esta opção resulta num modelo mais verboso, mas garante que o sistema consegue responder a questões de negócio relevantes, como o histórico de alertas de stock, a rastreabilidade de ajustes de inventário ou o estado de cada encomenda ao longo do seu ciclo de vida, sem recorrer a informação inferida ou reconstruída.

2.17.1 Identificação e Caracterização das Entidades

Com base na análise dos requisitos funcionais, dos requisitos não funcionais e das *User Stories* levantadas, foram identificadas as entidades do domínio do sistema Mini-Formiga. Nesta fase

conceptual, não são considerados atributos — o foco incide exclusivamente sobre os conceitos do domínio e os seus relacionamentos. A decisão sobre como as entidades irão ser integradas no sistema é adiada para a próxima etapa do projeto.

Para ilustrar o processo de identificação seguido pela equipa, apresentam-se de seguida dois exemplos representativos.

A entidade **Stock** foi identificada a partir do requisito funcional RF-05, que estabelece que o sistema deve monitorizar continuamente os níveis de *stock* e emitir alertas automáticos ao gestor e ao gerente quando um produto atingir o nível mínimo configurado, e da *User Story* US-08, na qual o gerente de loja expressa a necessidade de ver o *stock* atual de todos os produtos para saber o que precisa de reposição. A análise destes artefactos tornou evidente a existência de um conceito autónomo que regista o nível de inventário de um produto numa loja específica, justificando a sua elevação a entidade própria.

A entidade **FechoCaixa** foi identificada a partir da *User Story* US-09, em que o gerente de loja solicita o registo do fecho de caixa diário com totalização automática para garantir o controlo financeiro, e do requisito funcional RF-08, que especifica que o sistema deve registar automaticamente o fecho de caixa diário, totalizando as transações do dia. A recorrência deste conceito nas entrevistas simuladas ao gerente confirmou que se trata de um conceito de domínio independente e não de um mero estado de uma venda.

As restantes entidades foram identificadas segundo a mesma metodologia e encontram-se na Tabela 2.14.

Entidade	Descrição	Origem
Loja	Estabelecimento físico da cadeia onde decorrem as operações comerciais.	RNF-09; US-01; Stakeholder: Sr. Formiga
Utilizador	Pessoa com acesso autenticado ao sistema.	RNF-04; US-11; Stakeholder: Gerente de Loja
Perfil	Conjunto de permissões que define o nível de acesso de um utilizador ao sistema (ex.: Gestor, Gerente, Funcionário, Responsável de Armazém).	RNF-04; RF-10; US-11
Produto	Artigo comercializado nas lojas, identificado de forma única no catálogo.	RF-12; RF-07; US-13; US-08
Categoria	Classificação temática que agrupa produtos com características semelhantes (ex.: bebidas, higiene, <i>snacks</i>).	RF-02; US-03; Stakeholder: Sr. Formiga
Fornecedor	Entidade externa responsável pelo abastecimento de produtos à cadeia.	RF-03; US-04; Stakeholder: Fornecedores

(continua na página seguinte)

Entidade	Descrição	Origem
Condicao Comercial	Condições acordadas entre a cadeia e um fornecedor para o abastecimento de um produto (ex.: preço, prazo de entrega, quantidade mínima).	RF-03; US-04; Stakeholder: Fornecedores
Stock	Registo do nível de inventário de um produto numa determinada loja.	RF-05; RF-07; US-08
NivelMinimo	Limiar de quantidade abaixo do qual um produto numa loja deve desencadear um alerta de reposição.	RF-05; US-02; US-08
AlertaStock	Notificação emitida automaticamente quando o <i>stock</i> de um produto atinge ou desce abaixo do nível mínimo.	RF-05; US-02; Stakeholder: Sr. Formiga
Localizacao Produto	Posição física de um produto no armazém, identificada por corredor e prateleira.	RF-18; US-21; Stakeholder: Resp. Armazém
Venda	Transação comercial realizada no PDV, correspondente a uma compra efetuada por um cliente.	RF-12; RF-08; US-13; US-09
LinhaVenda	Detalhe de uma venda, associando um produto a uma quantidade transacionada.	RF-12; RF-15; US-13; US-16
MeioPagamento	Forma utilizada para liquidar uma venda (ex.: numerário, cartão bancário, MB Way).	RF-13; US-14; Stakeholder: Funcionário
Fatura	Documento fiscal emitido no âmbito de uma venda, em conformidade com as normas da Autoridade Tributária.	RF-14; RNF-08; US-15
TaxaIVA	Taxa de imposto sobre o valor acrescentado aplicável a um produto.	RF-14; RNF-08; US-15
FechoCaixa	Registo do encerramento diário do terminal de caixa com totalização das transações do dia.	RF-08; US-09; Stakeholder: Gerente de Loja
Encomenda	Pedido formal de reposição de produtos dirigido a um fornecedor.	RF-03; US-04; US-19
LinhaEncomenda	Detalhe de uma encomenda, especificando o produto e a quantidade solicitada.	RF-03; US-04
Estado Encomenda	Situação em que se encontra uma encomenda ao longo do seu ciclo de vida (ex.: pendente, enviada, recebida, cancelada).	RF-03; US-04; Stakeholder: Fornecedores

(continua na página seguinte)

Entidade	Descrição	Origem
GuiaRemessa	Documento que acompanha a entrega física de mercadoria por parte do fornecedor.	RF-06; US-07; US-19; Stakeholder: Resp. Armazém
Entrada Mercadoria	Registo da receção efetiva de mercadoria numa loja, associada a uma guia de remessa.	RF-06; US-07; US-19
Ajuste Inventario	Correção manual ao <i>stock</i> de um produto, com indicação do motivo.	RF-11; US-12; Stakeholder: Gerente de Loja
MotivoAjuste	Razão que justifica um ajuste de inventário (ex.: quebra, desperdício, correção de erro).	RF-11; US-12
Inventario Fisico	Contagem física periódica dos produtos em armazém para verificação do <i>stock</i> registado.	RF-19; US-20; Stakeholder: Resp. Armazém
Linha Inventario	Detalhe de um inventário físico, com a contagem registada para cada produto e a discrepância face ao <i>stock</i> do sistema.	RF-19; US-20
LogAuditoria	Registo imutável de operações financeiras e sensíveis realizadas no sistema, com identificação do utilizador responsável.	RNF-05; US-11; Stakeholder: Contabilista
TipoOperacao	Classificação da operação registada no <i>log</i> de auditoria (ex.: venda, fecho de caixa, ajuste de <i>stock</i> , acesso).	RNF-05; US-09; US-11
Sincronizacao	Registo do processo de sincronização entre o terminal local de uma loja e o servidor central.	RNF-03; RF-17; US-18
Estado Sincronizacao	Situação em que se encontra uma sincronização (ex.: pendente, concluída, com conflitos).	RNF-03; RF-17
Devolucao	Registo de devolução de um produto por parte de um cliente, associada a uma venda original.	RF-16; US-17; Stakeholder: Funcionário

Tabela 2.14: Entidades do Modelo de Domínio do Mini-Formiga

Identificação e Caracterização dos Relacionamentos

Com base nas entidades identificadas, procedeu-se ao levantamento dos relacionamentos do domínio. A notação adotada é **(min, max)**, onde **0** indica participação opcional e **1** indica participação obrigatória; **N** representa cardinalidade ilimitada.

Para ilustrar o processo de identificação dos relacionamentos, apresentam-se de seguida dois exemplos representativos.

O relacionamento **Venda** — contém — **LinhaVenda** foi identificado a partir do requisito funcional RF-12, que especifica que o PDV deve registrar produtos por leitura de código de barras ou por pesquisa e finalizar transações de forma rápida, e da *User Story* US-16, que prevê a possibilidade de anular linhas de venda antes de finalizar a transação. A referência explícita a linhas de uma transação evidencia que uma *Venda* é composta por múltiplas *LinhasVenda*, definindo um relacionamento de composição com participação total e cardinalidade (1,1) do lado da *Venda* e (1,N) do lado da *LinhaVenda*.

O relacionamento **NivelMinimo** — desencadeia — **AlertaStock** foi identificado a partir do requisito funcional RF-05, que determina que o sistema deve monitorizar continuamente os níveis de *stock* e emitir alertas automáticos quando um produto atinge o valor mínimo configurado, e da *User Story* US-02, onde o Sr. Formiga solicita que o sistema avise automaticamente quando o *stock* de alguma loja estiver a baixar. A análise destes artefactos revela que o *AlertaStock* é consequência direta de um *NivelMinimo*, com participação opcional do lado do *NivelMinimo* — pois pode existir um nível mínimo configurado sem que nenhum alerta tenha ainda sido gerado — e participação total do lado do *AlertaStock*, que necessariamente resulta de um nível mínimo.

Os restantes relacionamentos foram identificados segundo a mesma metodologia e encontram-se na Tabela 6.1, presente no Anexo 3.

2.18 Casos de Uso

2.18.1 Diagrama de Use Cases

O diagrama de casos de uso da Figura 2.3 representa graficamente o âmbito funcional do sistema Mini-Formiga, identificando os atores que interagem com o sistema e as funcionalidades que cada um pode desencadear. O diagrama está organizado em torno de uma fronteira de sistema que delimita as 14 funcionalidades especificadas, com os atores humanos posicionados no exterior — Gestor da Cadeia, Gerente de Loja, Funcionário e Responsável de Armazém — e o ator sistema, que representa os processos automáticos desencadeados sem intervenção humana direta. As linhas de associação indicam quais os atores que participam em cada caso de uso, e as relações de *extend* assinalam funcionalidades que estendem o comportamento de outra em contextos específicos.

O diagrama foi construído a partir das *user stories* e dos requisitos funcionais identificados acima, seguindo um processo de três etapas. Em primeiro lugar, os requisitos funcionais foram agrupados por ator principal, identificando para cada perfil de utilizador o conjunto de funcionalidades que lhe competia desencadear. Em segundo lugar, as funcionalidades com o mesmo ator e a mesma intenção de negócio foram fundidas num único caso de uso, como sucedeu com o UC-06, resultante da fusão dos requisitos RF-01 e RF-02. Em terceiro lugar, as dependências entre casos de uso foram analisadas e representadas apenas quando uma funcionalidade estende o comportamento de outra em situações opcionais, evitando o uso de

include nos fluxos de interação.



Figura 2.3: Diagrama de Casos de Uso do sistema Mini-Formiga

2.18.2 Especificação de Use Cases

A especificação dos casos de uso foi elaborada com base nos requisitos funcionais, nas *user stories* e nos requisitos de domínio identificados na Secção 2. Para cada caso de uso foram definidos: o ator principal, uma descrição sumária, um cenário ilustrativo, as pré e pós-condições, o fluxo normal de interações, os fluxos alternativos ou de exceção relevantes, e o campo **origem**, que estabelece a rastreabilidade aos requisitos que lhe deram origem.

Para ilustrar o processo de derivação seguido, apresentam-se dois exemplos representativos antes da listagem completa.

O caso de uso **UC-02 Registrar Venda no PDV** foi derivado diretamente do requisito funcional **RF-12**, que especifica a necessidade de um terminal PDV com registo de produtos por código de barras ou pesquisa, e das *user stories* **US-13** e **US-14**, onde o Funcionário descreve a necessidade de um processo de venda rápido com suporte a múltiplos meios de pagamento. O requisito não funcional **RNF-01**, que impõe um tempo de resposta inferior a 2 segundos, condicionou a especificação do fluxo normal. A pós-condição de atualização automática do stock decorre do requisito **RF-12** em conjugação com o requisito de domínio **RD-04**, que proíbe a existência de stock negativo.

O caso de uso **UC-06 Consultar Indicadores e Gerar Relatórios** resulta da fusão de dois

casos de uso inicialmente separados, correspondentes aos requisitos **RF-01** e **RF-02**. A fusão foi justificada pelo facto de ambos partilharem o mesmo ator (Gestor da Cadeia), os mesmos dados de base e as mesmas *user stories* (**US-01** e **US-03**). Na contextualização do projeto, o Sr. Formiga descreve as duas necessidades como uma única intenção de negócio, visualizar e exportar, pelo que a exportação foi modelada como um fluxo alternativo da consulta e não como um caso de uso autónomo.

Os restantes casos de uso foram definidos seguindo o mesmo método, partindo sempre dos requisitos funcionais e das *user stories* correspondentes, e encontram-se especificados nas tabelas seguintes.

Autenticar no Sistema

Use Case	Autenticar no Sistema
Descrição	Permite a qualquer utilizador (Gestor, Gerente, Funcionário ou Responsável de Armazém) aceder ao sistema com credenciais individuais, obtendo o nível de acesso correspondente ao seu perfil.
Cenários	Gerente introduz username e password e acede ao painel da sua loja.
Pré-condição	Utilizador tem conta ativa no sistema.
Pós-condição	Sessão iniciada com permissões do perfil atribuído; acesso registado no log de auditoria.
Fluxo Normal	1. Utilizador introduz as suas credenciais (username e password). 2. Sistema valida as credenciais. 3. Sistema obtém o perfil e permissões associados ao utilizador. 4. Sistema regista o acesso no log de auditoria. 5. Sistema apresenta o painel correspondente ao perfil do utilizador.
Fluxo de Exceção	<i>[Credenciais inválidas] (passo 2)</i> 2.1 Sistema informa que as credenciais estão incorretas. 2.2 Sistema regista a tentativa falhada e bloqueia a conta após 5 falhas consecutivas.
Origem	RF-10; RNF-04; US-11

Tabela 2.15: UC-01 – Autenticar no Sistema

Registrar Venda no PDV

Use Case	Registrar Venda no PDV
Descrição	Permite ao Funcionário registrar uma transação de venda no terminal PDV, adicionando produtos por código de barras ou pesquisa, selecionando o meio de pagamento e finalizando a transação.
Cenários	Funcionário lê códigos de barras de três produtos, cliente paga com MB Way, sistema emite recibo.
Pré-condição	Funcionário autenticado; PDV em modo operacional (online ou offline); os produtos a registrar existem no catálogo com stock disponível.
Pós-condição	Venda registrada, stock atualizado, recibo emitido e transação incluída no fecho de caixa do dia.
Fluxo Normal	1. Funcionário inicia nova transação no PDV. 2. Funcionário registra produto por leitura de código de barras ou pesquisa. 3. Sistema adiciona linha de venda com produto, quantidade e preço unitário. 4. Funcionário repete passos 2–3 para todos os produtos. 5. Sistema apresenta total da venda com IVA discriminado. 6. Funcionário seleciona meio de pagamento. 7. Sistema processa pagamento e confirma transação. 8. Sistema atualiza stock dos produtos vendidos. 9. Sistema emite recibo. 10. Sistema registra a transação no log de auditoria.
Fluxo de Exceção (1)	<i>[Produto não encontrado pelo código de barras] (passo 2)</i> 2.1 Sistema informa que o produto não existe no catálogo. 2.2 Funcionário tenta nova leitura ou pesquisa manual.
Fluxo de Exceção (2)	<i>[Stock insuficiente para a quantidade pretendida] (passo 3)</i> 3.1 Sistema bloqueia a adição e informa o Funcionário da quantidade disponível.
Fluxo Alternativo	<i>[Remoção de linha de venda antes de finalizar] (passo 4)</i> 4.1 Funcionário seleciona linha a remover. 4.2 Sistema remove a linha e recalcula o total. 4.3 Regressa a 4.
Origem	RF-12; RF-13; RF-15; RNF-01; RNF-02; RD-04; US-13; US-14; US-16; US-18

Tabela 2.16: UC-02 – Registrar Venda no PDV

Emitir Fatura

Use Case	Emitir Fatura
Descrição	Permite ao Funcionário emitir fatura completa ou simplificada no fecho de uma venda, em conformidade com as normas da Autoridade Tributária portuguesa (Art.º 40.º CIVA).
Cenários	Cliente solicita fatura com NIF após pagamento; sistema emite fatura completa com número sequencial.
Pré-condição	Venda finalizada e pagamento confirmado.
Pós-condição	Fatura emitida com número sequencial único, associada à venda e registada no sistema.
Fluxo Normal	1. Funcionário indica que o cliente solicita fatura e introduz o NIF. 2. Sistema verifica se o valor da venda exige fatura completa ($> 1\,000\text{ €}$) ou simplificada. 3. Sistema gera número sequencial único para a série em curso. 4. Sistema emite fatura com IVA discriminado por taxa. 5. Sistema associa a fatura à venda e regista no log de auditoria.
Fluxo Alternativo	<i>[Venda $\leq 1\,000\text{ €}$ e cliente não fornece NIF] (passo 1)</i> 1.1 Sistema emite fatura simplificada sem NIF do cliente. 1.2 Regressa a 5.
Origem	RF-14; RNF-08; RD-01; RD-02; RD-03; US-15

Tabela 2.17: UC-03 – Emitir Fatura

Processar Devolução de Produto

Use Case	Processar Devolução de Produto
Descrição	Permite ao Funcionário registrar a devolução de um produto por parte de um cliente, revertendo o impacto financeiro no histórico de vendas e repondo o stock correspondente.
Cenários	Cliente devolve produto defeituoso; Funcionário regista devolução e stock é repostado.
Pré-condição	Funcionário autenticado; venda original registada no sistema; produto físico presente.
Pós-condição	Devolução registada, stock repostado e impacto financeiro revertido no histórico.
Fluxo Normal	1. Funcionário pesquisa e seleciona a venda original. 2. Funcionário indica o produto e a quantidade a devolver. 3. Sistema regista a devolução associada à venda original. 4. Sistema repõe o stock do produto na loja. 5. Sistema reverte o impacto financeiro e emite documento interno de devolução numerado no formato NC/<ano>/<sequência>. 6. Sistema regista a operação no log de auditoria.
Fluxo de Exceção (1)	<i>[Venda original não encontrada] (passo 1)</i> 1.1 Sistema informa impossibilidade de localizar a venda original. 1.2 Funcionário cancela a operação ou encaminha para o Gerente.
Origem	RF-16; US-17

Tabela 2.18: UC-04 – Processar Devolução de Produto

Registrar Fecho de Caixa Diário

Use Case	Registrar Fecho de Caixa Diário
Descrição	Permite ao Gerente de Loja encerrar o terminal de caixa no final do dia, com totalização automática das transações por meio de pagamento e apresentação do resumo para conferência.
Cenários	Gerente encerra caixa às 20h00; sistema totaliza vendas e apresenta resumo por meio de pagamento.
Pré-condição	Gerente autenticado; existem transações registradas no dia corrente.
Pós-condição	Fecho de caixa registado com totais confirmados; sincronização com o servidor central agendada; operação registada no log de auditoria.
Fluxo Normal	1. Gerente inicia o processo de fecho de caixa. 2. Sistema totaliza automaticamente as vendas do dia por meio de pagamento. 3. Sistema apresenta resumo ao Gerente: total numerário, cartão, MB Way e total geral. 4. Gerente confere os valores e confirma o fecho. 5. Sistema regista o fecho de caixa com os totais confirmados. 6. Sistema regista a operação no log de auditoria. 7. Sistema agenda a sincronização automática com o servidor central.
Fluxo Alternativo (1)	<i>[Gerente deteta discrepância nos valores contados] (passo 4)</i> 4.1 Gerente regista uma observação de discrepância. 4.2 Sistema associa a observação ao registo do fecho de caixa. 4.3 Regressa a 5.
Origem	RF-08; RNF-05; US-09

Tabela 2.19: UC-05 – Registrar Fecho de Caixa Diário

Consultar Indicadores e Gerar Relatórios

Use Case	Consultar Indicadores e Gerar Relatórios
Descrição	Permite ao Gestor da Cadeia visualizar indicadores consolidados de vendas, stock e rentabilidade de todas as lojas, com filtros por loja, período e categoria de produto, e exportar os dados em PDF ou CSV para apoio à decisão e comunicação com o contabilista.
Cenários	Sr. Formiga consulta as vendas do mês corrente de todas as lojas com representação gráfica e exporta o relatório para CSV para enviar ao contabilista.
Pré-condição	Gestor autenticado; dados de pelo menos um período sincronizados no servidor central.
Pós-condição	Dashboard apresentado com indicadores atualizados segundo os filtros selecionados; relatório exportado no formato selecionado quando solicitado.
Fluxo Normal	1. Gestor acede ao módulo de Dashboard. 2. Sistema apresenta indicadores consolidados do período corrente por defeito. 3. Gestor aplica filtros: loja, período e/ou categoria de produto. 4. Sistema atualiza a visualização com os dados filtrados e representação gráfica.
Fluxo de Exceção (1)	<i>[Dados insuficientes para o período ou loja selecionados] (passo 4)</i> 4.1 Sistema informa ausência de dados para os filtros aplicados e sugere ajuste.
Fluxo Alternativo (1)	<i>[Gestor pretende exportar os dados visualizados] (passo 4)</i> 4.1 Gestor seleciona formato de exportação (PDF ou CSV). 4.2 Sistema gera relatório com os dados filtrados, margens e estrutura normalizada. 4.3 Sistema disponibiliza o ficheiro para download.
Fluxo Alternativo (2)	<i>[Gestor acede via dispositivo móvel] (passo 1)</i> 1.1 Sistema apresenta versão responsiva do dashboard com indicadores principais. 1.2 Regressa a 3.
Origem	RF-01; RF-02; RNF-09; RNF-11; US-01; US-03; US-05; US-06

Tabela 2.20: UC-06 – Consultar Indicadores e Gerar Relatórios

Consultar Stock e Receber Alertas de Reposição

Use Case	Consultar Stock e Receber Alertas de Reposição
Descrição	Permite ao Gerente de Loja consultar os níveis de stock atuais de todos os produtos e receber alertas automáticos quando um produto atinge o nível mínimo configurado.
Cenários	Gerente consulta listagem de stock; sistema destaca 3 produtos abaixo do mínimo e emite alertas.
Pré-condição	Gerente autenticado; níveis mínimos de stock configurados para os produtos relevantes.
Pós-condição	Listagem de stock apresentada com produtos abaixo do mínimo destacados; alertas emitidos aos utilizadores com perfil relevante.
Fluxo Normal	1. Gerente acede ao módulo de gestão de stock da sua loja. 2. Sistema apresenta listagem atualizada de stock de todos os produtos. 3. Sistema destaca produtos com stock abaixo do nível mínimo configurado. 4. Sistema emite alertas automáticos para utilizadores com perfil relevante.
Fluxo Alternativo (1)	<i>[Gerente pretende configurar nível mínimo de um produto] (passo 2)</i> 2.1 Gerente seleciona produto e define novo nível mínimo. 2.2 Sistema guarda a configuração do nível mínimo. 2.3 Regressa a 2.
Fluxo Alternativo (2)	<i>[Gerente deteta produtos em rutura e pretende criar encomenda] (passo 3)</i> 3.1 Gerente navega para o módulo de gestão de encomendas. 3.2 Gerente cria encomenda ao fornecedor para os produtos em falta.
Origem	RF-05; RF-07; US-02; US-08
Fluxo Alternativo (3)	<i>[Gerente pretende fechar um alerta resolvido] (passo 4)</i> 4.1 Gerente seleciona o alerta e marca-o como resolvido. 4.2 Sistema regista a dataResolucao e impede a emissão de novo alerta duplicado para o mesmo produto enquanto o anterior não estiver fechado.

Tabela 2.21: UC-07 – Consultar Stock e Receber Alertas de Reposição

Registrar Entrada de Mercadoria

Use Case	Registrar Entrada de Mercadoria
Descrição	Permite ao Responsável de Armazém registrar a recepção de mercadoria entregue por um fornecedor, associando-a a uma guia de remessa e atualizando automaticamente os níveis de stock.
Cenários	Resp. Armazém recebe entrega do fornecedor, regista a guia de remessa e o sistema atualiza o stock de 10 produtos.
Pré-condição	Responsável de Armazém autenticado; encomenda correspondente previamente registada no sistema com estado “pendente” ou “enviada”.
Pós-condição	Entrada de mercadoria registada; stock atualizado; guia de remessa associada à encomenda; estado da encomenda atualizado para “recebida”.
Fluxo Normal	1. Responsável de Armazém seleciona a encomenda correspondente à entrega. 2. Responsável indica o número da guia de remessa do fornecedor. 3. Sistema regista a guia de remessa associada à encomenda. 4. Responsável regista os produtos recebidos e respetivas quantidades. 5. Sistema regista a entrada de mercadoria na loja. 6. Sistema atualiza automaticamente o stock de cada produto recebido. 7. Sistema atualiza o estado da encomenda para “recebida”.
Fluxo de Exceção (1)	<i>[Quantidade recebida diverge da encomendada] (passo 4)</i> 4.1 Sistema deteta discrepância e alerta o Responsável. 4.2 Responsável confirma a quantidade efetivamente recebida. 4.3 Sistema regista a discrepância e regressa a 5.
Origem	RF-06; US-07; US-19

Tabela 2.22: UC-08 – Registrar Entrada de Mercadoria

Criar Encomenda a Fornecedor

Use Case	Criar Encomenda a Fornecedor
Descrição	Permite ao Gestor ou Gerente criar uma encomenda de reposição dirigida a um fornecedor, com base em alertas de stock mínimo ou por iniciativa própria, com validação do horário de processamento do armazém.
Cenários	Sistema emite alerta de stock mínimo; Gestor cria encomenda consolidada para o fornecedor principal.
Pré-condição	Utilizador autenticado com perfil de Gestor ou Gerente; fornecedor registado no sistema com condições comerciais definidas.
Pós-condição	Encomenda registada com estado “pendente”; data esperada de processamento calculada e comunicada ao utilizador.
Fluxo Normal	1. Utilizador acede ao módulo de gestão de encomendas. 2. Sistema apresenta sugestões automáticas de reposição com base nos alertas de stock ativos. 3. Utilizador seleciona o fornecedor e indica os produtos e quantidades a encomendar. 4. Sistema verifica o horário de funcionamento do armazém do fornecedor (seg–sex 08h00–18h00). 5. Sistema regista a encomenda com estado “pendente”. 6. Sistema informa a data e hora esperadas de processamento da encomenda.
Fluxo Alternativo (1)	<i>[Encomenda submetida fora do horário do armazém do fornecedor] (passo 4)</i> 4.1 Sistema informa que a encomenda apenas será processada no próximo dia útil dentro do horário (seg–sex 08h00–18h00). 4.2 Sistema regista a encomenda e calcula a data esperada de processamento. 4.3 Regressa a 6.
Origem	RF-03; RD-06; US-04; US-19

Tabela 2.23: UC-09 – Criar Encomenda a Fornecedor

Registrar Ajuste Manual de Inventário

Use Case	Registrar Ajuste Manual de Inventário
Descrição	Permite ao Gerente de Loja corrigir manualmente o stock de um produto, indicando a quantidade e o motivo do ajuste (quebra, desperdício ou correção de erro). A operação fica rastreada no log de auditoria.
Cenários	Gerente deteta produto danificado em prateleira e regista uma quebra de 5 unidades.
Pré-condição	Gerente autenticado; produto a ajustar registado no sistema com stock atual conhecido.
Pós-condição	Ajuste registado, stock corrigido e operação registada no log de auditoria com identificação do responsável e motivo.
Fluxo Normal	1. Gerente seleciona o produto a ajustar. 2. Gerente indica a quantidade de ajuste (positiva ou negativa) e o motivo. 3. Sistema regista o ajuste com identificação do utilizador responsável. 4. Sistema atualiza o stock do produto na loja. 5. Sistema regista a operação no log de auditoria.
Fluxo de Exceção (1)	<i>[Ajuste negativo resultaria em stock negativo] (passo 3)</i> 3.1 Sistema bloqueia o ajuste e informa o Gerente do stock atual disponível. 3.2 Gerente corrige a quantidade e regressa a 3.
Origem	RF-11; RNF-05; US-12

Tabela 2.24: UC-10 – Registrar Ajuste Manual de Inventário

Realizar Inventário Físico

Use Case	Realizar Inventário Físico
Descrição	Permite ao Responsável de Armazém registrar contagens físicas de produtos e obter automaticamente as discrepâncias face ao stock registado no sistema.
Cenários	Resp. Armazém realiza inventário mensal, conta 45 unidades de produto X; sistema registava 50 — discrepância de -5.
Pré-condição	Responsável de Armazém autenticado. Caso se detetem discrepâncias que exijam correção, o Gerente deve ter acesso ao módulo de ajustes de inventário.
Pós-condição	Inventário físico fechado com discrepâncias calculadas e visíveis para o Gerente.
Fluxo Normal	1. Responsável inicia novo inventário físico para a loja. 2. Sistema cria automaticamente uma linha de inventário para cada produto com <i>stock</i> registado na loja, incluindo <i>snapshot</i> da quantidade atual registada no sistema. 3. Responsável regista a quantidade contada para cada produto. 4. Sistema calcula automaticamente a discrepância (contado vs. sistema) para cada produto. 5. Sistema apresenta resumo das discrepâncias detetadas. 6. Responsável confirma e fecha o inventário.
Fluxo Alternativo (1)	<i>[Responsável deteta erro de contagem antes de fechar] (passo 5)</i> 5.1 Responsável seleciona o produto com erro e corrige a quantidade contada. 5.2 Sistema recalcula a discrepância. 5.3 Regressa a 5.
Origem	RF-19; US-20

Tabela 2.25: UC-11 – Realizar Inventário Físico

Gerir Utilizadores e Acessos

Use Case	Gerir Utilizadores e Acessos
Descrição	Permite ao Gerente de Loja criar, editar e desativar contas de utilizadores da sua loja, definindo o perfil de acesso de cada colaborador. Todas as alterações ficam registadas no log de auditoria.
Cenários	Gerente cria conta para novo Funcionário com perfil “Funcionário” para operar o PDV.
Pré-condição	Gerente autenticado.
Pós-condição	Conta de utilizador criada, atualizada ou desativada; alteração registada no log de auditoria.
Fluxo Normal	1. Gerente acede ao módulo de gestão de utilizadores. 2. Gerente seleciona a operação pretendida: criar, editar ou desativar conta. 3. Gerente preenche os dados do utilizador e seleciona o perfil de acesso. 4. Sistema valida os dados introduzidos. 5. Sistema executa a operação sobre a conta. 6. Sistema regista a operação no log de auditoria.
Fluxo de Exceção (1)	<i>[Username já existe ao criar nova conta] (passo 4)</i> 4.1 Sistema informa que o username já está em uso. 4.2 Gerente corrige o username e regressa a 4.
Origem	RF-10; RNF-04; RNF-05; US-11

Tabela 2.26: UC-12 – Gerir Utilizadores e Acessos

Sincronizar Dados com Servidor Central

Use Case	Sincronizar Dados com Servidor Central
Descrição	O sistema da loja sincroniza automaticamente no final de cada dia os dados locais (vendas, stock, fechos de caixa) com o servidor central, assegurando a consolidação da informação da cadeia. A comunicação é cifrada com TLS 1.2+.
Cenários	Após o fecho de caixa às 20h00, o sistema da loja inicia a sincronização automática com o servidor central.
Pré-condição	Fecho de caixa diário registado; ligação à rede disponível; dados locais pendentes de sincronização existentes.
Pós-condição	Dados integrados no servidor central; estado da sincronização registado com resultado (concluída ou com conflitos).
Fluxo Normal	1. Sistema deteta o agendamento de sincronização (fim de jornada ou ligação restabelecida). 2. Sistema recolhe todos os dados locais pendentes de sincronização. 3. Sistema inicia transmissão cifrada (TLS 1.2+) para o servidor central. 4. Servidor central valida e integra os dados recebidos. 5. Sistema regista o estado da sincronização.
Fluxo de Exceção (1)	<i>[Falha de ligação durante a transmissão] (passo 3)</i> 3.1 Sistema regista sincronização com estado “pendente” e agenda nova tentativa. 3.2 PDV continua a operar em modo offline; registos são mantidos localmente.
Fluxo de Exceção (2)	<i>[Conflito de dados detetado no servidor] (passo 4)</i> 4.1 Servidor regista o conflito e aplica regra de resolução (<i>last-write-wins</i>). 4.2 Sistema notifica o Gestor dos conflitos detetados.
Origem	RF-17; RNF-02; RNF-03; RNF-06; US-01; US-18

Tabela 2.27: UC-13 – Sincronizar Dados com Servidor Central

Gerir Fornecedores e Condições Comerciais

Use Case	Gerir Fornecedores e Condições Comerciais
Descrição	Permite ao Gestor da Cadeia registar e gerir fornecedores, incluindo os produtos que cada um fornece e as condições comerciais associadas (preço, prazo de entrega, quantidade mínima de encomenda).
Cenários	Gestor regista novo fornecedor de bebidas e associa produtos do catálogo com condições de preço e prazo de entrega.
Pré-condição	Gestor autenticado; produtos a associar ao fornecedor já registados no catálogo.
Pós-condição	Fornecedor registado ou atualizado; produtos e condições comerciais associados e disponíveis para criação de encomendas.
Fluxo Normal	1. Gestor acede ao módulo de gestão de fornecedores. 2. Gestor preenche os dados do fornecedor (nome, NIF, contacto). 3. Sistema valida e regista o fornecedor. 4. Gestor associa produtos do catálogo ao fornecedor. 5. Gestor define as condições comerciais por produto (preço, prazo, quantidade mínima). 6. Sistema guarda as condições comerciais por produto e fornecedor.
Fluxo Alternativo (1)	<i>[Gestor pretende editar dados de fornecedor existente] (passo 1)</i> 1.1 Gestor pesquisa e seleciona o fornecedor a editar. 1.2 Regressa a 2.
Fluxo Alternativo (2)	<i>[Gestor pretende desativar fornecedor] (passo 1)</i> 1.1 Gestor seleciona o fornecedor e confirma a desativação. 1.2 Sistema verifica se existem encomendas pendentes para o fornecedor. 1.3 Sistema desativa o fornecedor e regista a operação no log de auditoria.
Origem	RF-03; US-04

Tabela 2.28: UC-14 – Gerir Fornecedores e Condições Comerciais

2.19 Utilização de Agentes LLM na Engenharia de Requisitos

O desenvolvimento do presente documento SRS contou com a integração sistemática de agentes de Inteligência Artificial Generativa (*Large Language Models*) ao longo de todas as atividades de Engenharia de Requisitos. Esta secção descreve o papel desempenhado por cada agente, a metodologia de interação adotada e os *prompts* representativos utilizados em cada fase, assegurando a rastreabilidade e a transparência do processo.

Agentes Utilizados e Papéis Atribuídos

Foram utilizados dois agentes LLM com papéis complementares e distintos:

- **Agente Principal — *ChatGPT 5.2 (OpenAI)***: responsável pela condução das atividades de elicitação, refinamento, estruturação e especificação de requisitos. Este agente foi utilizado de forma ativa e iterativa ao longo de todo o processo, servindo como ferramenta de apoio cognitivo na simulação de entrevistas, geração de *User Stories*, identificação de ambiguidades e formulação dos requisitos funcionais e não funcionais.
- **Agente de Validação — *Claude Sonnet 4.6 (Anthropic)***: utilizado exclusivamente numa fase posterior, com o objetivo de rever e validar de forma independente os artefactos produzidos pelo agente principal. A sua intervenção visou identificar inconsistências, lacunas, ambiguidades e imprecisões que pudessem ter escapado à análise inicial, funcionando como um revisor externo imparcial.

Em ambos os casos, todas as decisões de estruturação, priorização e validação final dos requisitos foram tomadas pela equipa, utilizando os agentes como ferramentas de suporte e não como substitutos de pensamento crítico.

Fase 1 — Simulação de Entrevistas com *Stakeholders*

A primeira utilização do agente LLM principal incidiu sobre a simulação de entrevistas com os *stakeholders*. Uma vez que o acesso direto e recorrente a todos os intervenientes reais não era praticável no contexto da unidade curricular, o agente foi instruído a adotar o papel de cada *stakeholder* e a responder a perguntas da equipa de forma contextualizada e consistente com o perfil definido.

O prompt base utilizado para iniciar cada sessão de entrevista simulada seguiu a seguinte estrutura:

“Vais assumir o papel de [nome/perfil do stakeholder]. O teu perfil é o seguinte: [descrição detalhada do papel, responsabilidades e contexto]. Responde às minhas perguntas como se fosses esta pessoa, de forma realista e coerente com o contexto de uma cadeia de lojas de conveniência em expansão. Não respondas de forma genérica, baseia-te sempre nas especificidades do negócio do Sr. Formiga.”

A título de exemplo, o prompt utilizado para iniciar a entrevista ao Gerente de Loja foi o seguinte:

“Vais assumir o papel de Gerente de Loja da cadeia Mini-Formiga. Tens 35 anos, trabalhas na loja há 4 anos e és responsável pela gestão diária do estabelecimento: abertura, fecho de caixa, receção de mercadoria, controlo de stock e gestão da equipa. Tens algum conhecimento informático básico, mas não és técnico.”

Responde às perguntas que te vou colocar como se estivesses a ser entrevistado para identificar as tuas necessidades em relação a um novo sistema de gestão.”

Fase 2 — Geração e Refinamento de *User Stories*

Após a realização das entrevistas simuladas, o agente principal foi utilizado para apoiar a formulação das *User Stories* a partir dos conteúdos recolhidos. O prompt utilizado seguiu o seguinte modelo:

“Com base na seguinte transcrição de entrevista, extrai as necessidades implícitas e explícitas do utilizador e formula-as como User Stories no formato: ‘Como [papel], quero [funcionalidade], para que [benefício].’ Para cada User Story, sugere também uma prioridade MoSCoW (Must Have, Should Have, Could Have, Won’t Have) justificando brevemente a tua escolha.”

Os resultados gerados pelo agente foram analisados criticamente pela equipa, que procedeu à seleção, reformulação e priorização final das *User Stories*, eliminando duplicados, fundindo histórias redundantes e ajustando formulações imprecisas.

Fase 3 — Especificação de Requisitos Funcionais e Não Funcionais

O agente principal foi igualmente utilizado para apoiar a transição das *User Stories* para requisitos formais, segundo o *standard* IEEE 830/29148. O prompt utilizado seguiu o seguinte modelo:

“Com base nas seguintes User Stories, gera uma lista de requisitos funcionais e não funcionais segundo o standard IEEE 830. Para cada requisito: (1) atribui um identificador único; (2) redige a descrição de forma clara, objetiva e verificável; (3) indica a User Story de origem; (4) classifica a prioridade segundo MoSCoW. Sinaliza qualquer ambiguidade ou incompletude que detetares.”

O agente produziu um conjunto inicial de requisitos que foi posteriormente revisto pela equipa, com especial atenção à verificabilidade de cada requisito, à ausência de sobreposições entre requisitos funcionais e não funcionais, e à consistência das prioridades atribuídas.

Fase 4 — Identificação de Requisitos de Domínio

Para além dos requisitos funcionais e não funcionais, o agente principal foi utilizado para apoiar a identificação de obrigações legais e regras de negócio do setor que o sistema deve obrigatoriamente respeitar. O prompt utilizado foi o seguinte:

“Com base no contexto do sistema Mini-Formiga, identifica os requisitos de domínio relevantes, nomeadamente obrigações legais (fiscalidade, RGPD, faturação) e regras de negócio específicas do setor do retalho em Portugal. Para cada requisito, indica a sua fonte (artigo de lei, regulamento ou regra de negócio). Mantém a lista concisa e focada no que é diretamente relevante para o âmbito do sistema.”

Os resultados gerados pelo agente foram revistos pela equipa, que validou a pertinência de cada requisito face ao âmbito do sistema, eliminou entradas excessivamente complexas e confirmou as fontes legais referenciadas. Os requisitos de domínio definitivos encontram-se na Tabela 2.12.

Fase 5 — Identificação de Entidades e Relacionamentos do Domínio

Na fase de modelação conceptual do domínio, o agente principal foi utilizado para identificar entidades e relacionamentos a partir dos requisitos especificados. O prompt utilizado foi o seguinte:

“Analisa os seguintes requisitos funcionais, não funcionais e de domínio e identifica as entidades do modelo conceptual do sistema. Para cada entidade: (1) atribui um nome em PascalCase; (2) fornece uma descrição concisa; (3) indica os requisitos que justificam a sua existência. Não incluas atributos — foca-te apenas nos conceitos do domínio.”

Os resultados foram revistos e ajustados pela equipa. As entidades e relacionamentos definitivos encontram-se nas Tabelas 2.14 e 6.1.

Fase 6 — Validação Cruzada com Agente Independente

Na fase final, os artefactos produzidos foram submetidos a validação cruzada pelo agente de validação (Claude Sonnet 4.6). O prompt utilizado foi o seguinte:

“Atua como revisor independente de um documento SRS segundo o standard IEEE 830/29148. Revê os seguintes artefactos: [User Stories], [requisitos funcionais], [requisitos não funcionais] e [requisitos de domínio]. Para cada conjunto, verifica: (1) Completude; (2) Consistência; (3) Verificabilidade; (4) Rastreabilidade; (5) Ambiguidades. Apresenta os problemas encontrados indicando o ID do requisito e a natureza do problema.”

As observações produzidas pelo agente foram analisadas pela equipa, que procedeu à correção das inconsistências identificadas, nomeadamente a sobreposição parcial entre os requisitos de domínio RD-01 a RD-03 e os requisitos funcionais e não funcionais de faturação, que foi

resolvida clarificando que os requisitos de domínio estabelecem a obrigação legal e os requisitos funcionais especificam o comportamento do sistema para a satisfazer.

3 Arquitetura e Design do Software utilizando LLM

3.1 Arquitetura Global do Sistema

O sistema Mini-Formiga adota uma arquitetura cliente-servidor multicamada, concebida para suportar simultaneamente a operação autónoma de cada loja e a consolidação centralizada da informação ao nível da cadeia. Conforme ilustrado na Figura 3.1, a arquitetura encontra-se organizada em três grandes camadas: apresentação, lógica de negócio e dados.

A solução foi desenhada segundo um modelo distribuído, em que cada loja possui um servidor local responsável pela execução da lógica principal do sistema e pela persistência operacional em base de dados local, enquanto um servidor central assegura a consolidação e sincronização global da informação proveniente de todas as lojas.

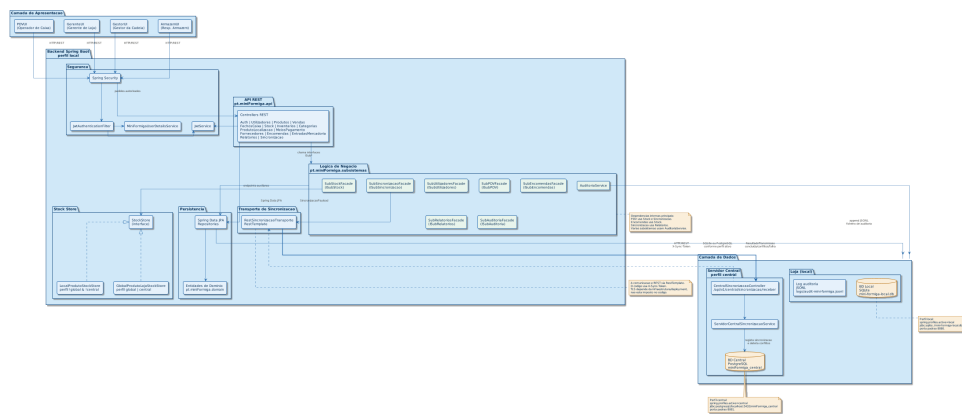


Figura 3.1: Arquitetura global do sistema Mini-Formiga

Camada de Apresentação

A camada de apresentação é composta pelos diferentes clientes responsáveis pela interação com os utilizadores do sistema. Todos os clientes comunicam com o backend local através de serviços HTTP/REST.

Os componentes identificados no diagrama são:

- **PDVUI** — interface utilizada pelos operadores de caixa para execução de vendas, pagamentos e operações correntes de ponto de venda;
- **GerenteUI** — interface destinada ao gerente de loja, permitindo a gestão operacional da loja, incluindo stock, encomendas, fechos de caixa e consultas locais;
- **GestorUI** — interface utilizada pelo gestor da cadeia para acesso a relatórios consolidados, auditoria e monitorização global das lojas;
- **ArmazemUI** — interface orientada ao responsável de armazém para registo de entradas de mercadoria, inventários e controlo logístico.

Todas estas interfaces comunicam com a API REST do sistema através de pedidos autenticados.

Camada de Lógica de Negócio

A lógica de negócio encontra-se implementada num backend, executado localmente em cada loja. Esta camada constitui o núcleo funcional do sistema e encontra-se organizada segundo uma arquitetura modular baseada em subsistemas especializados.

O acesso externo às funcionalidades é realizado através da API REST, composta por vários controladores REST responsáveis por domínios funcionais distintos, incluindo:

- autenticação e utilizadores;
- produtos e categorias;
- fornecedores e encomendas;
- inventários e stock;
- vendas e meios de pagamento;
- relatórios;
- sincronização;
- entradas de mercadoria.

Internamente, a lógica de negócio encontra-se organizada em vários subsistemas independentes, cada um exposto através da respetiva interface de fachada:

- **SubStockFacade (ISubStock)** — gestão de stock, inventários e movimentos de armazém;

- **SubSincronizacaoFacade (ISubSincronizacao)** — preparação, transporte e controle da sincronização de dados entre loja e servidor central;
- **SubUtilizadoresFacade (ISubUtilizadores)** — gestão de utilizadores, autenticação e permissões;
- **SubPDVFacade (ISubPDV)** — operações de ponto de venda e processamento de vendas;
- **SubEncomendasFacade (ISubEncomendas)** — gestão de encomendas a fornecedores;
- **SubRelatoriosFacade (ISubRelatorios)** — geração de relatórios operacionais e estatísticos;
- **SubAuditoriaFacade (ISubAuditoria)** — registo de auditoria e rastreabilidade de operações;

A separação por subsistemas promove modularidade, baixo acoplamento e maior facilidade de manutenção e evolução futura do sistema.

Persistência e Armazenamento Local

A persistência de dados é realizada através do módulo **Repositories**, responsável pelo acesso às entidades de domínio.

Cada loja possui uma base de dados local, executada no próprio servidor local da loja. Esta base de dados garante persistência imediata das operações críticas do sistema, incluindo:

- vendas e transações de PDV;
- movimentos de stock;
- inventários;
- fechos de caixa;
- encomendas;
- entradas de mercadoria;
- registos de auditoria.

O funcionamento local permite que cada loja continue operacional mesmo na ausência de ligação ao servidor central, assegurando tolerância a falhas de rede e continuidade de serviço em modo *offline*.

Mecanismo de Sincronização

A sincronização entre as lojas e o servidor central é suportada pelo módulo **RestSincronizacaoTransporte**, responsável pelo transporte REST dos dados sincronizados.

Os dados são enviados através de pedidos HTTP/REST para o componente **CentralSincronizacaoController**, alojado no servidor central. O processamento da sincronização é coordenado pelo **ServidorCentralSincronizacaoService**, responsável pela validação e integração dos dados recebidos.

O mecanismo de sincronização opera sobre mensagens serializadas em JSON e inclui capacidade de retenção local em caso de indisponibilidade da rede. Quando ocorre falha de comunicação, os pedidos permanecem localmente até nova tentativa automática de envio.

A comunicação entre loja e servidor central é realizada sobre ligações seguras TLS, assegurando confidencialidade e integridade dos dados transmitidos.

Camada de Dados Central

O servidor central possui uma base de dados, responsável pela consolidação da informação proveniente de todas as lojas da cadeia.

Esta base de dados central constitui a fonte de dados consolidada para:

- relatórios globais da cadeia;
- auditoria centralizada;
- análise estatística;
- monitorização operacional;
- apoio à decisão estratégica.

Enquanto a base de dados local privilegia disponibilidade operacional e autonomia da loja, a base de dados central privilegia integração, consistência global e análise consolidada da informação.

3.2 Diagrama de Classes

O Diagrama de Classes constitui a espinha dorsal estrutural do sistema Mini-Formiga. A sua elaboração partiu da necessidade de traduzir, de forma rigorosa e implementável, os contratos funcionais expressos nos casos de utilização e os requisitos arquiteturais definidos para o

sistema. Em particular, a arquitetura distribuída, em que cada loja opera de forma autónoma com uma base de dados local, sincronizando periodicamente com um servidor central, exigiu decisões de modelação que vão além da simples representação de entidades: foi necessário garantir que as classes de domínio suportam operação *offline*, resolução de conflitos, rastreabilidade auditável e separação clara entre subsistemas. O diagrama cobre exclusivamente a camada de lógica de negócio e ao domínio de dados descritos na arquitetura global. Em 3.2 encontra-se o diagrama de classes do sistema mini-formiga.

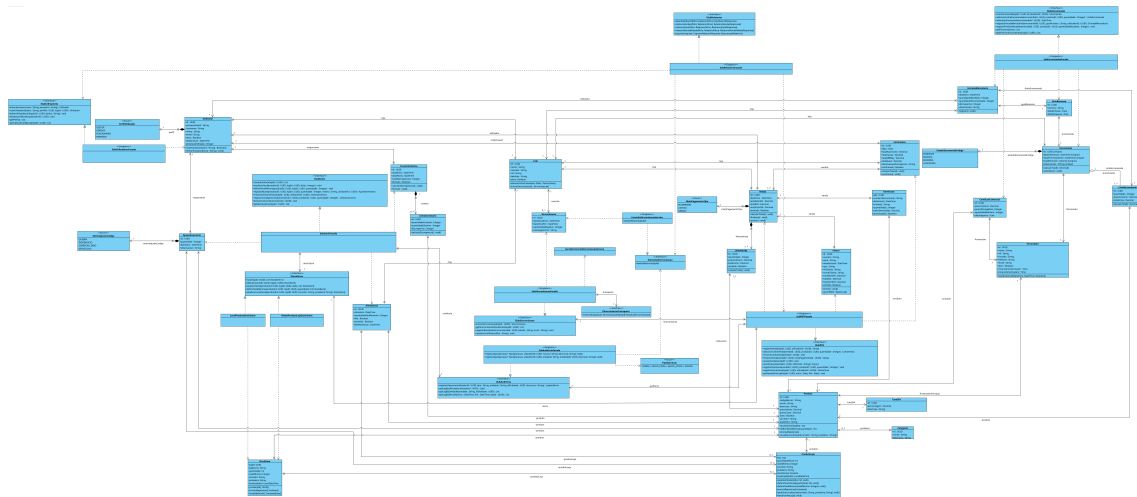


Figura 3.2: Diagrama de Classes do sistema Mini-Formiga

3.2.1 Organização Modular e Responsabilidades

A lógica de negócio foi organizada em seis subsistemas funcionais, cada um com um conjunto coeso de responsabilidades e exposto ao exterior através de uma interface de fachada. As classes de domínio pertencem exclusivamente ao subsistema que as gere e não são acessíveis diretamente pelas camadas superiores.

SubUtilizadores

Este subsistema centraliza a autenticação e a gestão de acessos. A classe *Utilizador* representa qualquer ator do sistema, Gestor, Gerente, Funcionário ou Responsável de Armazém, e associa-se a um *Perfil* (modelado como *PerfilUtilizador* por composição) que define as permissões do utilizador. A interface *ISubUtilizadores*, implementada por *SubUtilizadoresFacade*, expõe as operações de autenticação, criação, edição e desativação de contas. Cada *Utilizador* está obrigatoriamente ligado a uma *Loja*, impedindo a existência de utilizadores sem contexto organizacional.

SubAuditoria

O subsistema de auditoria regista de forma imutável todas as operações sensíveis do sistema. A classe `LogAuditoria` representa cada entrada de registo e classifica-se em `TipoOperacao`, permitindo filtrar e pesquisar registos por categoria de ação (acesso, venda, ajuste de inventário, etc.). A `SubAuditoriaFacade` implementa `ISubAuditoria` e é invocada transversalmente pelos restantes subsistemas — em particular pelo `SubPDVFacade` e pelo `SubStockFacade`, que mantêm uma referência à interface `ISubAuditoria` — sempre que uma operação rastreável é executada. Esta dependência cruzada é a única exceção intencional ao isolamento entre subsistemas, justificada pelo requisito transversal de auditabilidade (RNF-05). Adicionalmente, o `AuditoriaService` funciona como serviço de suporte interno, sendo utilizado pelas facades para centralizar a lógica de construção e persistência dos registos de auditoria.

SubSincronizacao

Este subsistema é crítico para a operação distribuída do sistema. Gere o fluxo de dados entre a base de dados local de cada loja e o servidor central, suportando a estratégia *last-write-wins* para resolução de conflitos (RF-17). A classe `Sincronizacao` representa cada processo de sincronização executado por uma Loja, e associa-se a `EstadoSincronizacao` para rastrear o historial de transições (pendente, em curso, concluída, com conflitos). A classe `Conflito` regista as situações em que o servidor central detetou colisões nos dados recebidos. A `SubSincronizacaoFacade` implementa `ISubSincronizacao` e depende de `ISincronizacaoTransporte`, implementada por `RestSincronizacaoTransporte`, que encapsula a comunicação HTTP/REST com o `ServidorCentralSincronizacaoService`. Esta separação através de interface permite substituir o mecanismo de transporte sem alterar a lógica do subsistema. O `ConsultaSincronizacaoService` implementa `IConsultaSincronizacao` e fornece operações de consulta do estado de sincronização, separando a responsabilidade de escrita da de leitura.

SubStock

O subsistema de stock implementa o controlo de quantidades em tempo real e a gestão de inventários. A classe `Stock` monitoriza os níveis de inventário por loja e produto; quando um produto atinge o limiar configurado em `NivelMinimo`, este desencadeia um `AlertaStock`, que é recebido pelos utilizadores com perfil relevante. O processo de `InventarioFisico` é composto por múltiplas `LinhaInventario` (composição 1..*), cada uma referenciando um `Produto` e calculando automaticamente a discrepância face ao stock registado. Os ajustes manuais são registados em `AjusteInventario`, associado ao `MotivoAjuste` e ao `Utilizador` responsável, garantindo a rastreabilidade auditável das correções. A `SubStockFacade` implementa `ISubStock` e depende de `IStockStore`, uma interface implementada por duas classes distintas: `LocalProdutoStockStore` para a persistência local em loja, e `GlobalProdutoLojaStockStore` para a perspetiva consolidada no servidor central.

SubPDV

O núcleo transacional do sistema. A classe Venda agrega, por composição, múltiplas LinhaVenda (1..*), cada uma referenciando um Produto e aplicando a respetiva TaxaIVA para o cálculo automático dos totais com IVA discriminado. O processo culmina na emissão opcional de uma Fatura (associação 0..1), controlada por FaturaSequencia para garantir a numeração sequencial única exigida pela Autoridade Tributária (RD-02 e RD-03), e no registo do FechoCaixa, que consolida as vendas do dia e classifica os pagamentos por MeioPagamentoTipo. A classe Devolucao associa-se diretamente a uma Venda (1..1) e a um Produto, garantindo a rastreabilidade da devolução à transação original. A SubPDVFacade implementa ISubPDV e mantém referências a ISubAuditoria e ISubSincronizacao, assegurando que todas as transações de venda são registadas e que os dados ficam disponíveis para sincronização.

SubEncomendas

Gere a cadeia de abastecimento desde a criação da encomenda até à receção de mercadoria. A classe Fornecedor encapsula dados comerciais e lógica de verificação de disponibilidade do armazém, calculando a data esperada de processamento com base no horário definido (RF-03, RD-06). A Encomenda associa-se a um Fornecedor (1) e a uma Loja (1), e é composta por LinhaEncomenda (1..*, agregação partilhada). O estado da encomenda é rastreado por EstadoEncomenda, associado a um EstadoEncomendaCodigo (por composição), permitindo registar o historial de transições. A receção de mercadoria é registada em EntradaMercadoria, que valida a GuiaRemessa e referencia as linhas de encomenda recebidas. Toda a lógica de negócio deste subsistema é exposta por SubEncomendasFacade, que implementa ISubEncomendas. O subsistema de relatórios, gerido por SubRelatoriosFacade (que implementa ISubRelatorios), agrega dados de vários subsistemas para produzir os indicadores consolidados necessários ao *dashboard* do Gestor da Cadeia (UC-06).

3.2.2 Padrões de Projeto Utilizados

Facade por Subsistema

O padrão de projeto central da arquitetura é o **Facade**. Cada subsistema expõe uma interface pública, ISubPDV, ISubStock, ISubEncomendas, ISubStock, ISubUtilizadores, ISubAuditoria, ISubSincronizacao e ISubRelatorios, implementada por uma classe *Facade* dedicada. Este padrão foi adotado por três razões fundamentais: primeiro, encapsula a complexidade interna de cada subsistema, impedindo que as camadas superiores (controladores REST) dependam diretamente das classes de domínio; segundo, define contratos estáveis entre camadas, de modo que alterações internas a um subsistema não propagam mudanças para o exterior; terceiro, facilita a testabilidade, uma vez que as interfaces permitem a substituição das implementações reais por *mocks* nos testes unitários e de integração. A comunicação

entre subsistemas é sempre mediada pelas interfaces, nunca pelas classes *Facade* concretas, o que reforça o baixo acoplamento.

Strategy (via Interface de Transporte e de Armazenamento)

O padrão **Strategy** foi aplicado em dois pontos distintos do diagrama. No `SubSincronizacao`, a interface `ISincronizacaoTransporte` abstrai o mecanismo de envio dos dados para o servidor central; a implementação atual é `RestSincronizacaoTransporte`, mas a arquitetura permite introduzir um mecanismo alternativo (por exemplo, baseado em filas de mensagens) sem qualquer alteração à `SubSincronizacaoFacade`. No `SubStock`, a interface `IStockStore` abstrai o repositório de dados de stock; existem duas implementações concretas, `LocalProdutoStockStore` para o contexto de loja e `GlobalProdutoLojaStockStore` para o servidor central, permitindo que o `SubStockFacade` opere de forma idêntica em ambos os contextos, injetando a implementação adequada em cada ambiente.

Service Layer

O `AuditoriaService` e o `ConsultaSincronizacaoService` materializam o padrão **Service Layer**, centralizando lógica de negócio transversal que não pertence a nenhuma entidade de domínio individualmente. O `AuditoriaService` é utilizado pelas *facades* que necessitam de registar operações auditáveis, evitando duplicação de código. O `ConsultaSincronizacaoService` separa a responsabilidade de consulta do estado de sincronização da responsabilidade de execução da mesma, seguindo o princípio de separação de preocupações.

3.2.3 Decisões de Design Orientado a Objetos

Composição vs. Associação

A distinção entre composição e associação foi aplicada de forma sistemática com base no ciclo de vida das entidades. Utilizou-se **composição** nos casos em que os elementos dependentes não fazem sentido fora do seu agregador:

- `Venda` $\diamond\rightarrow$ `LinhaVenda` (1..*): uma linha de venda não existe independentemente da venda a que pertence; a sua destruição ocorre sempre com a da venda;
- `InventarioFisico` $\diamond\rightarrow$ `LinhaInventario` (0..*): cada linha de inventário é um detalhe interno do inventário físico que a contém;
- `Utilizador` $\diamond\rightarrow$ `PerfilUtilizador` (1): o perfil é um atributo intrínseco do utilizador, não partilhado nem reutilizável fora desse contexto;

- `Encomenda` $\diamond \rightarrow$ `EstadoEncomendaCodigo` (1) e `EstadoSincronizacaoCodigo` nas respectivas classes: o código de estado é parte integral da entidade que o contém.

Nos restantes casos, optou-se por **associação simples**, preservando a independência dos ciclos de vida. Por exemplo, `AjusteInventario` associa-se a `Stock` (e não por composição), porque o registo histórico do ajuste deve ser preservado para auditoria mesmo que o valor de stock seja posteriormente corrigido. Da mesma forma, `Venda` associa-se a `Fatura` (0..1) por associação simples, pois uma fatura pode existir sem estar permanentemente acoplada ao ciclo de vida da venda, sendo necessária a sua consulta independente.

A `Encomenda` associa-se às `LinhaEncomenda` por **agregação partilhada** (1..*), dado que as linhas podem ser referenciadas por `EntradaMercadoria` aquando da receção de mercadoria, implicando um ciclo de vida semi-independente.

Identificação por UUID

Todas as entidades utilizam UUID (*Universally Unique Identifier*) como chave primária. Esta decisão é estrutural para a arquitetura distribuída: permite que registos criados localmente em qualquer loja, durante operação *offline*, não entrem em conflito com registos de outras lojas no momento da consolidação no servidor central. A alternativa — chaves numéricas sequenciais geridas pela base de dados — seria impraticável num contexto em que múltiplas instâncias criam registos simultaneamente e de forma independente.

Lógica de Negócio Descentralizada nas Classes de Domínio

Seguindo os princípios do *Domain-Driven Design*, operações cujo resultado depende exclusivamente do estado da própria entidade foram encapsuladas nas classes de domínio, e não nos *Facades*. É o caso de `calcularMargem()` em `Produto`, `calcularDiscrepancia()` em `LinhaInventario` e `estaDisponivel()` em `Fornecedor`. Esta abordagem reduz o acoplamento entre as classes de domínio e as *facades*, tornando cada entidade individualmente testável por testes unitários sem necessidade de instanciar o subsistema completo.

Classes de Estado como Entidades Autónomas

As classes `EstadoEncomenda` e `EstadoSincronizacao` foram modeladas como entidades independentes, em vez de simples enumerações. Esta decisão permite rastrear o historial de transições de estado e associar metadados (data, utilizador responsável, mensagem descritiva) a cada transição, sendo essencial para os requisitos de auditabilidade dos processos de encomenda e sincronização. As enumerações `EstadoEncomendaCodigo` e `EstadoSincronizacaoCodigo`, associadas por composição, representam o valor discreto do estado atual, separando a identidade do estado dos seus atributos históricos.

3.2.4 Exemplos Concretos Derivados de Casos de Utilização

A título ilustrativo, descrevem-se de seguida duas classes cujos atributos e operações foram diretamente derivados de casos de utilização específicos.

Classe *FechoCaixa* — derivada de UC-05

O caso de utilização UC-05 (*Registar Fecho de Caixa Diário*) especifica que o sistema deve totalizar as vendas do dia por meio de pagamento, apresentar um resumo ao Gerente com totais por numerário, cartão, MB Way e total geral, e registar o fecho com os valores confirmados. O fluxo alternativo prevê ainda o registo de uma observação de discrepância. Estes requisitos traduziram-se diretamente na classe *FechoCaixa*, que agrega as vendas do dia (associação com *Venda*), referencia o *Utilizador* responsável pelo fecho (o Gerente), pertence a uma Loja e é classificada por um *MeioPagamentoTipo* por composição, o que permite armazenar os totais discriminados por forma de pagamento sem recalculá-los a partir das linhas de venda em cada consulta. O atributo *observacaoDiscrepancia* corresponde diretamente ao fluxo alternativo do UC-05.

Classe *EntradaMercadoria* — derivada de UC-08

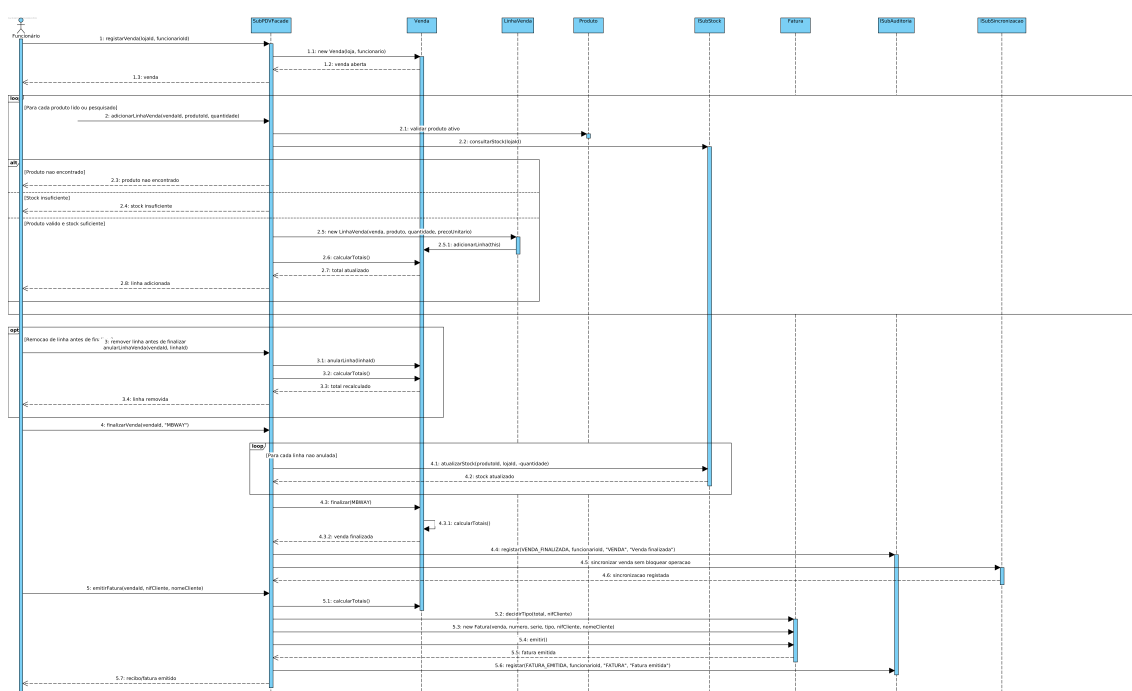
O caso de utilização UC-08 (*Registar Entrada de Mercadoria*) especifica que o Responsável de Armazém deve selecionar a encomenda correspondente à entrega, indicar o número da guia de remessa, registar os produtos e quantidades recebidas, e que o sistema deve atualizar automaticamente o stock e o estado da encomenda. O fluxo de exceção prevê o registo de discrepâncias entre quantidades encomendadas e recebidas. Estes requisitos materializaram-se na classe *EntradaMercadoria*, que referencia obrigatoriamente uma *GuiaRemessa* (associação), as *LinhaEncomenda* recebidas (associação), o *Utilizador* que registou a entrada e a Loja onde foi efetuada. A operação de atualização de stock — descrita no passo 6 do fluxo normal do UC-08 — é delegada para o *SubStockFacade* através da interface *ISubStock*, mantendo o baixo acoplamento entre subsistemas e garantindo que a lógica de stock permanece encapsulada no subsistema responsável.

3.3 Diagramas de Sequência

Os diagramas de sequência foram elaborados com base nos casos de uso especificados no Capítulo 2, nos requisitos funcionais e no diagrama de classes. Para cada caso de uso foi selecionado o método ou fluxo de maior relevância do ponto de vista da interação entre os participantes do sistema. Por decisão metodológica, os diagramas não incluem componentes de interface de utilizador: os participantes são exclusivamente o ator, as *Facades* dos subsistemas envolvidos e as classes de domínio invocadas, tornando os diagramas diretamente rastreáveis ao

Foram selecionados três para análise detalhada nesta secção. Os diagramas escolhidos, registar venda, confirmar fecho de caixa e iniciar sincronização, formam em conjunto o ciclo diário completo de operação do sistema, abrangem os três atores principais e são os que melhor ilustram as decisões arquiteturais mais relevantes do projeto. A sua análise aprofundada permite compreender não só o funcionamento individual de cada fluxo, mas também a forma como os subsistemas colaboram entre si para suportar uma operação distribuída, autónoma e consistente.

Este diagrama modela o fluxo de negócio do caso de uso UC-02, *Registrar Venda no PDV*. O objetivo é representar a sequência de interações entre o *Funcionario*, a *SubPDVFacade*. O diagrama foca-se, portanto, na lógica essencial da operação: abertura da venda, registo das linhas, validação de stock, finalização, atualização de stock, emissão do documento fiscal e registo de auditoria.



O fluxo é iniciado pelo `Funcionario`, que solicita à `SubPDVFacade` a criação de uma nova transação através da operação `registrarVenda(lojaId, funcionarioId)`. A fachada atua como ponto de entrada do subsistema de PDV e cria uma nova instância de `Venda`, associada

à loja e ao funcionário autenticado. Esta venda representa a transação ainda em aberto, à qual serão adicionadas as linhas correspondentes aos produtos adquiridos pelo cliente.

De seguida, para cada produto lido por código de barras ou selecionado por pesquisa, o `Funcionario` invoca a operação `adicionarLinhaVenda(vendaId, produtoId, quantidade)`. A `SubPDVFacade` valida se o produto existe, se está ativo e se existe stock suficiente para a quantidade pretendida. Caso o produto não exista ou o stock seja insuficiente, o fluxo é interrompido e o funcionário é informado do erro, sem alterar a venda. Quando as validações são satisfeitas, é criada uma instância de `LinhaVenda`, contendo o produto, a quantidade e o preço unitário no momento da venda. A linha é associada à `Venda`, que recalcula os seus totais, incluindo o valor sem IVA, o IVA e o total com IVA.

O diagrama inclui também o fluxo alternativo de remoção de uma linha antes da finalização. Neste caso, o `Funcionario` solicita a anulação da linha através de `anularLinhaVenda(vendaId, linhaId)`. A venda marca essa linha como anulada e recalcula os totais, permitindo que o funcionário continue a transação com os valores corrigidos.

Quando todos os produtos estão registados, o `Funcionario` seleciona o meio de pagamento e solicita a finalização da venda. A `SubPDVFacade` percorre todas as linhas não anuladas e invoca o subsistema de stock para atualizar as quantidades disponíveis de cada produto vendido. Só depois a `Venda` é finalizada, ficando associado o meio de pagamento e os totais finais da transação.

Após a finalização, a operação é registada no subsistema de auditoria, garantindo rastreabilidade da ação efetuada pelo funcionário. A fachada desencadeia ainda a sincronização da venda de forma não bloqueante, para que a operação local do PDV não dependa da disponibilidade imediata do servidor central. Esta característica é importante para cumprir os requisitos de funcionamento em modo offline.

Por fim, o `Funcionario` solicita a emissão do recibo ou fatura através de `emitirFatura(vendaId, nifCliente, nomeCliente)`. A fachada recalcula os totais da venda, determina o tipo de documento fiscal, cria a entidade `Fatura` e invoca a sua emissão. A emissão do documento é também registada no log de auditoria através da operação `FATURA_EMITIDA`. No final do fluxo, a venda encontra-se registada, o stock atualizado, o documento fiscal emitido e a transação pronta para ser considerada no fecho de caixa do dia.

Registar Fecho de Caixa Diário

Este diagrama modela o processo de fecho diário da caixa pelo Gerente de Loja. O fluxo corresponde ao UC-05 e representa a forma como o subsistema de PDV cria o fecho, calcula automaticamente os totais das vendas do dia, permite a conferência pelo gerente, confirma o fecho e desencadeia a sincronização com o servidor central. O caso de uso satisfaz os requisitos RF-08 e RNF-05, garantindo a totalização das transações e o registo auditável da operação.

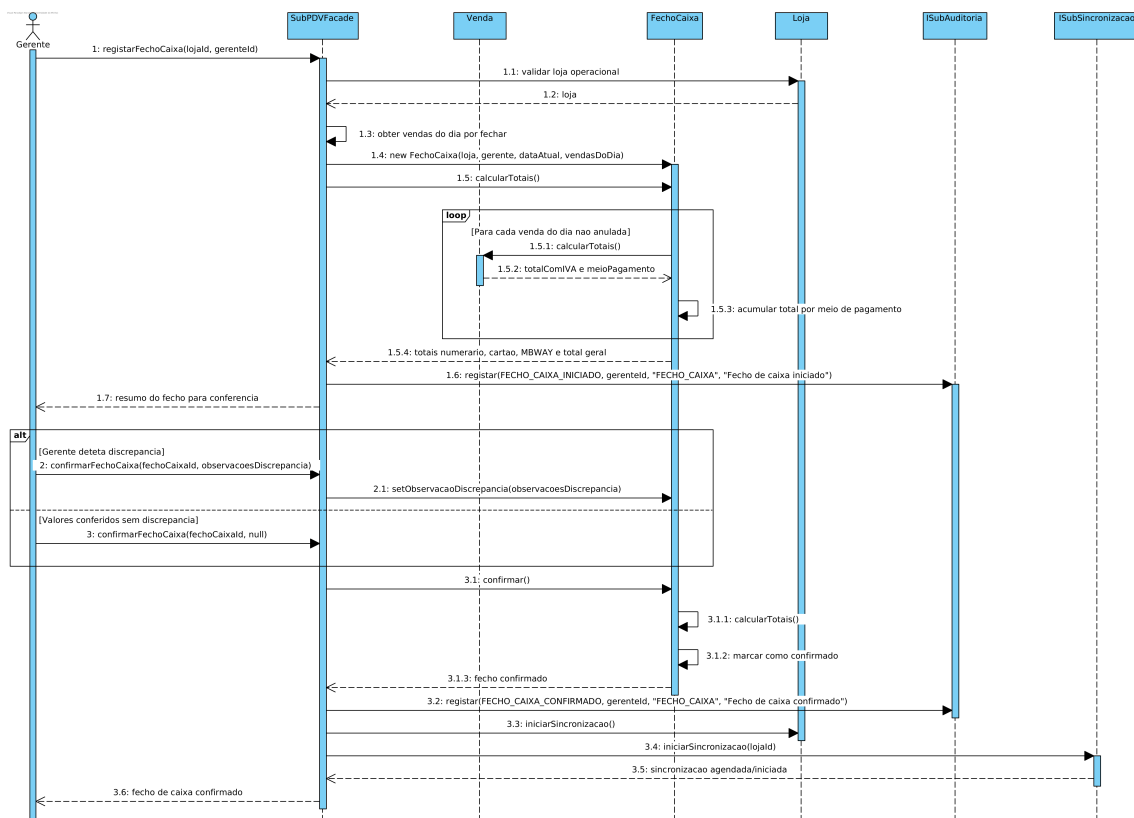


Figura 3.4: Diagrama de sequência do caso de uso UC-05 – Registrar Fecho de Caixa Diário

O fluxo é iniciado pelo Gerente, que solicita à SubPDVFacade o início do fecho de caixa através da operação `registrarFechoCaixa(lojaId, gerenteId)`. A fachada valida a loja e identifica as vendas do dia que ainda não foram incluídas num fecho anterior. Com base nessas vendas, é criada uma instância de `FechoCaixa`, associada à loja, ao gerente responsável e à data corrente.

Após a criação do fecho, a SubPDVFacade invoca `calcularTotais()` sobre a entidade `FechoCaixa`. Esta operação percorre as vendas do dia, ignora vendas anuladas e, para cada venda válida, recalcula os respetivos totais. Em seguida, acumula os valores por meio de pagamento, distinguindo numerário, cartão e MB Way, e calcula o total geral. O resultado é devolvido ao gerente sob a forma de resumo para conferência.

Nesta fase, o fecho ainda não está confirmado. O gerente pode comparar os valores apresentados pelo sistema com os valores fisicamente contados em caixa. Caso detete uma discrepância, pode fornecer uma observação através da operação `confirmarFechoCaixa(fechoCaixaId, observacoesDiscrepancia)`. Se não existir discrepância, a mesma operação é invocada sem observações. Em ambos os casos, a observação fica associada ao registo do fecho, preservando contexto para consulta posterior.

A confirmação do fecho é realizada pela própria entidade `FechoCaixa`, através do método

O processo começa com a verificação da existência de uma sincronização em estado PENDENTE. Caso não exista, a `SubSincronizacaoFacade` agenda uma nova sincronização para a loja, criando uma entidade `Sincronizacao` associada à respectiva Loja. De seguida, é construído o *payload* de sincronização, contendo os registos locais alterados desde a última sincronização concluída ou concluída com conflitos. O *payload* inclui vendas, faturas, estado de stock, ajustes de stock, fechos de caixa, entradas de mercadoria, dados agregados para relatórios e registos de auditoria.

Depois de construído o *payload*, a entidade `Sincronizacao` passa para o estado `EM_CURSO`, guardando o conteúdo serializado do *payload* e a quantidade de registos incluídos. A transmissão é então delegada ao componente `SincronizacaoTransporte`, que comunica com o servidor central através da configuração de URL central e token de sincronização.

No servidor central, o `ServidorCentralSincronizacaoService` recebe o *payload*, obtém ou cria a representação da loja e verifica conflitos através da comparação dos campos `updatedAt` e `version` dos registos recebidos com os registos já existentes no servidor central. Quando é detetada divergência, o conflito é registado com a regra aplicada, seguindo a estratégia *last-write-wins*.

Se a transmissão falhar, por exemplo por servidor central indisponível ou erro de comunicação, a sincronização local não é perdida. A entidade `Sincronizacao` volta ao estado PENDENTE, mantém o *payload* serializado, regista a mensagem de erro e define uma próxima tentativa. Desta forma, o PDV pode continuar a operar localmente e os dados permanecem disponíveis para nova tentativa de envio.

Se a transmissão for bem-sucedida e não forem detetados conflitos, a sincronização local é concluída com o estado CONCLUIDA. Caso existam conflitos, é concluída com o estado `COM_CONFLITOS`, guardando em JSON a lista de conflitos devolvida pelo servidor central e o número de conflitos resolvidos/detetados. Estes conflitos podem posteriormente ser consultados através das operações de histórico e consulta de conflitos do subsistema de sincronização.

3.4 Diagrama de Componentes

O Diagrama de Componentes apresentado na Figura 3.6 representa a arquitetura estrutural do sistema Mini-Formiga, evidenciando os principais componentes de software e as interfaces que suportam a comunicação entre eles. O diagrama demonstra a separação da aplicação em três níveis principais: a camada de apresentação, a camada de entrada da aplicação através da API REST e a camada de lógica de negócio organizada em subsistemas especializados.

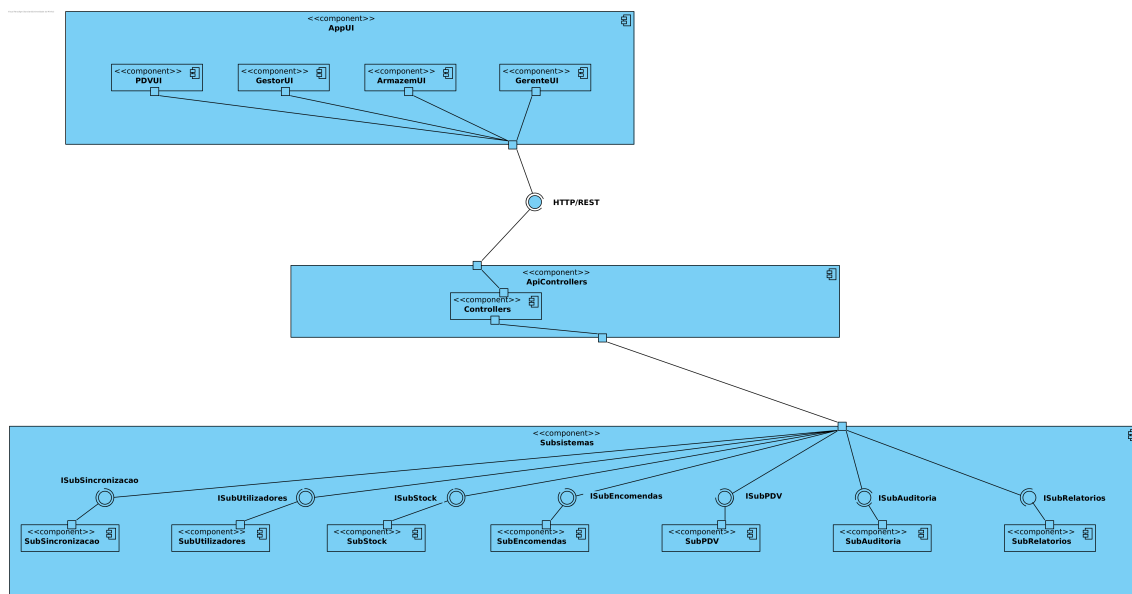


Figura 3.6: Diagrama de Componentes do sistema Mini-Formiga

A camada de apresentação é representada pelo componente *AppUI*, responsável por agregar os vários clientes da aplicação. Este componente inclui quatro interfaces distintas, correspondentes aos diferentes perfis de utilização existentes no sistema.

Estas interfaces comunicam exclusivamente com o backend através de serviços HTTP/REST, garantindo uma separação clara entre a camada de apresentação e a lógica interna da aplicação. Desta forma, os componentes de interface não acedem diretamente aos módulos de negócio, dependendo apenas da API disponibilizada pelo sistema.

A camada intermédia é representada pelo componente *ApiControllers*, que constitui o ponto de entrada da aplicação no backend. Este componente contém os vários *controllers* REST responsáveis por receber os pedidos HTTP provenientes das interfaces cliente, validar os dados recebidos e encaminhar as operações para os subsistemas apropriados da lógica de negócio.

Os *controllers* funcionam assim como adaptadores entre o protocolo REST e os serviços internos da aplicação, isolando a camada de apresentação dos detalhes de implementação dos subsistemas.

A camada de lógica de negócio é representada pelo componente *Subsistemas*, que agrega os vários módulos funcionais do sistema Mini-Formiga. Cada subsistema encapsula responsabilidades específicas do domínio da aplicação e é disponibilizado através de uma interface própria, utilizada diretamente pelos *controllers* da API.

Conforme representado no diagrama, os *controllers* da API comunicam diretamente com as interfaces dos respectivos subsistemas. Não existe um componente central agregador da lógica de negócio, sendo a coordenação funcional distribuída pelos vários subsistemas especializados. Esta abordagem reduz o acoplamento entre módulos e favorece uma arquitetura mais modular e extensível.

A utilização de interfaces explícitas para cada subsistema permite desacoplar a camada de API das implementações concretas da lógica de negócio. Assim, alterações internas a um subsistema podem ser realizadas sem impacto direto nos restantes componentes, desde que o contrato definido pela interface seja preservado.

A separação entre interfaces de utilizador, controladores REST e subsistemas de negócio contribui para uma arquitetura coerente, modular e de fácil manutenção. Além disso, a organização da lógica por subsistemas independentes favorece a evolução incremental da aplicação, bem como a realização de testes isolados sobre cada módulo funcional.

3.5 Definição da Stack Tecnológica

A seleção da *stack* tecnológica foi realizada antes do início da implementação e condicionou diretamente as decisões arquiteturais documentadas na Secção 3.9. Para cada área tecnológica foram identificadas pelo menos duas alternativas viáveis, avaliadas segundo três critérios:

1. adequação aos requisitos funcionais e não funcionais do sistema;
2. maturidade e estabilidade da tecnologia em contexto empresarial;
3. compatibilidade com o modelo distribuído adotado (operação *offline* por loja + servidor central).

Linguagem e *Framework* de *Backend*

Opção	Vantagens	Desvantagens	Decisão
Java 21 + Spring Boot 3	Ecossistema maduro, Spring Data JPA abstrai a persistência, suporte nativo a múltiplos perfis de configuração (local/central), vasto conjunto de bibliotecas empresariais.	Arranque mais lento comparado com alternativas reativas; verbosidade do código Java em relação a Kotlin.	Adotado
Python + FastAPI	Rápido de prototipar, sintaxe concisa, ecossistema de dados rico.	Menor robustez para sistemas de gestão empresarial de longa duração; digitação opcional pode introduzir erros em runtime.	Rejeitado
Node.js + NestJS	Performance assíncrona, partilha de código com o <i>frontend</i> React.	Modelo de concorrência menos intuitivo para operações intensivas em BD; menos maduro para sistemas distribuídos com requisitos de auditoria.	Rejeitado

Tabela 3.1: Comparação de alternativas para o *backend*

A escolha de **Java 21 com Spring Boot 3.3.5** justifica-se pela produtividade conferida pelas abstrações do ecossistema Spring (*Spring Web*, *Spring Data JPA*, *Spring Security*), pela robustez comprovada em ambientes empresariais e pelo suporte nativo a perfis de configuração distintos para a base de dados local (SQLite) e central (PostgreSQL), sem alteração de código. O **Maven** foi adotado como ferramenta de construção pela uniformidade que confere ao processo de compilação, execução de testes e geração de artefactos entre todos os elementos da equipa.

Linguagem e *Framework* de *Frontend*

Opção	Vantagens	Desvantagens	Decisão
React 18 + TypeScript + Vite	Componentes reutilizáveis, TypeScript elimina erros em <i>runtime</i> , Vite proporciona <i>hot reload</i> instantâneo, ecossistema maduro.	Curva de aprendizagem inicial do TypeScript; maior <i>boilerplate</i> comparado com Vue.	Adotado
Vue 3 + Vite	Sintaxe mais simples, menor <i>boilerplate</i> , documentação excelente.	Ecossistema ligeiramente menor; menor familiaridade da equipa.	Rejeitado
Angular 17	<i>Framework</i> completo com <i>opinionated structure</i> , TypeScript nativo, ideal para grandes equipas.	Curva de aprendizagem elevada; complexidade excessiva para o âmbito do projeto.	Rejeitado

Tabela 3.2: Comparação de alternativas para o *frontend*

A adoção de **React 18 com TypeScript** justifica-se pelo paradigma baseado em componentes, que se adequa à construção de interfaces distintas por perfil de utilizador, e pelo suporte de **TypeScript** à deteção precoce de erros e à refatoração segura. O **Vite** foi selecionado como ferramenta de *build* pela velocidade de arranque e atualização, essencial num ciclo de desenvolvimento iterativo. O estilo visual é assegurado por **Tailwind CSS**, que permite a construção rápida de interfaces responsivas sem CSS personalizado extenso.

Base de Dados

Contexto	Opção	Vantagens	Desvantagens	Decisão
BD Local	SQLite	Sem servidor dedicado, ficheiro único, operação <i>embedded</i> , sem configuração de rede, ideal para cenários <i>offline</i> .	Concorrência limitada em escritas simultâneas; não adequado para consolidação central.	Adotado
BD Local	H2 (em memória)	Leve, suportado nativamente pelo Spring Boot para testes.	Não persiste dados entre reinícios; não adequado para produção.	Rejeitado
BD Central	PostgreSQL	ACID completo, suporte a JSON, escalabilidade, maturidade comprovada, suporte a <i>optimistic locking</i> via versioning nativo.	Requer instalação e administração de servidor.	Adotado
BD Central	MySQL / MariaDB	Largamente adotado, boa performance em leituras.	Menor riqueza de tipos de dados e menor suporte a operações analíticas comparado com PostgreSQL.	Rejeitado

Tabela 3.3: Comparação de alternativas para persistência de dados

A combinação **SQLite** (local) com **PostgreSQL** (central) foi a escolha natural dado o modelo operacional do sistema: a loja necessita de uma base de dados *embedded*, sem dependência de servidor, que funcione de forma autónoma; o servidor central necessita de uma base de dados robusta, transacional e capaz de consolidar dados de múltiplas lojas com garantias ACID.

Arquitetura de Lógica de Negócio: Monolítico vs. Micro-serviços

Opção	Vantagens	Desvantagens	Decisão
Monolítico modular (Facade por subsistema)	O sistema Mini-Formiga opera em até 5 lojas, cada uma com um servidor local que necessita de funcionar de forma autónoma e offline. Um único processo executável por loja elimina a necessidade de orquestração entre serviços, simplifica a instalação em contexto de loja de conveniência e garante que qualquer técnico consegue manter o sistema sem conhecimentos avançados de infraestrutura distribuída. A separação de responsabilidades é assegurada pelos contratos de interface entre subsistemas.	A escalabilidade horizontal está limitada ao número de lojas da cadeia, que no horizonte do projeto é de 5 unidades — um volume que não justifica complexidade adicional.	Adotado
Micro-serviços	Escalabilidade independente por serviço em cenários de alto volume transacional.	Cada loja necessitaria de orquestrar múltiplos serviços independentes localmente, tornando a instalação, manutenção e operação offline incompatíveis com o perfil técnico dos colaboradores da cadeia Mini-Formiga. A gestão de falhas parciais entre serviços numa loja de conveniência sem equipa de TI dedicada representa um risco operacional inaceitável.	Rejeitado

Tabela 3.4: Comparação entre arquitetura monolítica modular e micro-serviços

A decisão de adotar uma **arquitetura monolítica modular** foi alinhada com o contexto operacional do Mini-Formiga: cada loja executa uma instância local simples, autónoma e fácil de manter. A fundamentação completa desta decisão é apresentada na Secção 3.9, em DA-01.

Infraestrutura e Ferramentas de Apoio

- **Docker Compose:** orquestração dos serviços de desenvolvimento (PostgreSQL central, *frontend*) para garantir reprodutibilidade do ambiente entre elementos da equipa.
- **JUnit 5 + JaCoCo:** testes unitários e medição de cobertura de código com limiares mínimos definidos no processo de construção.
- **Logback + SLF4J:** persistência dos logs de auditoria em ficheiros JSON rotativos, sem impacto na base de dados operacional (DA-07).

3.6 Especificação de API

Os *endpoints* da API REST encontram-se organizados por recurso, conforme as tabelas seguintes. A API é disponibilizada pelo backend Spring Boot sob o prefixo `/api/v1`. No perfil local, a aplicação é normalmente executada em `http://localhost:8080/api/v1`. No perfil central, os endpoints específicos de sincronização central usam o mesmo prefixo, podendo ser disponibilizados pelo endereço configurado para o servidor central. A especificação da API foi documentada com recurso a Swagger/OpenAPI, permitindo consultar de forma interativa os endpoints disponíveis, os métodos HTTP suportados, os parâmetros esperados, os códigos de resposta e os esquemas dos objetos de entrada e saída. Esta documentação serviu também como mecanismo de validação durante o desenvolvimento, facilitando a confirmação do comportamento exposto por cada controlador e a realização de testes manuais sobre os principais fluxos funcionais.

As convenções gerais aplicadas são as seguintes:

- **Base URL local:** `http://localhost:8080/api/v1`
- **Base URL central:** `https://central.mini-formiga.pt/api/v1`
- **Formato:** `Content-Type: application/json`
- **Autenticação:** token JWT enviado como Bearer Token
- **Erros:** estrutura uniforme `{"code": "...", "message": "...", "details": {...}}`
- **Paginação:** parâmetros `page`, `size` e `sort` nos endpoints que devolvem páginas

Autenticação

Método	Endpoint	Autorização	Descrição
POST	/auth/login	Público	Autentica o utilizador e devolve um token JWT.
POST	/auth/logout	Utilizador autenticado	Regista auditoria do logout. Como a autenticação é JWT/stateless, não invalida sessão no servidor.

Tabela 3.5: Endpoints – Autenticação

Utilizadores

Método	Endpoint	Autorização	Descrição
GET	/utilizadores	GLOBAL_ADMIN, UTILIZADORES_READ	Lista utilizadores, com paginação e filtro opcional por loja.
GET	/utilizadores/perfis	GLOBAL_ADMIN, UTILIZADORES_READ, UTILIZADORES_WRITE	Lista os perfis disponíveis.
GET	/utilizadores/lojas	GLOBAL_ADMIN, UTILIZADORES_READ, UTILIZADORES_WRITE	Lista lojas disponíveis para associação a utilizadores.
GET	/utilizadores/{id}	GLOBAL_ADMIN, UTILIZADORES_READ	Devolve os dados de um utilizador.
POST	/utilizadores	GLOBAL_ADMIN, UTILIZADORES_WRITE	Cria uma nova conta de utilizador.
PUT	/utilizadores/{id}	GLOBAL_ADMIN, UTILIZADORES_WRITE	Atualiza dados, perfil, loja, palavra-passe ou estado de um utilizador.
DELETE	/utilizadores/{id}	GLOBAL_ADMIN, UTILIZADORES_DELETE	Desativa logicamente uma conta.

Tabela 3.6: Endpoints – Utilizadores

Produtos, Categorias e Meios de Pagamento

Método	Endpoint	Autorização	Descrição
GET	/produtos	GLOBAL_ADMIN, PDV_WRITE, STOCK_WRITE, RELATOS_READ	Lista produtos, com paginação e filtro opcional por loja.
POST	/produtos	GLOBAL_ADMIN, STOCK_WRITE	Cria um novo produto.
GET	/produtos/{id}	GLOBAL_ADMIN, PDV_WRITE, STOCK_WRITE, RELATOS_READ	Obtém produto por identificador.
GET	/produtos/barcode/{codigo}	GLOBAL_ADMIN, PDV_WRITE, STOCK_WRITE, RELATOS_READ	Obtém produto por código de barras.
PUT	/produtos/{id}	GLOBAL_ADMIN, STOCK_WRITE	Atualiza dados de um produto.
GET	/produtos/{id}/localizacao	STOCK_WRITE, ARMAZEM	Obtém a localização de um produto numa loja.
PUT	/produtos/{id}/localizacao	STOCK_WRITE, ARMAZEM	Atualiza corredor e prateleira de um produto.
GET	/categorias	GLOBAL_ADMIN, PDV_WRITE, STOCK_WRITE, RELATOS_READ	Lista categorias de produtos.
GET	/meios-pagamento	GLOBAL_ADMIN, PDV_WRITE, STOCK_WRITE, RELATOS_READ	Lista os meios de pagamento disponíveis.

Tabela 3.7: Endpoints – Produtos, Categorias e Meios de Pagamento

Vendas, Faturas, Devoluções e Fechos de Caixa

Método	Endpoint	Autorização	Descrição
POST	/vendas	PDV_WRITE	Inicia uma nova venda no PDV.
POST	/vendas/{id}/linhas	PDV_WRITE	Adiciona uma linha a uma venda.
DELETE	/vendas/{id}/linhas/{linhaId}	PDV_WRITE	Remove ou anula uma linha da venda.
POST	/vendas/{id}/finalizar	PDV_WRITE	Finaliza uma venda com o meio de pagamento indicado.
POST	/vendas/{id}/anular	PDV_WRITE, STOCK_WRITE	Anula uma venda.
GET	/vendas/{id}	GLOBAL_ADMIN, PDV_WRITE, RELATORIOS_READ	Obtém o detalhe de uma venda.
GET	/vendas	GLOBAL_ADMIN, PDV_WRITE, RELATORIOS_READ	Lista vendas por loja, período e estado de fecho.
POST	/vendas/{id}/fatura	PDV_WRITE	Emitte fatura associada a uma venda.
GET	/vendas/faturas	GLOBAL_ADMIN, PDV_WRITE, RELATORIOS_READ	Lista faturas de uma loja, com pesquisa opcional por cliente.
GET	/vendas/faturas/{id}	GLOBAL_ADMIN, PDV_WRITE, RELATORIOS_READ	Obtém uma fatura por identificador.
GET	/vendas/faturas/{id}/documento	GLOBAL_ADMIN, PDV_WRITE, RELATORIOS_READ	Gera documento fiscal em PDF.
GET	/vendas/faturas/numero	GLOBAL_ADMIN, PDV_WRITE, RELATORIOS_READ	Obtém uma fatura pelo número fiscal.
POST	/vendas/{id}/devolucao	PDV_WRITE	Processa uma devolução e atualiza o estado da venda.
GET	/vendas/devolucoes	GLOBAL_ADMIN, PDV_WRITE, RELATORIOS_READ	Lista devoluções registadas numa loja.
POST	/fechos-caixa	STOCK_WRITE	Regista um fecho de caixa.

(continua na página seguinte)

Método	Endpoint	Autorização	Descrição
POST	/fechos-caixa/{id}/confirmar	STOCK_WRITE	Confirma um fecho de caixa.
GET	/fechos-caixa	GLOBAL_ADMIN, RE-LATORIOS_READ	Lista fechos de caixa por loja.
GET	/fechos-caixa/{id}	GLOBAL_ADMIN, RE-LATORIOS_READ	Obtém o detalhe de um fecho de caixa.

Tabela 3.8: Endpoints – Vendas, Faturas, Devoluções e Fechos de Caixa

3.7 Esboço das Interfaces do Sistema

Nesta secção apresentam-se os principais mockups desenvolvidos para o sistema *Mini-Formiga*. As interfaces foram organizadas por perfil de utilizador e por módulo funcional, de forma a refletir a estrutura do sistema definida nos requisitos. O objetivo destes esboços é ilustrar a organização visual das funcionalidades principais, evidenciando a adaptação da interface às responsabilidades de cada utilizador e à natureza das operações realizadas.

As decisões de design assentam num conjunto de princípios transversais a todo o sistema. Foi adotado um tema escuro (*dark mode*) como base visual, motivado pela natureza do contexto operacional: o sistema é utilizado maioritariamente em ambientes de loja e armazém, onde os ecrãs permanecem ativos durante longos períodos. O tema escuro reduz a fadiga visual dos utilizadores, melhora a legibilidade em condições de iluminação variável e confere uma identidade visual coesa a todo o sistema. A paleta cromática complementa esta escolha com acentos a azul para elementos de navegação e ação primária, vermelho e laranja para alertas e situações críticas, e verde para indicadores positivos, criando um sistema de cores semântico que comunica o estado da informação sem necessidade de leitura textual detalhada.

A tipografia foi pensada para estabelecer uma hierarquia visual clara: valores numéricos relevantes surgem em destaque com corpo maior, enquanto os rótulos descritivos adotam um peso menor e uma dimensão reduzida. Esta separação permite ao utilizador identificar rapidamente a informação mais relevante em cada ecrã, sem que elementos secundários disputem a atenção visual.

3.7.1 Autenticação e seleção de perfil

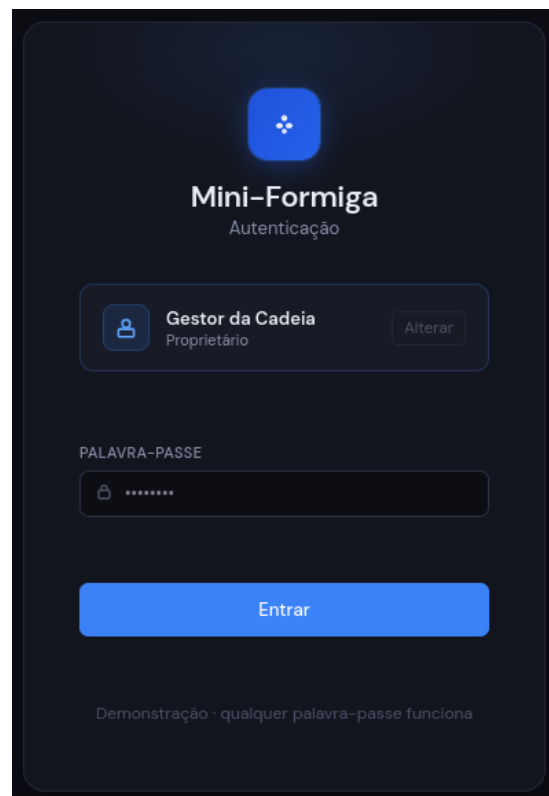
As primeiras interfaces representam o processo de entrada no sistema e materializam uma decisão arquitetural fundamental: a separação entre a seleção do perfil e a autenticação propriamente dita. Esta abordagem em dois passos foi adotada deliberadamente em alternativa a um formulário de *login* tradicional com campos de utilizador e palavra-passe simultâneos.

A interface de seleção de perfil (Figura 3.7a) apresenta os quatro perfis disponíveis no sistema: Gestor da Cadeia, Gerente de Loja, Funcionário e Responsável de Armazém, sob a forma de cartões. Cada cartão agrega o ícone identificativo do papel, o nome do perfil e um subtítulo contextual que indica o posto ou a loja associada. Esta organização em grelha simétrica de dois por dois facilita a leitura e a seleção, mesmo em ecrãs de dimensão reduzida. A presença de uma seta de navegação em cada cartão reforça a percepção de que a interação é possível, guiando o utilizador de forma intuitiva. O destaque cromático diferenciado de cada ícone por perfil: azul para o Gestor, verde para o Gerente, vermelho para o Funcionário e amarelo para o Responsável de Armazém, permite uma distinção visual imediata que se prolonga pelas restantes interfaces do sistema, criando continuidade de identidade entre o perfil selecionado e o contexto de utilização subsequente.

Após a seleção do perfil, a interface de autenticação (Figura 3.7b) apresenta unicamente o campo de palavra-passe, mantendo visível o perfil escolhido e oferecendo a possibilidade de o alterar. Esta decisão de design serve dois propósitos: por um lado, confirma ao utilizador o contexto em que irá operar, reduzindo erros de acesso; por outro, simplifica o formulário ao mínimo necessário, o que é particularmente relevante para perfis como o Funcionário de PDV, que necessita de aceder rapidamente ao sistema no início do turno. A concentração do ecrã num único campo de entrada e num botão de ação principal elimina distrações e acelera o processo de autenticação.



(a) Seleção do perfil de utilizador



(b) Autenticação do utilizador

3.7.2 Interfaces do Gestor da Cadeia

As interfaces do Gestor da Cadeia foram concebidas para disponibilizar uma visão global da operação do sistema. Estas vistas privilegiam o acompanhamento consolidado do desempenho das lojas, a consulta de relatórios, a gestão de fornecedores, o controlo de encomendas, a administração de utilizadores e a monitorização do estado de sincronização entre as lojas e o servidor central.

A estrutura de navegação adotada para este perfil recorre a uma barra lateral fixa (*sidebar*), visível em todas as interfaces do gestor. Esta escolha contrasta deliberadamente com a ausência de navegação persistente nas interfaces do Funcionário de PDV, onde a simplicidade e o foco na tarefa imediata são prioritários. Para o Gestor da Cadeia, a *sidebar* disponibiliza acesso permanente aos seis módulos principais do sistema: *Dashboard*, Relatórios, Fornecedores, Encomendas, Utilizadores e Sincronização, sem que seja necessário regressar a um ecrã intermédio. Esta decisão reduz a profundidade de navegação e permite ao gestor transitar rapidamente entre contextos distintos de análise e gestão.

Dashboard

O *dashboard* constitui o ponto de entrada do Gestor da Cadeia e foi desenhado em torno de um princípio central: permitir a leitura do estado global da operação num único olhar, sem necessidade de navegar para outros módulos para obter uma visão de conjunto.

A secção superior do ecrã agrupa quatro indicadores-chave de desempenho (*KPIs*) sob a forma de cartões individuais: o volume de vendas do dia, a margem média, o número de lojas ativas e o número de alertas de stock. Esta disposição horizontal segue uma lógica de prioridade decrescente da esquerda para a direita, do indicador financeiro mais imediato para o operacional, e permite ao gestor quantificar rapidamente o estado do negócio. Cada cartão inclui um indicador de tendência ou estado secundário (por exemplo, a variação percentual face ao dia anterior, ou a indicação de que todas as lojas estão operacionais), acrescentando profundidade informativa sem aumentar a complexidade visual.

A secção intermédia divide-se em dois painéis complementares: um gráfico de barras com as vendas por loja no dia corrente e uma tabela de desempenho por loja com os valores de vendas e margem. A coexistência destas duas representações é intencional, o gráfico oferece uma comparação visual e imediata entre lojas, enquanto a tabela disponibiliza os valores exatos para uma análise mais rigorosa. O gestor pode assim optar pela representação que melhor serve o seu propósito em cada momento, sem necessidade de alternar entre ecrãs.

A secção inferior é dedicada aos alertas de stock, apresentando em lista os produtos abaixo do mínimo definido, com indicação da loja afetada, da quantidade atual e do limiar configurado. A decisão de incluir esta informação diretamente no *dashboard*, em lugar de a relegar exclusivamente para o módulo de stock, reflete a importância operacional destes alertas: situações de rutura de stock têm impacto direto nas vendas e requerem ação imediata, pelo que a sua

visibilidade imediata foi considerada prioritária.

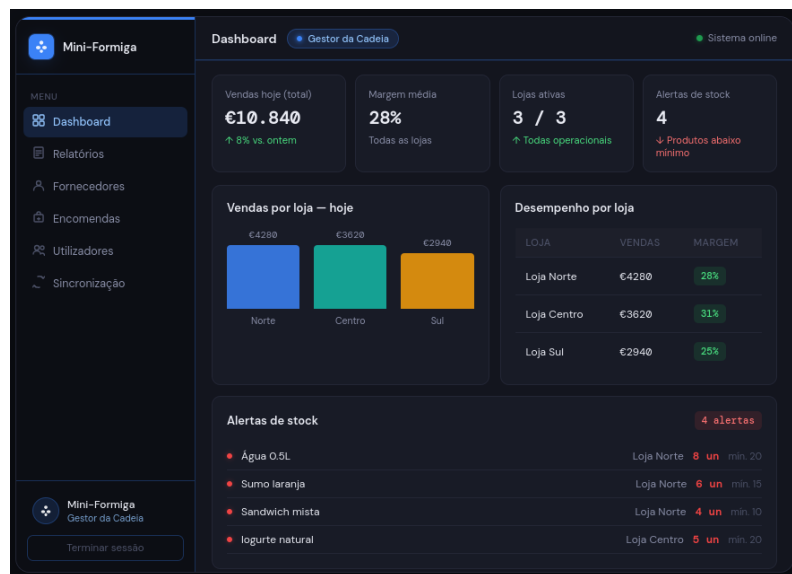


Figura 3.8: Mockup do *dashboard* do Gestor da Cadeia

As restantes interfaces do sistema encontram-se documentadas no Anexo 6.3.

3.8 Conceção do Modelo da Persistência de Dados

Concluída a especificação da API a equipa procedeu à conceção do modelo de persistência de dados. Esta fase teve como ponto de partida os casos de uso especificados na etapa anterior, os requisitos funcionais e não funcionais identificados, os requisitos de domínio que impõem invariantes sobre os dados, e o modelo de domínio. O resultado é um modelo lógico relacional organizado em duas variantes: um modelo global, orientado ao servidor central, e um modelo local, orientado a cada loja.

A escolha de bases de dados relacionais como tecnologia de persistência do projeto *Mini-Formiga* não foi tomada por omissão, mas decorre de propriedades específicas do domínio. O sistema opera sobre dados com elevada integridade referencial entre entidades, por exemplo uma *linhas_venda* só existe no contexto de uma *venda*, que por sua vez exige um *utilizador_id* e um *loja_id* válidos. Estes impõem invariantes transacionais fortes, como a proibição de stock negativo (RD-04) e a unicidade do número sequencial de faturas por série (RD-01 a RD-03). Estes requisitos beneficiam diretamente das garantias ACID que os sistemas relacionais fornecem de forma nativa, garantias que bases de dados orientadas a documentos (e.g., MongoDB) ou de chave-valor (e.g., Redis) apenas oferecem de forma parcial ou com complexidade acrescida para o programador.

Adicionalmente, o modelo de dados exibe um esquema estável e bem definido desde a fase

de análise, com relações claras e tipadas entre entidades. Num sistema de gestão de retalho, alterar o esquema em produção é uma operação de custo elevado independentemente da tecnologia escolhida; a rigidez do esquema relacional constitui, neste contexto, uma vantagem, pois obriga à validação estrutural dos dados na camada de persistência e não apenas na lógica aplicacional. Bases de dados orientadas a grafos (e.g., Neo4j) seriam adequadas se o domínio central fosse a navegação em redes de relações arbitrárias, o que não é o caso.

Utilização de base de dados local e central

A decisão de utilizar duas tecnologias distintas de base de dados relacional resulta de requisitos operacionais contraditórios entre o nível de loja e o nível de cadeia.

Cada loja opera com uma instância local SQLite que suporta o funcionamento do PDV (UC-02), o registo de fechos de caixa (UC-05) e a gestão de stock (UC-07, UC-08, UC-10, UC-11). O requisito não funcional RNF-02 exige que o PDV continue a funcionar em modo *offline* quando a ligação ao servidor central é interrompida, o que torna impraticável uma arquitetura de base de dados puramente centralizada. O SQLite é uma escolha adequada para este cenário porque é uma biblioteca embebida sem processo servidor separado, eliminando a necessidade de administração de infraestrutura na loja, e porque o volume de dados concorrentes num único PDV é reduzido, não exigindo as capacidades de paralelismo que justificariam o PostgreSQL a nível local.

Embora cada instância local esteja associada operacionalmente a uma única loja, a tabela lojas é mantida também na base de dados local. Esta decisão permite reutilizar o mesmo modelo de domínio e a mesma camada de persistência nos perfis local e central, garantindo que entidades como vendas, utilizadores, encomendas, inventarios_fisicos, entradas_mercadoria, alertas_stock e sincronizacoes mantêm sempre uma referência explícita à loja de origem. No contexto local, esta tabela funciona sobretudo como identificação da instância da loja e como contexto para sincronização, permissões, filtragem e auditoria, não como um catálogo operacional de todas as lojas da cadeia.

O servidor central agrega os dados de todas as lojas sincronizadas (UC-13) e serve de base ao módulo de *dashboard* e relatórios consultado pelo Gestor da Cadeia (UC-06). PostgreSQL suporta múltiplas ligações concorrentes de todas as lojas durante a sincronização, oferece o tipo *jsonb* com indexação para os campos *payload_json* e *conflitos_json* do registo de sincronizações, e dispõe de capacidades analíticas (janelas, agregações, exportação CSV nativa) que suportam os requisitos RF-01 e RF-02. A utilização de *uuid* como chave primária em todas as entidades facilita a fusão de registos provenientes de lojas distintas sem colisão de identificadores, um requisito implícito da sincronização *last-write-wins* definida em UC-13.

Logs de Auditoria em Ficheiros JSON

Vários casos de uso impõem o registo de ações no *log* de auditoria, nomeadamente UC-01 (autenticação e tentativas falhadas), UC-02 (transações de venda), UC-04 (devoluções), UC-05 (fechos de caixa), UC-10 (ajustes de inventário) e UC-12 (gestão de utilizadores). Optou-se por persistir estes registos em ficheiros JSON em vez de os armazenar nas bases de dados relacionais, por três razões fundamentais.

Em primeiro lugar, os *logs* de auditoria têm semântica de **escrita única**: são criados no momento do evento e nunca alterados ou eliminados, pelo que não beneficiam das capacidades de atualização transacional que as bases de dados relacionais fornecem. Em segundo lugar, o volume de entradas de auditoria pode crescer de forma descontrolada ao longo do tempo, e a sua inclusão na base de dados operacional competiria com as operações OLTP por recursos de I/O e espaço em disco, degradando o desempenho do PDV em violação do requisito RNF-01. Em terceiro lugar, o formato JSON permite registar o contexto completo de cada evento (utilizador, entidade afetada, valores anteriores e posteriores, *timestamp*), sem impor um esquema fixo, o que é vantajoso porque diferentes tipos de eventos têm estruturas de contexto distintas. Os ficheiros são estruturados por data e tipo de evento, permitindo a sua ingestão futura numa ferramenta de análise de *logs* (e.g., ELK Stack) sem necessidade de migração de esquema.

A passagem do modelo conceptual ao modelo lógico relacional envolveu decisões de representação que se ilustram de seguida com dois exemplos de entidades convertidas em tabelas e um exemplo de conceito que não originou uma tabela independente.

Consequentemente, a auditoria não é representada no modelo lógico relacional por uma tabela própria. A versão atual da aplicação não mantém uma entidade LogAuditoria persistida na base de dados, nem um repositório associado. Os eventos são registados diretamente pelo subsistema de auditoria através de ficheiros JSONL, sendo posteriormente incluídos no processo de sincronização como entradas textuais do *log*. Esta opção evita misturar dados operacionais transacionais com dados de auditoria, que têm natureza essencialmente append-only e crescimento contínuo.

Entidade produtos

A entidade *Produto* é uma das mais centrais do sistema e foi convertida em tabela com atributos diretos como *codigo_barras*, *preco_venda*, *preco_custo* e *ativo*. No perfil global/central, a gestão efetiva do stock por loja é realizada através da tabela *produtos_lojas*, que associa cada produto a uma loja e guarda a quantidade em stock, o nível mínimo e a localização física nessa loja. A tabela *produtos* mantém os dados de catálogo e, por compatibilidade com o perfil local, conserva também campos de stock local. Esta decisão foi motivada diretamente por **UC-07** (*Consultar Stock e Receber Alertas de Reposição*), que exige a consulta e configuração de níveis mínimos por produto por loja, e por **UC-10** (*Registar Ajuste Manual de Inventário*), que impõe a rastreabilidade de qualquer alteração ao stock com identi-

ficação do responsável. A relação com `categorias` e `taxas_iva` foi normalizada em tabelas de referência separadas para permitir a filtragem por categoria exigida em UC-06 e o cálculo de IVA discriminado por taxa exigido em UC-02 e UC-03.

Entidade vendas

A entidade *Venda* foi convertida na tabela `vendas` com os atributos de cabeçalho (`meio_pagamento`, `data_hora`, `anulada`), tendo os artigos individuais sido normalizados na tabela `linhas_venda`. Esta separação decorre diretamente de **UC-02** (*Registar Venda no PDV*), cujo fluxo normal prevê a adição iterativa de linhas de produto e a remoção individual de linhas antes da finalização (fluxo alternativo). Uma representação monolítica da venda — por exemplo, como um documento JSON numa única coluna — impossibilitaria a verificação de stock linha a linha (fluxo de exceção 2 do UC-02) e a anulação seletiva de linhas no contexto de devoluções parciais (UC-04). O campo `version` e `updated_at` em `vendas` suportam o controlo de concorrência otimista necessário à sincronização definida em **UC-13**, onde o servidor central deve detectar conflitos de dados provenientes da mesma loja.

Conceito perfil como *enum*

O conceito de *Perfil de Utilizador* — que na fase de análise aparecia como uma entidade candidata com atributos próprios (nome do perfil, conjunto de permissões) — não foi convertido numa tabela independente, tendo sido representado como o tipo enumerado `perfil_utilizador` com os valores `GESTOR`, `GERENTE`, `FUNCIONARIO` e `RESPONSAVEL_ARMAZEM`, diretamente no campo `perfil` da tabela `utilizadores`. Esta decisão foi motivada por **UC-01** (*Autenticar no Sistema*) e **UC-12** (*Gerir Utilizadores e Acessos*): em ambos os casos, o perfil é utilizado exclusivamente para determinar o nível de acesso e o painel apresentado ao utilizador, sem que haja qualquer requisito de configuração dinâmica de permissões por perfil. Uma tabela `perfis` com chave estrangeira em `utilizadores` introduziria complexidade sem benefício funcional, dado que os quatro perfis são entidades fechadas definidas nos requisitos e não se prevê a criação de novos perfis sem alteração do próprio sistema. A representação como *enum* garante ainda validação a nível da base de dados, impedindo a atribuição de um perfil não reconhecido.

De seguida encontram-se os modelos lógicos das bases de dados local e global respetivamente:

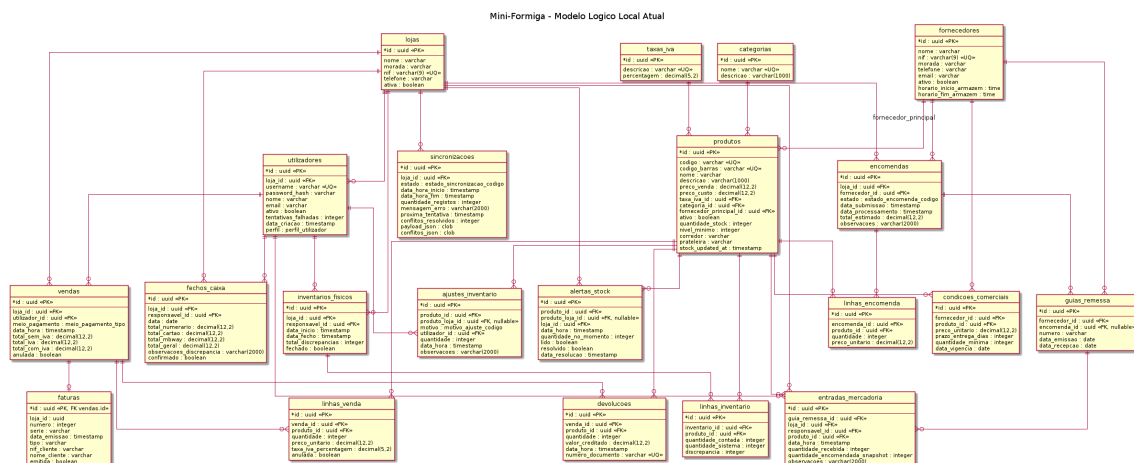


Figura 3.9: Modelo lógico da base de dados local do sistema Mini-Formiga

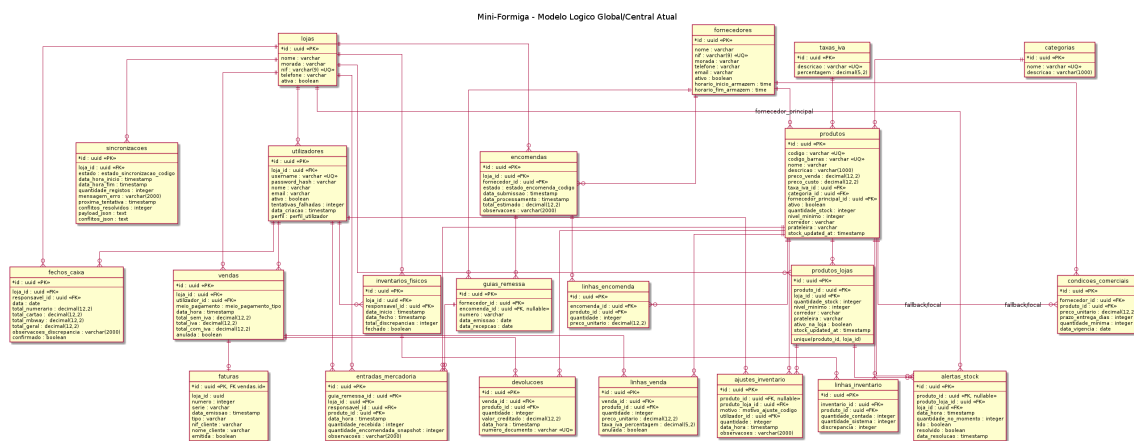


Figura 3.10: Modelo lógico da base de dados global do sistema Mini-Formiga

3.9 Decisões Arquiteturais

A concepção do sistema Mini-Formiga implicou um conjunto de decisões arquiteturais com impacto direto na maneabilidade, escalabilidade e robustez da solução. Importa salientar que a opção por uma arquitetura monolítica modular (DA-01) em detrimento de micro-serviços não constitui uma simplificação por limitação técnica ou académica, mas uma decisão deliberada e fundamentada, documentada na Tabela 3.4 da secção anterior. A separação de responsabilidades é garantida pela estrutura de subsistemas com interfaces explícitas, tornando a eventual migração para serviços independentes viável sem reescrita da lógica de domínio. As restantes decisões documentadas nesta secção derivam direta ou indiretamente desta escolha arquitetural central.

DA-01 — Padrão Facade por Subsistema em detrimento de Microserviços

O sistema Mini-Formiga destina-se a uma cadeia de até 5 lojas de conveniência, cada uma operada por funcionários sem formação técnica especializada e sem equipa de TI dedicada. Este contexto define de forma determinante a escolha arquitetural: o sistema tem de ser instalável, operável e mantível por pessoas cuja competência central é o atendimento ao cliente, não a gestão de infraestrutura de software. Uma arquitetura de micro-serviços implicaria que cada loja executasse e orquestrasse localmente múltiplos serviços independentes — PDV, stock, sincronização, autenticação — com comunicação entre eles, deteção de falhas parciais e recuperação automática. Em caso de falha de um serviço numa loja, o diagnóstico e resolução exigiriam competências técnicas que os colaboradores do Sr. Formiga não possuem nem deveriam necessitar de possuir. Numa loja de conveniência, uma falha do sistema durante o período de atendimento tem impacto direto e imediato na receita e na satisfação dos clientes.

A arquitetura monolítica modular, implementada através do padrão Facade por subsistema, resolve este problema de raiz: cada loja executa um único processo Java, sem dependências entre serviços locais, sem orquestração e sem pontos de falha adicionais. A separação de responsabilidades é garantida pelos contratos de interface entre os subsistemas existentes, o que preserva a coesão interna do sistema sem expor essa complexidade à operação diária. Em caso de necessidade futura de migração para serviços independentes, os contratos já definidos tornam essa transição viável sem necessidade de reescrita da lógica de domínio.

Por fim, importa sublinhar que o volume transacional esperado para uma cadeia de 5 lojas de conveniência não justifica, por si só, qualquer investimento em escalabilidade horizontal.

DA-02 — Sincronização por Lote (Fim de Jornada) em detrimento de Sincronização em Tempo Real

A sincronização contínua via *WebSockets* foi considerada como alternativa, por permitir que o servidor central disponha de dados atualizados em tempo real. Contudo, esta abordagem exigiria uma ligação de rede permanentemente estável entre cada loja e o servidor central, o que contraria diretamente o requisito RNF-02, que impõe disponibilidade local de pelo menos 99,5% do tempo, independentemente do estado da ligação à rede.

A equipa optou por uma sincronização em lote, desencadeada automaticamente no final de cada jornada após o registo do fecho de caixa, podendo existir uma tentativa oportunista não bloqueante após operações críticas conforme especificado no UC-13. Esta decisão reduz a carga no servidor central, torna o sistema resiliente a falhas temporárias de conectividade e alinha-se com o modelo de operação do negócio, em que a consolidação de dados no final do dia é suficiente para os processos de gestão e supervisão do Sr. Formiga. Os registos locais são mantidos de forma persistente até confirmação da sincronização bem-sucedida, garantindo que nenhuma transação é perdida em caso de falha de rede.

DA-03 — REST API com Serialização JSON em detrimento de gRPC

O protocolo gRPC foi avaliado pela sua eficiência em comunicações de alto débito entre serviços, nomeadamente na fase de sincronização entre os terminais de loja e o servidor central. Apesar das vantagens de desempenho, a adoção de gRPC implicaria a utilização de ferramentas de geração de código específicas, uma curva de aprendizagem adicional para a equipa e maior dificuldade na inspeção e depuração do tráfego de rede durante o desenvolvimento e a manutenção.

A equipa optou por uma REST API com serialização JSON, tecnologia amplamente dominada pela equipa, nativamente suportada pelos principais *frameworks* de desenvolvimento web e compatível com todas as plataformas alvo. O impacto de desempenho desta escolha é negligenciável no contexto do volume de dados transacionado diariamente por uma loja de conveniência, e os benefícios em termos de facilidade de integração, depuração e manutenção justificam plenamente a opção.

DA-04 — Identificação Única por UUID em detrimento de Chaves Sequenciais por Base de Dados

Em arquiteturas centralizadas, é comum a utilização de identificadores sequenciais gerados pela base de dados, por serem simples, compactos e ordenáveis. No contexto do Mini-Formiga, esta abordagem é inviável: múltiplas lojas podem criar registos de forma independente e simultânea durante a operação *offline*, e a consolidação posterior no servidor central resultaria inevitavelmente em colisões de identificadores.

A equipa optou pela utilização de *Universally Unique Identifiers* (UUID) como chave primária em todas as entidades do modelo. Esta decisão garante que qualquer registo criado localmente em qualquer loja é globalmente único, eliminando conflitos de identificação durante a sincronização e tornando o sistema horizontalmente escalável para um número arbitrário de lojas sem alterações ao esquema de dados.

DA-05 — *Optimistic Locking* em detrimento de *Pessimistic Locking*

A gestão de concorrência em edições simultâneas ao mesmo registo — por exemplo, dois gerentes de lojas distintas a atualizar o catálogo de produtos no servidor central — exige um mecanismo de controlo. O *pessimistic locking*, que bloqueia o registo durante toda a duração da edição, garantiria consistência absoluta mas introduziria latência e risco de bloqueios em cascata.

A equipa optou por *optimistic locking*, em que cada registo é versionado e a operação de escrita falha se a versão lida não corresponder à versão atual no servidor. Este mecanismo é adequado ao padrão de acesso do Mini-Formiga, onde conflitos de edição simultânea são raros e a penalização associada a uma falha ocasional é muito inferior ao custo de bloquear recursos de forma preventiva.

Esta decisão tem uma exceção deliberada: a geração de números sequenciais de faturas e documentos de devolução usa bloqueio pessimista de escrita. O requisito de domínio RD-03 exige numeração sequencial, única e ininterrupta por série; neste caso específico, uma falha de concorrência produziria uma lacuna na numeração fiscal, o que é legalmente inaceitável e não recuperável por nova tentativa.

DA-06 — Lógica de Negócio Descentralizada nas Classes de Domínio

Uma abordagem alternativa, frequente em arquiteturas de serviços, consiste em concentrar toda a lógica de negócio nas classes *Facade* ou em serviços dedicados, reduzindo as classes de domínio a meros contentores de dados. Esta abordagem, designada por *Anemic Domain Model*, simplifica a camada de persistência mas compromete a coesão e a testabilidade unitária das classes.

A equipa optou por encapsular operações de negócio diretamente nas classes de domínio onde fazem sentido semanticamente: `calcularMargem()` em `Produto`, `calcularDiscrepancia()` em `LinhaInventario`, `estaDisponivel()` em `Fornecedor` e `calcularTotais()` em `FechoCaixa`. Esta abordagem reduz o acoplamento entre as classes de domínio e os *Facades*, torna cada classe individualmente testável por testes unitários e alinha o modelo de objetos com os princípios do *Domain-Driven Design*, facilitando a manutenção e a evolução futura do sistema.

DA-07 — Logs guardados em ficheiros JSON em detrimento de na base de dados

A persistência do `LogAuditoria` na base de dados foi considerada como alternativa natural, dado que permitiria consultar o histórico de auditoria via API com filtros arbitrários por utilizador, data ou tipo de operação. Contudo, esta abordagem introduz um acoplamento indesejável entre o subsistema de auditoria e a base de dados operacional: a tabela de logs cresce de forma contínua e proporcional ao volume de transações, podendo degradar o desempenho de queries sobre as restantes entidades e aumentar a dimensão dos backups sem benefício proporcional para a operação do negócio.

A equipa optou por persistir os logs de auditoria em ficheiros JSON rotativos, geridos pelo *framework* Logback com serialização estruturada via *SLF4J*. Cada entrada de log é escrita como um objeto JSON autónomo, contendo os campos obrigatórios definidos no RNF-05:

identificador do utilizador, tipo de operação, data e hora, entidade afetada e respetivo identificador. A rotação diária dos ficheiros e a sua retenção configurável garantem rastreabilidade sem impacto na base de dados operacional.

DA-08 — Abstração de Stock por perfil de execução

A gestão de stock no Mini-Formiga apresenta uma particularidade arquitetural relevante: o sistema pode executar em contexto de loja local ou em contexto central/global. Em ambiente de loja, o stock relevante é o stock operacional da própria loja; em ambiente central, é necessário observar e consolidar o stock por produto e por loja, permitindo ao Gestor da Cadeia ter uma visão global da rede.

Uma alternativa seria duplicar a lógica do subsistema de stock, criando fachadas ou serviços distintos para a loja local e para a instância central. Esta solução aumentaria a complexidade do código, duplicaria regras de negócio e tornaria mais difícil garantir comportamento consistente entre os dois contextos de execução.

A equipa optou por introduzir uma abstração explícita, `StockStore`, responsável por esconder da camada de negócio a forma concreta de como o stock é armazenado e consultado. Esta interface possui implementações distintas selecionadas através de perfis. Em contexto local, a implementação atua diretamente sobre `Produto`, refletindo o stock disponível na loja onde o sistema está instalado. Em contexto central/global, a implementação utiliza `StockProdutoLoja`, permitindo representar a quantidade de cada produto em cada loja da cadeia.

Esta decisão permite que a `SubStockFacade` permaneça única e estável, independentemente do ambiente onde a aplicação está a correr. A fachada continua a expor os mesmos casos de uso — consulta de stock, ajustes, inventário e alertas — enquanto a infraestrutura decide qual a representação de stock adequada ao perfil ativo.

Desta forma, o sistema evita duplicação de lógica de negócio, preserva a simplicidade operacional da instalação local e mantém a capacidade de consolidação central. A decisão é coerente com a arquitetura monolítica modular adotada em DA-01, pois reforça a separação de responsabilidades dentro do próprio monólito sem introduzir serviços independentes ou complexidade operacional adicional.

3.10 Exploração de LLM no Design Arquitetural

Neste processo foi utilizado principalmente um agente de LLM baseado em **GPT**, integrado no ambiente de desenvolvimento como assistente de programação e revisão arquitetural. Em momentos de exploração conceptual, o modelo foi usado para comparar alternativas arquiteturais e para formular justificações em linguagem técnica. Assim, a utilização do LLM combinou

dois papéis: assistente de implementação sobre o código existente e interlocutor crítico na validação do desenho de domínio.

Durante o desenvolvimento do Mini-Formiga, os *Large Language Models* (LLM) foram utilizados como instrumentos de apoio à reflexão arquitetural, revisão de desenho de domínio e validação crítica de decisões técnicas. A sua utilização não substituiu a decisão da equipa, no entanto funcionou antes como um mecanismo de confronto, permitindo testar alternativas, identificar incoerências no modelo e tornar explícitas algumas decisões que estavam inicialmente implícitas no código.

A interação com o agente de LLM foi conduzida de forma iterativa. Em vez de se pedir uma solução final única, foram apresentados excertos do código, decisões arquiteturais já tomadas, diagramas parciais e dúvidas concretas sobre relações entre entidades. O agente foi usado sobretudo para responder a perguntas como: “esta classe ainda faz sentido no domínio?”, “esta relação é redundante?”, “esta entidade representa lógica de negócio ou apenas uma tabela técnica?” e “o modelo atual reflete corretamente os casos de uso?”. Esta abordagem revelou-se particularmente útil nos subsistemas mais complexos, como sincronização, stock e encomendas.

Interação 1: Sincronização, Operação Offline e Estratégia Distribuída

Uma das primeiras utilizações do LLM incidiu sobre a sincronização entre lojas e servidor central. O objetivo era validar a coerência da arquitetura perante o requisito de funcionamento local mesmo em caso de falha de rede.

Prompt submetido ao agente de LLM: *“Atua como um arquiteto de software sénior. O sistema Mini-Formiga gere PDV, stock, encomendas e sincronização para uma pequena cadeia de lojas. Cada loja deve continuar a funcionar localmente mesmo sem ligação ao servidor central. Que arquitetura recomendarias para sincronização, identificação de registos e resolução de conflitos?”*

A resposta inicial do modelo privilegiou uma arquitetura mais distribuída, sugerindo micro-serviços, comunicação eficiente entre serviços e mecanismos robustos de reconciliação de dados. A equipa usou esta resposta como ponto de partida, mas rejeitou parte significativa da recomendação por não se adequar ao contexto operacional do Mini-Formiga. Uma cadeia de até cinco lojas, sem equipa técnica local, não justificava a complexidade de múltiplos serviços independentes, orquestração local e diagnóstico de falhas distribuídas.

Desta análise resultou o reforço de várias decisões arquiteturais:

- A adoção de uma arquitetura monolítica modular com Facade por subsistema, em vez de micro-serviços, por simplificar instalação, operação e manutenção.

- A escolha de REST com JSON para a comunicação entre loja e servidor central, por ser simples de testar, depurar e manter.
- A utilização de UUID como identificador global das entidades, evitando colisões entre registos criados offline em lojas diferentes.
- A adoção de sincronização por lote, no fim da jornada, em vez de sincronização contínua em tempo real.
- A aplicação de *last-write-wins* para resolução automática de conflitos, mantendo o registo dos conflitos para consulta posterior.

Nesta interação, o valor principal do LLM não esteve na aceitação direta da proposta, mas na sua função de contraste. Ao discordar da sugestão de maior complexidade, a equipa tornou mais clara a justificação para as decisões DA-01, DA-02, DA-03, DA-04 e DA-05.

Interação 2: Remoção de Classes Legadas e Clarificação do Domínio

Numa fase posterior, o agente de LLM foi usado para rever o modelo de domínio e identificar classes que já não eram necessárias. Esta revisão surgiu após várias alterações no código, nas quais enums e classes auxiliares antigas tinham sido substituídos por modelos mais simples e próximos da lógica atualmente usada.

Prompt submetido ao agente de LLM: *“Verifica no projeto todas as classes que têm comportamento legacy e diz se é possível eliminá-las. Quero saber se ainda são usadas pelas funcionalidades atuais ou se ficaram apenas como compatibilidade antiga.”*

A análise identificou várias classes que já não representavam o modelo real do sistema. Entre os exemplos discutidos estavam classes antigas relacionadas com estados, perfis, meios de pagamento e stock. A equipa validou cada caso por pesquisa no código, execução de testes e análise dos pontos de utilização.

Esta interação conduziu à remoção ou substituição de elementos legados como `EstadoEncomenda`, `MeioPagamento`, `Perfil`, `Stock`, `NivelMinimo`, `MotivoAjuste`, `FaturaSequencia` e `LocalizacaoProduto`. O modelo foi progressivamente simplificado, mantendo apenas as entidades realmente usadas pelas funcionalidades atuais.

Um resultado importante desta revisão foi a clarificação do subsistema de stock. O modelo antigo tinha entidades próprias para representar stock, níveis mínimos e motivos de ajuste. Após a revisão, a equipa consolidou a lógica de stock em torno de `Produto` e da abstração `StockStore`, mantendo uma implementação local e uma implementação global/central selecionadas por perfil de execução. Esta decisão veio mais tarde a ser formalizada como DA-09.

Interação 3: Reavaliação do Modelo de Stock

O subsistema de stock foi uma das áreas em que a interação com o LLM teve maior impacto. Inicialmente existiam dúvidas sobre a presença de classes como `ProdutoLoja`, `Stock`, `NivelMinimo` e `MotivoAjuste`. A questão central era perceber se estas classes pertenciam realmente ao domínio ou se eram apenas artefactos técnicos herdados de uma versão anterior do modelo.

Prompt submetido ao agente de LLM: *“Explica a parte do stock e o que está a ser utilizado atualmente. Para eliminar corretamente, o caminho seria remover `Stock`, `NivelMinimo` e `MotivoAjuste`? Depois disso o domínio fica mais limpo: stock passa a ser só `Produto/ProdutoLoja`?”*

A resposta ajudou a separar dois conceitos que estavam misturados: stock como quantidade operacional de um produto numa loja, e stock como entidade autónoma. A equipa concluiu que `Stock` e `NivelMinimo` não tinham comportamento suficiente para justificar entidades próprias. Em contexto local, a quantidade de stock e o nível mínimo pertencem ao próprio `Produto`. Em contexto central, a relação entre produto e loja passou a ser representada por `StockProdutoLoja`.

A discussão conduziu ainda à criação de uma abstração explícita, `StockStore`, com implementações distintas por perfil de execução. Esta solução permitiu manter uma única `SubStockFacade`, evitando duplicação de casos de uso entre loja local e servidor central.

Interação 4: Relações no Subsistema de Encomendas

Outra utilização relevante do agente ocorreu na revisão do subsistema de encomendas. A equipa questionou relações que pareciam redundantes ou pouco claras no diagrama de classes, nomeadamente entre `Encomenda`, `LinhaEncomenda`, `GuiaRemessa`, `EntradaMercadoria`, `Fornecedor` e `Produto`.

Prompt submetido ao agente de LLM: *“Tendo em conta as Encomendas, como é que uma linha de encomenda tem uma encomenda e uma encomenda tem uma linha de encomenda? Porque é que a guia de remessa tem o fornecedor e a encomenda também tem fornecedor? Não estamos a repetir?”*

A análise permitiu distinguir entre relações bidirecionais legítimas e redundância real. A relação entre `Encomenda` e `LinhaEncomenda` foi mantida como uma relação bidirecional normal de agregado: a encomenda contém várias linhas, mas cada linha mantém a referência à encomenda por ser a entidade que possui a chave estrangeira.

Por outro lado, a relação direta entre `GuiaRemessa` e `Fornecedor` foi considerada redundante. Como a guia de remessa passou a estar sempre associada a uma encomenda, o fornecedor

passou a ser derivado por `guia.getEncomenda().getFornecedor()`. A equipa implementou esta alteração, removendo o fornecedor direto da guia e tornando a encomenda obrigatória.

A mesma revisão levou a uma segunda melhoria: `EntradaMercadoria` deixou de referenciar diretamente `Produto`. A partir da guia é possível obter a encomenda, mas não é possível saber a que produto específico se refere uma entrada, pois uma encomenda pode ter várias linhas. A solução adotada foi ligar `EntradaMercadoria` a `LinhaEncomenda`, passando o produto a ser derivado por `entrada.getLinhaEncomenda().getProduto()`.

Interação 5: Entidades Associativas e Condição Comercial

A modelação de `CondicaoComercial` também foi discutida com apoio do LLM. A dúvida colocada foi se a condição comercial deveria pertencer diretamente ao fornecedor ou se deveria ser modelada como uma entidade autónoma ligada ao fornecedor e ao produto.

Prompt submetido ao agente de LLM: *“Não deveria ser o fornecedor a ter a condição comercial, e não a condição comercial a ter o fornecedor? Como ficaria isto num diagrama de classes?”*

A resposta ajudou a clarificar que `CondicaoComercial` não é apenas uma propriedade do fornecedor. Ela representa uma relação com atributos próprios entre `Fornecedor` e `Produto`: preço unitário, quantidade mínima, prazo de entrega e data de vigência. Por isso, foi mantida como entidade associativa.

Cada condição comercial pertence a um fornecedor e a um produto. Um fornecedor pode ter várias condições comerciais e um produto pode ser fornecido por vários fornecedores em condições distintas.

Interação 6: Organização do Projeto e Testes

O agente de LLM foi também usado como apoio à reorganização do projeto em pacotes mais explícitos. O código foi estruturado por subsistemas, separando *facade*, *service*, *repository*, *dto* e *transport*. Esta reorganização tornou mais visível a arquitetura modular definida em DA-01.

Após a reorganização, alguns testes ficaram desatualizados por referenciarem classes, DTOs ou exceções que tinham sido movidos. O agente ajudou a identificar e corrigir estes problemas através da execução iterativa dos testes, leitura dos erros de compilação e ajuste dos imports ou construtores usados. Esta prática foi particularmente útil porque permitiu validar continuamente que as alterações estruturais não quebravam o comportamento esperado.

Síntese das Decisões Assistidas

A Tabela 3.9 resume as principais interações com o agente de LLM e o respetivo impacto no projeto.

Âmbito	Contributo do LLM	Decisão da Equipa	Resultado
Sincronização e operação offline	Sugeriu alternativas distribuídas e padrões de sincronização.	Rejeição de microserviços; adoção de monólito modular, REST JSON, UUID e sincronização por lote.	Parcialmente aceite
Classes legadas	Identificou classes sem uso funcional atual.	Remoção de classes antigas e simplificação do domínio.	Aceite
Stock	Ajudou a distinguir stock como entidade autónoma de stock como quantidade operacional.	Remoção do modelo antigo e adoção de StockStore com implementações por perfil.	Aceite
Encomendas	Identificou redundância entre GuiaRemessa e Fornecedor.	GuiaRemessa passou a depender de Encomenda; fornecedor passou a ser derivado.	Aceite
Entrada de mercadoria	Questionou a derivação do produto a partir da guia.	EntradaMercadoria passou a referenciar LinhaEncomenda, derivando o produto dessa linha.	Aceite
Condição comercial	Clarificou o papel de entidade associativa entre fornecedor e produto.	Manutenção de CondicaoComercial como entidade própria com atributos da relação comercial.	Aceite
Organização do código	Apoiou a reorganização por subsistema e camada.	Separação em <i>facade</i> , <i>service</i> , <i>repository</i> , <i>dto</i> e <i>transport</i> .	Aceite

Tabela 3.9: Principais interações com o agente de LLM e impacto nas decisões de design

Em síntese, a utilização do LLM foi mais produtiva quando a equipa o tratou como um agente de revisão crítica e não como uma fonte automática de decisões finais. As respostas foram úteis para levantar alternativas, revelar redundâncias e questionar relações do modelo, mas todas as sugestões foram validadas contra o contexto real do Mini-Formiga, os requisitos do projeto e os testes existentes. Esta utilização iterativa contribuiu para um domínio mais limpo, uma arquitetura mais explícita e uma maior coerência entre código, diagramas e decisões arquiteturais.

4 Implementação e Desenvolvimento Assistido por LLM

4.1 Configuração do Ambiente

A configuração do ambiente de desenvolvimento foi definida com o objetivo de garantir reprodutibilidade, consistência entre máquinas de trabalho e separação clara entre o contexto local de desenvolvimento e o contexto de execução em integração. Para tal, o projeto foi estruturado em dois blocos principais: um *backend* em **Java** com **Spring Boot** e um *frontend* em **React** com **TypeScript**, ambos suportados por ferramentas de construção e validação próprias.

No *backend*, a construção do projeto é assegurada por **Maven**, que centraliza a gestão de dependências, a compilação do código, a execução dos testes automatizados e a geração do artefacto executável. Esta decisão permitiu uniformizar o fluxo de trabalho da equipa e reduzir discrepâncias entre ambientes de desenvolvimento distintos. A execução da aplicação depende ainda de ficheiros de configuração externalizados, nomeadamente os perfis de aplicação e as propriedades associadas à ligação à base de dados e ao comportamento em contexto local ou central.

No *frontend*, a utilização de **Vite** como ferramenta de arranque e *build* proporcionou um ciclo de desenvolvimento mais rápido, especialmente relevante numa fase de implementação incremental. A presença de **TypeScript** e de **ESLint** contribuiu para reforçar a qualidade do código e para reduzir erros de natureza sintática e estrutural antes da integração final.

A nível infraestrutural, a equipa recorreu a **Docker Compose** para orquestrar os serviços necessários ao desenvolvimento e à demonstração do sistema, incluindo a base de dados central e os componentes de aplicação. Esta abordagem simplificou a replicação do ambiente entre elementos da equipa e favoreceu a execução consistente dos módulos dependentes de serviços externos. Em paralelo, foram definidos procedimentos de configuração específicos para a base de dados local de cada loja e para a base de dados central, garantindo alinhamento com o modelo híbrido online/offline assumido na arquitetura do sistema.

Os procedimentos completos de **utilização, arranque, validação e manutenção do trabalho** encontram-se sistematizados no **Anexo 6**, permitindo que a solução possa ser executada e mantida de forma reprodutível por outros elementos da equipa ou por avaliadores externos.

Em síntese, a configuração do ambiente procurou equilibrar robustez técnica e simplicidade

operacional. A combinação entre ferramentas maduras, configuração declarativa e containerização permitiu estabelecer uma base sólida para o desenvolvimento, a validação e a eventual continuidade futura do projeto.

4.2 Implementação por Módulos

A implementação do sistema foi conduzida segundo uma lógica estritamente modular, em conformidade com a arquitetura definida nas fases anteriores do relatório. Esta opção teve como finalidade principal reduzir o acoplamento entre áreas funcionais, facilitar a validação incremental e permitir que cada subsistema fosse desenvolvido e testado de forma relativamente autônoma antes da integração global.

No *backend*, a decomposição do sistema em módulos funcionais traduziu-se na separação entre as responsabilidades de **SubUtilizadores**, **SubPDV**, **SubStock**, **SubEncomendas**, **SubSincronizacao**, **SubAuditoria** e **SubRelatorios**. Cada módulo agrega entidades, serviços e interfaces orientados a uma responsabilidade bem delimitada, o que favorece a legibilidade do código e a manutenção futura. Esta organização reflete o princípio de responsabilidade única e aproxima a estrutura de implementação da modelação conceptual apresentada no capítulo de arquitetura.

O desenvolvimento por módulos foi executado de forma progressiva, iniciando-se pelas entidades centrais e pelas dependências transversais, como Loja, Produto, Utilizador e os mecanismos de auditoria. Só depois se avançou para os processos de maior complexidade operacional, como o registo de vendas, a gestão de inventário, a emissão de documentos e os fluxos de sincronização. Esta sequência permitiu consolidar primeiro a espinha dorsal do domínio e reduzir o risco de retrabalho em fases posteriores.

A nível de integração, cada módulo foi desenvolvido com contratos explícitos entre camadas, de modo a preservar a independência das implementações internas. As interfaces públicas dos subsistemas funcionam como pontos estáveis de comunicação com a camada de apresentação e com os restantes componentes do sistema, tornando possível substituir ou evoluir uma parte específica sem comprometer a estrutura global.

No *frontend*, a implementação modular traduziu-se na decomposição da interface em páginas e componentes reutilizáveis, agrupados por perfil funcional. Esta abordagem permitiu isolar as responsabilidades de autenticação, operação de ponto de venda, gestão de stock, administração e consulta de relatórios, reforçando a clareza da navegação e a reutilização de elementos visuais e comportamentais. Em termos práticos, a modularização foi determinante para garantir que a interface acompanhasse a separação funcional do *backend*.

Assim, a implementação por módulos não foi apenas uma decisão de organização interna do código, mas sim um instrumento metodológico para controlar complexidade, promover testabilidade e assegurar coerência entre a especificação, a arquitetura e a concretização técnica do sistema.

4.3 Geração de Código com LLM

A geração de código com recurso a LLM foi usada como mecanismo de apoio ao processo de implementação, e não como substituto da decisão técnica da equipa. A utilização destas ferramentas centrou-se sobretudo na aceleração de tarefas repetitivas, na produção inicial de estruturas-base e na sugestão de alternativas de implementação para blocos de código com menor densidade conceptual. Em todos os casos, o resultado obtido foi sujeito a revisão manual antes de ser integrado no repositório do projeto.

O processo de geração assentou numa metodologia de *prompting* orientada por contexto. Sempre que possível, forneceu-se ao modelo informação suficiente sobre o objetivo funcional, o enquadramento arquitetural, as classes envolvidas e as restrições de implementação relevantes. Esta prática revelou-se particularmente útil na criação de métodos auxiliares, estruturas de serviços, validações de entrada e excertos de integração entre componentes, reduzindo o tempo despendido em tarefas mecânicas sem comprometer a consistência do sistema.

Para ilustrar esta abordagem, apresenta-se de seguida um caso concreto em que o pedido inicial foi reformulado pela equipa para produzir uma resposta mais alinhada com a arquitetura do projeto.

Prompt inicial: “Gera o código para registar uma venda.”

Embora funcional em termos de intenção, este pedido não indicava o subsistema envolvido, o tipo de entidades a manipular, a forma de persistência nem as regras de negócio associadas ao cálculo de totais e impostos. A resposta obtida tendia, por isso, a ser demasiado abstrata e a omitir detalhes essenciais para integração no sistema.

Após revisão, o prompt foi reformulado pela equipa de modo a explicitar o contexto técnico e funcional:

Prompt reformulado: “Considerando o subsistema SubPDV em Java com Spring Boot, gera um método de serviço para registar uma Venda com múltiplas LinhaVenda, calcular o total com IVA por taxa, validar que o utilizador autenticado possui permissões de operador de caixa e devolver a Fatura gerada. Assume que as entidades Venda, LinhaVenda, Produto e Fatura já existem e respeita o estilo do projeto.”

Esta versão produziu resultados substancialmente mais úteis, por incluir o contexto de domínio, as classes relevantes, as restrições de autorização e o objetivo funcional concreto.

Contudo, a adoção de LLM na fase de implementação exigiu uma supervisão rigorosa. Nem todas as respostas produzidas eram imediatamente adequadas ao contexto do projeto, sobretudo quando envolviam decisões dependentes da arquitetura específica do sistema ou de convenções já estabelecidas no código existente. Por esse motivo, foi seguido um princípio de validação em três etapas: análise da resposta gerada, adaptação ao estilo e às dependências do projeto,

e verificação funcional através de execução local ou de testes automatizados quando aplicável.

Do ponto de vista pedagógico e metodológico, esta experiência confirmou que os LLM são mais úteis quando funcionam como aceleradores cognitivos do que como substitutos da autoria técnica. A sua principal vantagem residiu na capacidade de propor rapidamente estruturas iniciais e variantes plausíveis; a sua principal limitação esteve na tendência para gerar soluções genéricas, por vezes insuficientemente alinhadas com as regras de negócio ou com a organização concreta do código.

Em consequência, a equipa procurou utilizar a geração assistida por LLM de forma criteriosa, mantendo sempre a responsabilidade final sobre a correção, coerência e qualidade do código produzido. Esta postura permitiu beneficiar da produtividade acrescida destas ferramentas sem abdicar do controlo técnico necessário a um trabalho académico com exigência de rigor.

4.4 Decisões e Adaptações da Fase de Implementação

Ao longo da implementação do sistema Mini-Formiga, a equipa deparou-se com situações que exigiram decisões técnicas concretas não completamente especificadas nas fases anteriores do projeto. Esta secção documenta essas decisões, justificando as escolhas tomadas e assinalando os pontos em que a implementação detalha, adapta ou complementa a especificação produzida nas fases de Engenharia de Requisitos e Arquitetura.

Estrutura Física do Log de Auditoria

O requisito RNF-05 especifica que todas as operações financeiras devem ser registadas num *log* de auditoria imutável, com identificação do utilizador, data, hora e tipo de operação. A decisão arquitetural DA-07 determina que estes registos são persistidos em ficheiros JSON rotativos geridos pelo *framework* Logback, sem impacto na base de dados operacional.

Na implementação, a estrutura física de cada entrada de *log* foi normalizada com os seguintes campos: *utilizadorId*, *tipoOperacao*, *dataHora*, *entidade*, *entidadeId* e *descricao*. Esta normalização torna o requisito diretamente verificável e elimina ambiguidade entre termos como *recurso*, *operação* e *entidade afetada* que coexistiam na especificação conceptual.

Arquitetura de Perfis de Execução: Local e Central

A decisão arquitetural DA-01 especifica uma arquitetura monolítica modular sem se pronunciar sobre como a distinção entre o servidor local de cada loja e o servidor central seria concretizada.

Na implementação, optou-se por um único *codebase* executado em dois perfis de configuração distintos, sem necessidade de um segundo projeto de *software* independente.

No perfil *local*, a aplicação usa SQLite como base de dados embebida, constrói o *payload* de sincronização e envia-o para o URL central configurado na propriedade *mini-formiga.sincronizacao.central-url*. No perfil *central*, a aplicação usa PostgreSQL e expõe o *endpoint* `POST /api/v1/central/sincronizacao/receber` para receber e consolidar os *payloads* das lojas. Se o URL central não estiver disponível no momento da sincronização, o estado da sincronização local permanece *PENDENTE* com registo de *proximaTentativa*, cumprindo o requisito RNF-02 de disponibilidade local independente do estado da ligação à rede.

TLS como Responsabilidade de *Deployment*

O requisito RNF-06 e o UC-13 especificam que a comunicação entre terminais de loja e o servidor central deve ser cifrada com TLS 1.3. Na implementação local de desenvolvimento, o *backend* corre em HTTP para simplificar a execução sem infraestrutura adicional. A cifra TLS 1.3 é garantida pelo *reverse proxy* ou pelo *endpoint* central HTTPS em contexto de *deployment*, não constituindo uma limitação do sistema mas uma responsabilidade de configuração de infraestrutura, conforme é prática corrente em sistemas distribuídos desta natureza.

Configuração do Ambiente de Execução

O Docker Compose fornecido orquestra o PostgreSQL central, o backend central, o backend local e o frontend. Os serviços de backend são executados em containers Maven/Eclipse Temurin 21, com o código montado por volume e arranque via `mvn spring-boot:run`, permitindo um ciclo de desenvolvimento iterativo sem exigir a construção de uma imagem Docker dedicada para o backend. Em alternativa, o backend também pode ser executado localmente por Maven durante o desenvolvimento.

4.5 Revisão de código gerado

A verificação do código foi realizada como uma etapa contínua do processo de desenvolvimento, com o objetivo de garantir não apenas que as funcionalidades implementadas correspondiam aos requisitos definidos, mas também que a qualidade técnica do sistema se mantinha adequada. Esta verificação combinou revisão humana com apoio de ferramentas de IA, sendo a decisão final sempre assumida pela equipa.

O processo de validação incidiu sobre duas dimensões principais: a qualidade do código e a conformidade funcional. Na primeira dimensão, foram analisados aspetos como legibilidade,

duplicação, organização dos componentes, coerência com a arquitetura existente, utilização adequada das abstrações do projeto e ausência de código legado ou inutilizado. Na segunda dimensão, foram verificados os *user stories* e os casos de uso definidos, de modo a confirmar que o comportamento observado na aplicação correspondia ao comportamento esperado.

A verificação com recurso a IA foi utilizada sobretudo como apoio à deteção de problemas potenciais, identificação de código morto, sugestões de simplificação e análise de consistência entre frontend, backend e contratos de API. No entanto, estas sugestões foram sempre revistas manualmente, uma vez que a IA pode não compreender completamente as decisões arquiteturais específicas do projeto ou os compromissos assumidos pela equipa ao longo do desenvolvimento.

Do ponto de vista humano, a equipa realizou uma revisão direta das alterações introduzidas, avaliando se o código era compreensível, se respeitava os padrões já existentes no repositório e se não introduzia regressões em funcionalidades previamente implementadas. Esta revisão foi complementada com execução local da aplicação e testes automatizados sempre que aplicável.

Para a verificação da qualidade do código, foram seguidos os seguintes passos:

1. Identificação de código potencialmente obsoleto, duplicado ou utilizado apenas por compatibilidade com versões anteriores.
2. Confirmação, através de pesquisa no repositório, de que o código assinalado já não era referenciado por funcionalidades ativas.
3. Remoção conservadora apenas dos elementos considerados inutilizados, evitando alterações em código ainda ligado ao domínio ou a testes relevantes.
4. Simplificação de estruturas existentes quando tal melhorava a legibilidade sem alterar o comportamento funcional.
5. Verificação dos contratos entre frontend e backend, garantindo que os dados necessários eram enviados, persistidos e novamente apresentados.
6. Execução de testes automatizados e comandos de validação, nomeadamente compilação do frontend, análise estática e testes do backend.
7. Revisão final das diferenças introduzidas, confirmando que as alterações eram coerentes com o objetivo funcional e não afetavam casos de uso existentes.

A validação funcional foi feita com base nos *user stories* e nos casos de uso associados. Para cada funcionalidade relevante, confirmou-se se o fluxo implementado permitia ao utilizador realizar a ação prevista e obter o resultado esperado. Um exemplo concreto foi a verificação da *user story* US-21, relativa à localização de produtos no armazém.

US-21 – Teste: pesquisar um produto no sistema e verificar que é apresentada a sua localização no armazém, permitindo ao responsável identificar rapidamente

onde se encontra o artigo sem necessidade de consulta manual.

Neste caso, a verificação consistiu em confirmar que os campos de localização do produto, nomeadamente corredor e prateleira, eram apresentados no ecrã de stock do responsável de armazém, podiam ser editados e eram enviados para o backend ao guardar. Confirmou-se ainda que estes valores ficavam persistidos na base de dados, permitindo que, após sair e voltar a entrar na aplicação, a localização anteriormente definida continuasse visível no sistema.

Esta validação permitiu concluir que o critério de verificação da US-21 foi cumprido, uma vez que o responsável de armazém consegue consultar rapidamente a localização do artigo sem recorrer a registos manuais ou informação externa ao sistema.

Os restantes casos de uso seguiram o mesmo processo.

Assim, a verificação do código não se limitou à confirmação de que a aplicação compilava ou executava sem erros. Procurou-se garantir que o sistema permanecia tecnicamente sustentável, funcionalmente coerente com os requisitos e alinhado com os casos de uso definidos para o projeto.

4.6 Detecção de *smells* e sugestão de modificações

A deteção de *code smells* foi integrada no processo de revisão de código como mecanismo complementar à análise manual da equipa.

Para sistematizar esta análise, a equipa recorreu a um agente de linguagem configurado com um *prompt* estruturado por categorias de *smells*, instruindo o modelo a percorrer o código em três passagens sequenciais: leitura estrutural para compreensão do contexto e dependências, deteção por categoria com localização precisa, e síntese final com priorização e sugestões de refatorização. Esta abordagem permitiu combinar a capacidade de reconhecimento de padrões do modelo com o julgamento técnico da equipa, que manteve a responsabilidade final sobre a relevância e adequação de cada ocorrência identificada.

Os tipos de *code smells* considerados na análise foram organizados em quatro categorias, descritas na Tabela 4.1: smells de tamanho e complexidade, de responsabilidade, de duplicação e abstração, e de acoplamento.

Categoria	Code Smell	Descrição	Severidade Típica
Tamanho e Complexidade	<i>Long Method</i>	Métodos com mais de 20–30 linhas ou múltiplos níveis de abstração misturados.	Major

(continua na página seguinte)

Categoria	Code Smell	Descrição	Severidade Típica
Tamanho e Complexidade	<i>God Class</i>	Classe com responsabilidades excessivas, elevado número de dependências externas e baixa coesão.	Blocker
Tamanho e Complexidade	<i>Long Parameter List</i>	Funções ou construtores com mais de três a quatro parâmetros, dificultando leitura e teste.	Minor
Tamanho e Complexidade	<i>Complex Conditionals</i>	Lógica booleana com múltiplos níveis de aninhamento ou condições de difícil interpretação.	Major
Responsabilidade	<i>Feature Envy</i>	Método que utiliza mais dados e operações de outra classe do que da sua própria, sugerindo localização incorreta da lógica.	Major
Responsabilidade	<i>Divergent Change</i>	Classe que é alterada por razões distintas e independentes, violando o Princípio da Responsabilidade Única.	Major
Responsabilidade	<i>Shotgun Surgery</i>	Uma única alteração de requisito obriga a modificar múltiplos ficheiros ou classes não relacionados.	Blocker
Responsabilidade	<i>Inappropriate Intimacy</i>	Duas classes acedem excessivamente aos internos uma da outra, criando acoplamento bidirecional indesejado.	Major
Duplicação e Abstração	<i>Duplicate Code</i>	Lógica semanticamente equivalente repetida em múltiplos locais, mesmo que com variações superficiais de nomenclatura.	Major
Duplicação e Abstração	<i>Primitive Obsession</i>	Uso de tipos primitivos (String, int) onde deveria existir um <i>Value Object</i> ou tipo de domínio próprio.	Minor

(continua na página seguinte)

Categoria	Code Smell	Descrição	Severidade Típica
Duplicação e Abstração	<i>Data Clumps</i>	Grupos de variáveis que ocorrem sempre em conjunto e deveriam ser encapsulados numa classe própria.	Minor
Duplicação e Abstração	<i>Switch Statements</i>	Condicionais sobre tipo ou estado que deveriam ser substituídas por polimorfismo ou pelo padrão <i>Strategy</i> .	Major
Acoplamento	<i>Message Chains</i>	Encadeamentos do tipo <code>a.getB().getC().getD()</code> , violando a Lei de Demeter e aumentando a fragilidade.	Minor
Acoplamento	<i>Middle Man</i>	Classe que apenas delega chamadas para outra sem acrescentar lógica própria, tornando-se uma indireção desnecessária.	Minor
Acoplamento	<i>Dead Code</i>	Código comentado, variáveis não utilizadas ou métodos inacessíveis que poluem a base de código sem contribuição funcional.	Nit
Acoplamento	<i>Speculative Generality</i>	Abstrações ou parametrizações introduzidas antecipadamente para casos de uso hipotéticos sem aplicação atual.	Nit

Tabela 4.1: Taxonomia de *code smells* analisados pelo agente de IA, por categoria e severidade típica

A severidade atribuída a cada ocorrência seguiu uma escala de quatro níveis: *Blocker*, para problemas com impacto direto na manutenibilidade, testabilidade ou extensibilidade do sistema; *Major*, para situações que introduzem acoplamento significativo ou dificultam a evolução do código; *Minor*, para casos que reduzem a legibilidade ou violam convenções estabelecidas; e *Nit*, para melhorias de caráter cosmético ou estilístico sem consequências estruturais imediatas.

Importa sublinhar que a identificação automática de *code smells* não é equivalente à obrigação de os corrigir. Alguns padrões identificados pelo modelo revelaram-se justificados por condicionantes de desempenho, por decisões arquiteturais já documentadas ou por especificidades

do domínio. Nestes casos, a equipa optou por registar a ocorrência mas não proceder a qualquer alteração, mantendo o princípio de que a ferramenta de IA actua como acelerador da análise e não como árbitro da qualidade. A decisão final sobre a necessidade de refatorização permaneceu sempre sob responsabilidade humana.

Após a identificação dos *code smells*, a equipa procedeu a uma análise crítica das ocorrências sinalizadas, distinguindo entre problemas que podiam ser corrigidos de forma segura e situações cuja resolução exigiria uma reestruturação arquitetural mais profunda. A Tabela 4.2 apresenta os principais *smells* encontrados no código do projeto.

Categoria	Code Smell	Ocorrência Identificada	Severidade
Tamanho e Complexidade	<i>God Class</i>	A classe RelatoriosFacade concentra geração de dashboard, relatórios de vendas, stock, rentabilidade e exportações em CSV, PDF e XLSX.	Blocker
Tamanho e Complexidade	<i>Long Method</i>	Componentes React como FuncionarioSalePage, GestorOrdersPage, ReportsContent e StockContent concentram carregamento de dados, validação, submissão e renderização.	Major
Responsabilidade	<i>Divergent Change</i>	EncomendasFacade agrega fornecedores, condições comerciais, encomendas, sugestões automáticas e entrada de mercadoria.	Major
Responsabilidade	<i>Shotgun Surgery</i>	A gestão de perfis e permissões encontra-se distribuída entre backend, frontend, rotas, menus e autenticação.	Major
Duplicação e Abstração	<i>Duplicate Code</i>	Foram identificadas funções repetidas para descarga de ficheiros e formatação de datas em vários componentes do frontend.	Major
Duplicação e Abstração	<i>Primitive Obsession</i>	Conceitos como tipo de relatório, formato de exportação, turno, perfil e meio de pagamento surgem frequentemente representados como String.	Minor
Duplicação e Abstração	<i>Switch Statements</i>	Foram encontrados switch em mapeamentos de perfis, tipos de relatório, meios de pagamento, formatos de exportação e ícones.	Major
Acoplamento	<i>Message Chains</i>	Existem encadeamentos como <code>linha.getProduto().getCategoria().getNome()</code> e <code>fatura.getVenda().getLoja().getId()</code> .	Minor

Categoria	Code Smell	Ocorrência Identificada	Severidade
Acoplamento	<i>Dead Code</i>	Existiam elementos herdados da fase inicial, incluindo API de demonstração, <i>facade</i> em memória, dados mockados operacionais e assets não utilizados.	Nit
Acoplamento	<i>Speculative Generality</i>	Algumas interfaces de subsistema têm apenas uma implementação concreta, refletindo uma abstração introduzida por alinhamento arquitetural mais do que por necessidade atual.	Nit

Tabela 4.2: *Code smells* identificados no código do projeto

A partir desta análise, foram realizadas refatorizações seletivas. Foram removidos elementos de código morto, nomeadamente a API de demonstração baseada em `/api/demo`, a `MiniFormigaFacade`, a interface `IMiniFormigaLN`, configuração de compatibilidade de schema antigo, dados mockados operacionais já não utilizados pelo frontend e assets herdados do template inicial da aplicação. Também foram simplificados alguns pontos de duplicação no frontend, através da criação de funções utilitárias comuns para formatação de datas e descarga de ficheiros.

Além disso, alguns `switch` simples foram substituídos por mapas tipados, reduzindo a complexidade condicional em zonas como o mapeamento entre perfis de utilizador e rotas da aplicação, a resolução de endpoints de relatórios e o cálculo de totais por meio de pagamento. Estas alterações foram consideradas de baixo risco por não modificarem regras de negócio, apenas a forma como a lógica estava organizada.

Importa referir que algumas destas alterações implicaram afastar parcialmente a implementação final do plano inicialmente representado nos diagramas de sequência e no diagrama de classes. Em particular, a remoção da `MiniFormigaFacade` e da API de demonstração eliminou uma camada prevista numa fase inicial do desenho, mas que já não correspondia ao funcionamento real da aplicação. A evolução do projeto levou à substituição dessa estrutura por *facades* específicas por subsistema e controllers REST reais, mais alinhados com os fluxos efetivamente utilizados pelo frontend.

Esta decisão foi tomada de forma consciente, privilegiando a qualidade e a manutenibilidade do código executado em produção face à preservação de uma estrutura de demonstração que já não tinha utilidade funcional. Assim, embora os diagramas iniciais tenham servido como orientação arquitetural, a implementação final foi ajustada para refletir melhor as responsabilidades reais do sistema.

Nem todos os *smells* identificados foram corrigidos de imediato. Casos como a dimensão da `RelatoriosFacade` ou de alguns componentes React exigiriam uma refatorização mais profunda, com maior risco de introdução de regressões. Por esse motivo, estes casos foram

registados como dívida técnica, mas não foram alterados. A equipa optou por corrigir apenas ocorrências cuja remoção ou simplificação podia ser validada de forma objetiva através de compilação, análise estática e testes automatizados.

Após as alterações, a verificação foi realizada através de três mecanismos: compilação do frontend, execução do *lint* e execução dos testes automatizados do backend. Esta validação confirmou que as refatorizações efetuadas não quebraram os fluxos funcionais existentes.

5 Verificação, Validação e Avaliação da Qualidade do Software

5.1 Estratégia de Testes

No seguimento da implementação de código a equipa procedeu à definição da estratégia de testes de modo a garantir que as funcionalidades desenvolvidas cumpriam os requisitos definidos, se mantinham alinhadas com os casos de uso e não introduziam regressões nos fluxos já existentes. A definição desta estratégia surgiu também da necessidade de aumentar a confiança nas alterações realizadas ao longo do projeto, uma vez que a aplicação integra vários subsistemas dependentes entre si, como vendas, stock, encomendas, utilizadores, sincronização, relatórios e auditoria.

A equipa optou por uma abordagem multicamada, organizada de forma progressiva. Primeiro foram validadas as unidades mais pequenas do sistema, como entidades de domínio, serviços e fachadas. De seguida, foram testadas as interações entre componentes, incluindo controladores REST, segurança e persistência. Numa fase posterior, foram executados testes de sistema com o contexto completo da aplicação. Por fim, foram definidos testes de aceitação baseados nos fluxos funcionais descritos nas *User Stories* e nos casos de uso.

Esta organização permitiu que cada tipo de teste tivesse um propósito claro. Os testes unitários ajudaram a localizar erros em regras específicas; os testes de integração validaram a colaboração entre camadas; os testes de sistema confirmaram o funcionamento global da aplicação; e os testes de aceitação permitiram verificar se o comportamento final correspondia ao que era esperado pelos utilizadores. Desta forma, a estratégia de testes não foi tratada apenas como uma atividade final de verificação, mas como um processo de apoio à evolução segura do código.

A suite automatizada do backend foi implementada com **JUnit 5**, **Mockito**, **MockMvc** e testes **@SpringBootTest**. A sua qualidade estrutural foi acompanhada com **JaCoCo**, integrado na execução `mvn clean verify`, e complementada com testes de mutação através de **PIT**. A equipa utilizou a cobertura como indicador de apoio, mas não como critério único de qualidade, privilegiando a relevância dos cenários testados, a ligação aos requisitos e a capacidade dos testes detetarem regressões reais.

5.2 Testes Unitários

Os testes unitários constituíram a primeira etapa da estratégia de testes. A sua finalidade foi validar de forma isolada a lógica de domínio, serviços e fachadas, antes de estes componentes serem avaliados em conjunto com a API, a base de dados ou outros subsistemas. Esta separação permitiu confirmar que as regras fundamentais do sistema estavam corretas na sua origem.

A equipa começou por identificar as classes com maior concentração de regras de negócio, estados internos relevantes ou decisões condicionais importantes. A partir dessa análise foram definidos testes para cenários positivos, situações de erro e casos de fronteira. Esta seleção procurou cobrir tanto o comportamento esperado em condições normais como situações em que o sistema deveria rejeitar dados inválidos ou impedir transições de estado incorretas.

A implementação foi realizada com **JUnit 5** e **Mockito**. O uso de *mocks* permitiu substituir dependências externas e testar apenas a unidade em análise. Esta decisão foi importante para manter os testes rápidos, determinísticos e fáceis de interpretar. Quando um teste unitário falhava, a causa do problema ficava normalmente localizada numa regra ou componente específico, facilitando a correção.

Foram cobertos cenários associados a produto, categoria, loja, utilizador, venda, fatura, stock, encomenda, inventário e sincronização. A suite inclui também testes diretos a serviços de segurança, auditoria e tratamento de erros. Nesta fase foram acrescentados testes às fachadas de utilizadores e encomendas, ao transporte REST de sincronização e a entidades de referência, como perfis, estados de encomenda e estados de sincronização.

O objetivo desta etapa não foi apenas aumentar a cobertura, mas garantir que as regras centrais do domínio estavam devidamente protegidas contra alterações futuras. Assim, operações como cálculos de totais, validação de estados, emissão de documentos, atualização de stock ou criação de encomendas puderam ser verificadas de forma controlada antes de serem integradas em fluxos maiores.

5.3 Testes de Integração

Depois da validação unitária, a equipa avançou para testes de integração. O objetivo desta camada foi confirmar que os vários componentes do backend funcionavam corretamente quando executados em conjunto. Esta fase foi necessária porque muitas funcionalidades não dependem apenas de uma classe isolada, mas da articulação entre controladores REST, fachadas, serviços, segurança, repositórios e persistência.

Foram usados `MockMvc`, `@WebMvcTest`, `@SpringBootTest` e uma base de dados SQLite em memória. O processo seguido pela equipa consistiu em simular pedidos reais à API, verificar os códigos de resposta, validar os corpos das respostas, confirmar a serialização e desserialização dos dados, testar a validação dos pedidos e assegurar que os erros eram tratados de forma

uniforme.

Esta camada permitiu também validar o comportamento da segurança da API. Foram testados cenários de acesso sem token, que deveriam resultar em 401, acessos autenticados com token válido e casos em que o utilizador autenticado não possuía permissões suficientes, resultando em 403. Estes testes foram importantes porque a aplicação possui perfis de utilizador distintos e várias operações só podem ser executadas por determinados papéis ou permissões.

Nesta fase foram testados controladores de autenticação, utilizadores, vendas, stock, inventários, encomendas, sincronização, relatórios, categorias, fornecedores, produtos, fechos de caixa e meios de pagamento. A equipa procurou garantir que cada endpoint relevante expunha corretamente a funcionalidade da camada de negócio e que as respostas da API se mantinham consistentes com o contrato esperado pelo frontend.

Os testes de integração serviram, assim, como mecanismo de validação entre a lógica interna e a interface HTTP da aplicação. Sempre que uma alteração era feita numa fachada, entidade ou serviço, estes testes ajudavam a confirmar que o comportamento exposto ao exterior não tinha sido quebrado.

5.4 Testes de Sistema

Os testes de sistema foram definidos para avaliar a aplicação com o contexto Spring completo e dados de demonstração controlados. Ao contrário dos testes unitários e de integração, que se focam em componentes ou grupos de componentes, esta fase teve como objetivo validar fluxos completos da solução, aproximando a execução ao comportamento real esperado em ambiente de uso.

A equipa considerou esta camada necessária porque certos problemas apenas se manifestam quando o sistema é executado como um todo. Uma operação pode estar correta isoladamente, mas falhar quando depende de autenticação real, permissões, dados previamente existentes, persistência, comunicação entre subsistemas ou alterações de estado sucessivas. Os testes de sistema permitiram, por isso, verificar a continuidade dos principais processos de negócio.

O teste `SistemaApiAcceptanceTest` cobre a autenticação real, emissão de JWT, execução de operações de PDV, faturação, fecho de caixa, sincronização, alertas de stock, encomendas, entrada de mercadoria, inventário físico, identificação de discrepâncias e ajuste de stock. Estes cenários foram organizados de forma sequencial, reproduzindo dependências naturais entre operações. Por exemplo, uma venda pode originar faturação e impacto no stock; uma encomenda pode conduzir à entrada de mercadoria; e um inventário pode terminar na correção das quantidades registadas.

Esta abordagem permitiu à equipa avaliar se os fluxos principais permaneciam funcionais após alterações no código. Além disso, ajudou a detetar problemas de configuração, falhas em dados de demonstração, erros de permissões e inconsistências entre o comportamento esperado pela

API e o estado persistido na base de dados.

Além da suite automatizada do backend, a validação técnica do projeto inclui `npm run lint` e `npm run build` no frontend. Estes comandos foram usados para confirmar que a camada React se mantinha compilável, sem erros estáticos relevantes, e compatível com os contratos consumidos da API. Desta forma, a verificação do sistema não ficou limitada ao backend, incluindo também a qualidade mínima necessária da interface web.

5.5 Testes de Aceitação

Os testes de aceitação corresponderam à etapa final da estratégia. Nesta fase, o foco deixou de estar na estrutura interna do código e passou a estar na confirmação de que a aplicação respondia aos objetivos funcionais definidos para os utilizadores. Por esse motivo, estes testes foram orientados pelas *User Stories*, pelos requisitos do SRS e pelos casos de uso definidos no projeto.

A equipa começou por seleccionar os fluxos mais representativos de cada perfil de utilizador. Em seguida, esses fluxos foram transformados em cenários executáveis sobre a aplicação Spring real, usando dados de demonstração controlados e uma base de dados isolada para testes. Esta opção permitiu manter repetibilidade, ao mesmo tempo que se validava o comportamento da aplicação em condições próximas das reais.

Foram automatizados cenários representativos para funcionário de loja, gerente e gestor, incluindo:

- login, venda, fatura, fecho de caixa e sincronização;
- alerta de stock, encomenda, entrada de mercadoria e stock atualizado;
- inventário físico, registo de contagem, identificação de discrepância e ajuste.

Estes cenários permitiram verificar se os critérios de aceitação estavam cumpridos de ponta a ponta. Por exemplo, no caso de um fluxo de venda, não bastava testar a criação da venda isoladamente; era necessário confirmar a autenticação do utilizador, a seleção de produtos, o cálculo de totais, a finalização, a emissão de fatura e os efeitos resultantes no sistema. Do mesmo modo, nos fluxos de stock e encomendas, a equipa verificou se as operações produziam alterações persistentes e coerentes com o comportamento esperado.

A existência destes testes ajudou a equipa a validar o sistema a partir da perspetiva do utilizador final. Sempre que uma funcionalidade era alterada ou refatorada, os testes de aceitação serviam como garantia de que o objetivo funcional permanecia cumprido, mesmo que a implementação interna tivesse sido modificada.

No conjunto, esta estratégia permitiu combinar verificação técnica e validação funcional. A

equipa testou o sistema em diferentes níveis de detalhe, começando pelas unidades mais pequenas e terminando nos fluxos completos descritos pelos requisitos. Esta progressão tornou o processo de testes mais organizado, facilitou a identificação de defeitos e contribuiu para uma maior confiança na qualidade final da solução.

5.6 Uso de LLM para Criação de Testes

A criação e expansão da suite de testes foi apoiada por um LLM, usado como ferramenta auxiliar de análise e geração inicial de cenários. O modelo não substituiu a validação manual, o que obrigou a que cada teste sugerido fosse revisto, adaptado às convenções do projeto e executado localmente antes de ser aceite.

As estratégias de *prompting* usadas privilegiaram pedidos com contexto explícito da classe em teste, dependências relevantes, regras de negócio, casos já cobertos e objetivo de cobertura. Em vez de pedir testes genéricos, foram usados prompts orientados a comportamento, como por exemplo: identificar classes de equivalência, sugerir cenários de fronteira, listar caminhos negativos, verificar permissões por perfil e propor invariantes de domínio que deveriam permanecer verdadeiras após alterações.

O LLM foi também usado para apoiar a leitura dos relatórios de cobertura, ajudando a localizar zonas pouco exercitadas e a transformar essas lacunas em testes concretos. Esta abordagem permitiu reforçar cenários sobre fachadas, controladores, validação de pedidos, tratamento de exceções, sincronização, encomendas e entidades de domínio, mantendo os testes alinhados com o comportamento esperado e não apenas com detalhes internos da implementação.

No caso dos testes de mutação, o LLM foi usado para apoiar a configuração do PIT e a escolha de um subconjunto representativo de classes. O objetivo foi produzir um relatório demonstrável e rápido de executar, evitando que os testes de aceitação com contexto Spring completo tornassem a análise de mutantes demasiado demorada. Os mutantes continuaram a ser gerados automaticamente pelo PIT; o LLM apenas auxiliou na definição do alvo da execução e na interpretação dos resultados.

5.7 Cobertura de Código

A cobertura de código é monitorizada com **JaCoCo**, gerada automaticamente por `mvn clean verify`. O resumo visual da cobertura fica registado na Figura 5.1.

O `pom.xml` passou a aplicar limiares mínimos globais para linhas e ramos, mantendo também um limiar específico para o pacote de domínio. Assim, a cobertura deixa de ser apenas uma métrica informativa e passa a bloquear a verificação caso a suite regrida abaixo dos valores definidos.

Para além da cobertura por instruções, foi configurado **PIT Mutation Testing** no `pom.xml`, usando `pitest-maven` e o *plugin* JUnit 5. A execução foi delimitada a um conjunto representativo e rápido de classes de domínio, stock, sincronização e tratamento de erros, de forma a produzir um resumo de mutação adequado para evidência no relatório sem executar a aplicação inteira. O comando usado foi `mvn test-compile org.pitest:pitest-maven:mutationCoverage`.

O PIT gerou um relatório com mutantes mortos, sobreviventes e sem cobertura, permitindo avaliar a capacidade da suite em detetar alterações artificiais no código. Os relatórios HTML ficam disponíveis em `target/site/jacoco/index.html` e `target/pit-reports/index.html`.

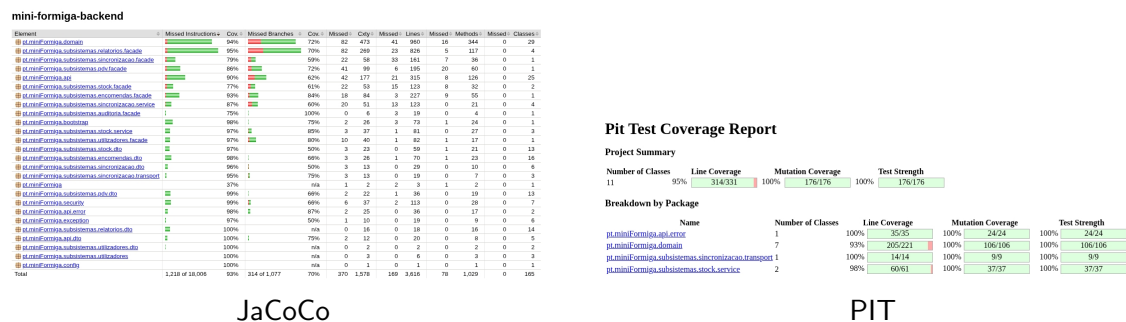


Figura 5.1: Relatórios de cobertura e testes de mutação

5.8 Avaliação segundo ISO/IEC 25010

A avaliação da qualidade do sistema Mini-Formiga foi conduzida com base no modelo de qualidade definido pela norma ISO/IEC 25010:2011, que estabelece um conjunto de características e subcaracterísticas para a avaliação de produtos de *software*. Este modelo foi utilizado como referencial estruturante para verificar em que medida o sistema desenvolvido satisfaz os atributos de qualidade relevantes para o contexto de uma cadeia de lojas de conveniência.

Das oito características de qualidade definidas pela norma, foram selecionadas para avaliação aquelas com maior impacto direto nos requisitos não funcionais especificados no documento SRS: **adequação funcional, desempenho e eficiência, fiabilidade, segurança, manutibilidade e usabilidade**. Para cada característica, foram definidos critérios de avaliação mensuráveis, derivados dos requisitos não funcionais identificados na Secção 2, e recolhidas evidências com base nos resultados dos testes executados e nas métricas de cobertura obtidas.

As características de compatibilidade e portabilidade não foram analisadas de forma autónoma, por terem menor impacto direto no âmbito funcional entregue e por estarem parcialmente cobertas pelas decisões arquiteturais de execução local/central e pela utilização de tecnologias multiplataforma.

A avaliação foi suportada por quatro tipos de evidência: execução da suite automatizada do

backend, validação da compilação e análise estática do frontend, análise de cobertura JaCoCo e relatório de testes de mutação PIT, presentes nas *sections* e *subsections* passadas.

Característica	Critério aplicado	Evidência recolhida	Resultado
Adequação funcional	Capacidade de executar os fluxos principais definidos no SRS: PDV, faturação, stock, encomendas, inventário, relatórios, utilizadores e sincronização.	A suite inclui testes de domínio, fachadas e controladores REST, bem como o teste <code>SistemaApiAcceptanceTest</code> , que percorre autenticação, venda, emissão de fatura, fecho de caixa, sincronização, alerta de stock, encomenda, entrada de mercadoria e inventário físico.	Elevado
Desempenho e eficiência	Operações críticas, em especial o PDV, devem responder de forma compatível com a utilização em loja.	Os testes automatizados exercitam os fluxos de venda e finalização, incluindo uma verificação temporal explícita em <code>SubPDVFacadeTest</code> que valida as operações críticas do PDV abaixo do limite de 2 segundos em condições normais de execução.	Bom
Fiabilidade	Continuidade da operação local, tratamento de falhas de sincronização, preservação de dados e comportamento previsível em cenários de erro.	A arquitetura local com SQLite permite operação independente do servidor central. Os testes de PDV e sincronização validam que uma falha de comunicação não impede a finalização da venda, mantém a sincronização pendente, agenda nova tentativa, conclui sincronizações com sucesso e aplica a estratégia <i>last-write-wins</i> em caso de conflito; os testes de tratamento de exceções validam respostas de erro uniformes.	Bom

(continua na página seguinte)

Característica	Critério aplicado	Evidência recolhida	Resultado
Segurança	Autenticação obrigatória, autorização por perfil, proteção de rotas e comunicação segura entre instâncias.	Foram validados login com JWT, rejeição de acessos sem token, respostas 401/403, permissões por perfil e isolamento de operações por loja. A cifra TLS foi assumida como responsabilidade de <i>deployment</i> , conforme documentado na Secção de Decisões e Adaptações da Implementação.	Bom
Manutenibilidade	Organização modular, testabilidade, deteção de regressões e controlo de dívida técnica.	O backend encontra-se organizado por subsistemas e fachadas, com testes unitários, integração e aceitação. JaCoCo e PIT fornecem métricas objetivas de cobertura. A análise de <i>code smells</i> identificou dívida técnica localizada, em particular na <i>RelatoriosFacade</i> e em alguns componentes React extensos.	Bom
Usabilidade	Interface coerente com os perfis de utilizador e capacidade de executar operações essenciais sem complexidade desnecessária.	O frontend disponibiliza áreas separadas para gestor, gerente, funcionário e responsável de armazém, com fluxos específicos para PDV, stock, relatórios, encomendas e inventário. O foco principal foi a aplicação de boas práticas de usabilidade para garantir uma experiência intuitiva e eficiente para o utilizador.	Bom

Tabela 5.1: Avaliação da qualidade segundo ISO/IEC 25010

De forma geral, conclui-se que o sistema apresenta qualidade adequada para o âmbito académico e profissional e para a demonstração funcional dos principais processos de negócio. As dimensões mais fortes são a adequação funcional, a fiabilidade local, a segurança aplicacional e a manutenibilidade suportada por testes automatizados. O desempenho do PDV e a continuidade operacional local encontram-se validados no âmbito da solução entregue, embora devam continuar a ser monitorizados em produção. As dimensões que exigiriam validação adicional num contexto de produção são sobretudo a usabilidade com utilizadores finais e a configuração efetiva de TLS no ambiente de implantação.

5.9 Verificação da satisfação dos SRS

A verificação da satisfação dos requisitos especificados no documento SRS constitui a etapa final do processo de validação do sistema Mini-Formiga. O objetivo desta análise é confirmar que cada requisito funcional, não funcional e de domínio especificado no Capítulo 2 se encontra efetivamente implementado e validado através de testes executáveis, garantindo a rastreabilidade entre a especificação e o sistema entregue.

A verificação foi conduzida por cruzamento entre a tabela de requisitos (Tabelas 2.10, 2.11 e 2.12) e os casos de teste definidos e executados ao longo das seções anteriores deste capítulo. Para cada requisito, foi registado o estado de satisfação: *satisfeito*, *parcialmente satisfeito* ou *não satisfeito*, acompanhado da referência ao teste ou conjunto de testes que suporta essa classificação. Os requisitos classificados como *Must Have* foram objeto de verificação obrigatória, sendo os restantes verificados na medida em que foram implementados dentro do âmbito do projeto.

O resultado desta análise é apresentado nas Tabelas 5.2, 5.3 e 5.4. A classificação ***satisfeito*** foi atribuída quando existe implementação funcional e evidência automatizada suficiente, ou quando o requisito é assegurado por uma decisão arquitetural validada por testes; ***parcialmente satisfeito*** foi atribuída quando a funcionalidade existe apenas em parte, quando depende de configuração externa ou quando não existe evidência suficiente no âmbito do projeto; e ***não satisfeito*** seria atribuída a requisitos sem implementação observável, situação que não se verificou nos requisitos obrigatórios centrais.

ID	Estado	Evidência principal	Observação
RF-01	<i>Satisfeito</i>	Controlador e fachada de relatórios, dashboard do gestor e testes RelatoriosControllerTest e SubRelatoriosFacadeTest.	Indicadores consolidados por loja e período.
RF-02	<i>Satisfeito</i>	Exportação em /relatorios/exportar, testes de CSV, PDF e XLSX na fachada de relatórios, e botões de exportação no frontend.	O RF exige PDF e CSV; XLSX foi também implementado.

(continua na página seguinte)

ID	Estado	Evidência principal	Observação
RF-03	<i>Parcialmente satisfeito</i>	Backend de fornecedores, condições comerciais e encomendas validado por OperacoesControllersTest e SubEncomendasFacadeTest; o frontend já permite criar fornecedores e submeter encomendas manuais ou consolidadas.	A gestão explícita de condições comerciais no frontend ainda é limitada, uma vez que não existe um ecrã completo para configurar preço, prazo de entrega e quantidade mínima por fornecedor/produto.
RF-04	<i>Satisfeito</i>	Frontend React com layout responsivo e acesso web aos perfis principais.	Foram executados testes em modo de emulação mobile através das ferramentas de desenvolvimento do browser.
RF-05	<i>Satisfeito</i>	SubStockFacadeTest, StockInventariosControllerTest e ecrãs de stock validam níveis mínimos e alertas ativos.	Inclui prevenção de alertas duplicados.
RF-06	<i>Satisfeito</i>	Fluxo de entrada de mercadoria coberto por SistemaApiAcceptanceTest, OperacoesControllersTest e SubEncomendasFacadeTest.	Atualiza stock após receção.
RF-07	<i>Satisfeito</i>	APIs de stock e alertas, testes de stock e frontend de gerente/armazém.	Identifica produtos abaixo do mínimo.
RF-08	<i>Satisfeito</i>	Testes de fecho de caixa em SubPDVDomainTest, SubPDVFacadeTest, OperacoesControllersTest e aceitação via API.	Agrega por meio de pagamento.
RF-09	<i>Satisfeito</i>	Relatórios de vendas filtráveis por loja, produto, turno e período em SubRelatoriosFacadeTest.	Também exposto na interface de relatórios.
RF-10	<i>Satisfeito</i>	UtilizadoresControllerTest, SubUtilizadoresFacadeTest e página de gestão de funcionários.	Inclui restrições por perfil e por loja.

(continua na página seguinte)

ID	Estado	Evidência principal	Observação
RF-11	<i>Satisfeito</i>	Ajustes de stock com motivo e utilizador em SubStockFacadeTest, StockInventariosControllerTest e teste de aceitação.	Regista operação em auditoria.
RF-12	<i>Satisfeito</i>	Fluxo PDV coberto por SubPDVFacadeTest, VendasControllerTest e SistemaApiAcceptanceTest.	Pesquisa por código de barras e código interno.
RF-13	<i>Satisfeito</i>	Testes de finalização de venda com numerário, cartão e MB Way em SubPDVFacadeTest.	Meios de pagamento modelados no domínio.
RF-14	<i>Satisfeito</i>	Emissão de fatura e documento fiscal validada por SubPDVDomainTest, SubPDVFacadeTest, FaturaPersistenceTest e aceitação.	Aplica regras RD-01 a RD-03.
RF-15	<i>Satisfeito</i>	Remoção/anulação de linhas antes da finalização validada em SubPDVFacadeTest e regras de domínio da venda.	Recalcula totais da transação.
RF-16	<i>Satisfeito</i>	Backend de devoluções coberto por SubPDVFacadeTest e DTOs/API de venda.	A interface exige introdução manual do UUID da venda, reduzindo a usabilidade do fluxo.
RF-17	<i>Satisfeito</i>	Perfis local/central, SQLite local, sincronização pendente, transporte REST e estratégia <i>last-write-wins</i> validados por testes de sincronização.	A automação operacional depende da configuração de <i>deployment</i> .
RF-18	<i>Satisfeito</i>	ProdutoLocalizacaoControllerTest, GlobalProdutoLojaStockStoreTest e formulário de localização no frontend.	Guarda corredor e prateleira por loja/produto.
RF-19	<i>Satisfeito</i>	Inventário físico, contagens, discrepâncias e fecho cobertos por SubStockFacadeTest, StockInventariosControllerTest e aceitação.	Permite posterior ajuste de stock.

Tabela 5.2: Verificação dos requisitos funcionais

ID	Estado	Evidência principal	Observação
RNF-01	<i>Satisfeito</i>	Fluxos PDV executados por testes automatizados e aceitação via API, complementados por SubPDVFacadeTest, que valida as operações críticas de venda dentro do limite de 2 segundos em condições normais de execução.	O requisito fica satisfeito no âmbito dos testes automatizados do projeto; em produção, o tempo de resposta deve continuar a ser monitorizado.
RNF-02	<i>Satisfeito</i>	Perfil local com SQLite, operação independente do servidor central e testes que confirmam finalização de venda com falha de sincronização, estado PENDENTE e agendamento de nova tentativa.	A disponibilidade local é assegurada pela arquitetura offline-first; a medição mensal deve ser acompanhada em exploração.
RNF-03	<i>Satisfeito</i>	SubSincronizacaoFacadeTest, ServidorCentralSincronizacaoServiceTest, RestSincronizacaoTransporteTest e aceitação via API.	Regista histórico e conflitos.
RNF-04	<i>Satisfeito</i>	AuthControllerTest, SecurityConfigTest, JwtServiceTest e testes de permissões em utilizadores.	Inclui respostas 401 e 403.
RNF-05	<i>Satisfeito</i>	Subsistema de auditoria, logs configurados em Logback e verificações em testes de PDV, stock e encomendas.	Log append-only em JSONL.
RNF-06	<i>Parcialmente satisfeito</i>	Transporte de sincronização suporta URL HTTPS e token de sincronização.	TLS não é ativado no ambiente local; fica a cargo do <i>reverse proxy</i> /deployment.
RNF-07	<i>Parcialmente satisfeito</i>	Dados pessoais limitados, autenticação, perfis de acesso e minimização de dados de cliente na faturação.	Não foi realizada auditoria jurídica profissional nem implementado ciclo formal de eliminação de dados.
RNF-08	<i>Satisfeito</i>	Testes de faturação validam tipo de fatura, NIF, totais, IVA e sequência ininterrupta.	Ver também RD-01 a RD-03.

(continua na página seguinte)

ID	Estado	Evidência principal	Observação
RNF-09	<i>Satisfeito</i>	Arquitetura central com PostgreSQL, UUIDs e agregação por loja; testes com múltiplas lojas nos relatórios e sincronização.	O sistema foi construído com foco no escalamento de lojas.
RNF-10	<i>Parcialmente satisfeito</i>	Interface PDV implementada e integrada com backend.	Falta teste de usabilidade com funcionários reais após sessão de treino de 30 minutos.
RNF-11	<i>Satisfeito</i>	Exportações CSV e XLSX validadas em SubRelatoriosFacadeTest, com campos contábilísticos normalizados.	O frontend disponibiliza exportação em CSV, PDF e XLSX.

Tabela 5.3: Verificação dos requisitos não funcionais

ID	Estado	Evidência principal	Observação
RD-01	<i>Satisfeito</i>	Entidade TaxaIVA, associação obrigatória ao produto e testes TaxaIVATest e SubPDVFacadeTest.	Suporta taxas 6%, 13% e 23%.
RD-02	<i>Satisfeito</i>	Testes faturaDeveDecidirTipoPelaRegraRd02, emitirFaturaSimplificadaSemNifAteMilEuros e emitirFaturaCompletaQuandoClienteForneceNif.	Diferencia fatura simplificada e completa.
RD-03	<i>Satisfeito</i>	faturaSequenciaDeveIncrementarNumero e emissaoFaturaUsaSequenciaIninterruptaRd03.	Sequência por série fiscal.
RD-04	<i>Satisfeito</i>	ProdutoLojaTest, SubStockFacadeTest e SubPDVFacadeTest bloqueiam stock negativo.	Aplica-se em stock e venda.
RD-05	<i>Satisfeito</i>	Teste registrarVendaForaHorarioGeraAuditoria em SubPDVFacadeTest.	Transações fora de horário são sinalizadas.
RD-06	<i>Satisfeito</i>	SubEncomendasDomainTest e SubEncomendasFacadeTest validam processamento segundo horário do fornecedor.	Informa data esperada de processamento.

(continua na página seguinte)

ID	Estado	Evidência principal	Observação
RD-07	<i>Satisfeito</i>	Faturação recolhe apenas NIF e nome quando necessário; segurança por perfil limita acesso aos dados.	Validação formal de todos os processos RGPD, em particular retenção e eliminação.

Tabela 5.4: Verificação dos requisitos de domínio

Da verificação realizada resulta que todos os requisitos funcionais *Must Have* se encontram satisfeitos ou, nos casos em que dependem de condições externas de operação, satisfatoriamente suportados pela arquitetura implementada. Os requisitos assinalados como parcialmente satisfeitos correspondem sobretudo a funcionalidades de menor prioridade, a validações empíricas ainda não executadas ou a responsabilidades de infraestrutura que não pertencem diretamente ao código aplicacional.

Adicionalmente, foi realizada uma sessão de verificação com o cliente, o Sr. Formiga, na qual a equipa apresentou e demonstrou o funcionamento do sistema implementado, percorrendo os requisitos funcionais, não funcionais e de domínio especificados. Esta sessão permitiu confirmar, do ponto de vista do utilizador final, que o sistema entregue corresponde às necessidades operacionais comunicadas ao longo do projeto, bem como esclarecer dúvidas levantadas pelo cliente durante a demonstração. A ata desta reunião encontra-se disponível no Anexo 6.17.

6 Conclusões e Trabalho Futuro

6.1 Apreciação Crítica do Trabalho Realizado

O presente projeto permitiu desenvolver, de forma estruturada e iterativa, a especificação completa do sistema Mini-Formiga, uma plataforma para a gestão integrada de cadeias de lojas de conveniência. Ao longo das diferentes fases do ciclo de vida do *software*, a equipa percorreu um processo que vai desde a elicitação de requisitos junto de *stakeholders* simulados até à definição de uma arquitetura coesa e fundamentada, incluindo a modelação de casos de uso, diagramas de classes, diagramas de sequência e especificação de API.

Uma das contribuições mais relevantes deste trabalho foi a integração sistemática de agentes LLM (Large Language Models) ao longo de todas as atividades de Engenharia de Requisitos e *Design* Arquitetural. A equipa adotou uma metodologia deliberada e transparente: o agente principal foi utilizado como ferramenta de elicitação e estruturação, enquanto um segundo agente independente desempenhou o papel de revisor crítico, identificando ambiguidades e inconsistências que escaparam à análise inicial. Esta abordagem de validação cruzada, documentada na Secção 2, demonstrou que a integração de LLMs no processo de engenharia de *software* é tanto mais eficaz quanto mais estruturada e reflexiva for a interação humana com o modelo, sendo os resultados gerados sempre sujeitos a revisão, seleção e validação pela equipa.

Do ponto de vista arquitetural, as decisões tomadas revelam maturidade técnica aliada à sensibilidade ao contexto de negócio. A opção pelo padrão *Facade* por subsistema em detrimento de uma arquitetura de microserviços é um exemplo paradigmático desta postura: a equipa reconheceu que a solução tecnicamente mais sofisticada nem sempre é a mais adequada para o contexto de uma PME em fase de expansão, onde a simplicidade operacional e a facilidade de manutenção têm tanto valor como a escalabilidade teórica. De forma semelhante, a adoção de sincronização por lote e resolução de conflitos por *last-write-wins* refletem um equilíbrio consciente entre robustez, desempenho e complexidade de implementação.

Pontos Fortes

O trabalho apresenta um conjunto de pontos fortes que merecem destaque.

Em primeiro lugar, a **rastreabilidade transversal** entre os diferentes artefactos produzidos. Cada requisito funcional pode ser rastreado até à *User Story* que lhe deu origem, à entrevista

que a fundamentou, ao caso de uso que o operacionaliza e às classes do diagrama que o suportam. Esta cadeia de rastreabilidade, raramente conseguida com esta consistência em projetos académicos, constitui uma garantia de coerência interna do documento e uma base sólida para a fase de verificação e validação.

Em segundo lugar, a **qualidade da especificação de requisitos**. Os requisitos funcionais e não funcionais foram formulados de acordo com os critérios do padrão IEEE 830, completude, consistência, verificabilidade e ausência de ambiguidade, e todos os requisitos não funcionais incluem critérios de aceitação mensuráveis.

Adicionalmente, destaca-se o **rigor na modelação do domínio**. A filosofia de elevação de conceitos a entidades autónomas sempre que apresentam ciclo de vida próprio ou são necessários para fins de auditoria, resulta num modelo capaz de responder a questões de negócio relevantes sem recorrer a informação inferida ou reconstruída a posteriori.

Por fim, evidencia-se a **coerência arquitetural** entre os diferentes níveis de abstração. O diagrama de componentes, o diagrama de classes e a especificação de API são consistentes entre si, utilizando a mesma nomenclatura, os mesmos contratos de interface e os mesmos padrões de organização modular, o que reduz significativamente o risco de ambiguidade na fase de implementação.

Pontos Fracos e Limitações

O trabalho apresenta igualmente um conjunto de limitações que importa reconhecer com honestidade.

A principal limitação reside na **natureza simulada das entrevistas com stakeholders**. Embora o agente LLM tenha sido instruído a adotar o perfil de cada interveniente de forma contextualizada, a profundidade e autenticidade das respostas obtidas são inevitavelmente inferiores às de entrevistas reais. É possível que necessidades implícitas não tenham sido captadas, e que alguns requisitos reflitam mais as convenções do domínio do retalho em geral do que as especificidades concretas do negócio do Sr. Formiga.

Uma segunda limitação prende-se com a **ausência de protótipos validados com utilizadores reais**. Os *mockups* de interface apresentados foram concebidos com base nos requisitos especificados, mas não foram sujeitos a testes de usabilidade com os perfis de utilizador reais, nomeadamente funcionários de loja com baixa literacia tecnológica. O requisito RNF-10, que estabelece que um funcionário sem formação técnica deve conseguir operar o PDV após 30 minutos de formação, permanece por validar empiricamente.

6.2 Trabalho Futuro

O sistema Mini-Formiga, tal como especificado neste documento, constitui uma base funcional e arquiteturalmente sólida para o negócio do Sr. Formiga na sua fase atual de expansão. Contudo, a evolução natural do negócio e as limitações identificadas sugerem um conjunto de direções para trabalho futuro que merecem consideração.

Validação de usabilidade com utilizadores reais

A realização de sessões de teste de usabilidade com funcionários de loja reais, recorrendo a protótipos de alta fidelidade da interface PDV, permitiria validar empiricamente o requisito RNF-10 e identificar dificuldades de interação não antecipadas na especificação. Esta validação é tanto mais urgente quanto a eficiência do ponto de venda tem impacto direto na satisfação dos clientes e na rentabilidade do negócio.

Aplicação móvel nativa para o Gestor da Cadeia

O requisito RF-04 especifica o suporte a dispositivos móveis através de uma interface responsiva. Uma evolução natural seria o desenvolvimento de uma aplicação móvel nativa para iOS e Android, que permitisse ao Gestor da Cadeia receber notificações *push* de alertas de *stock*, aprovar encomendas e consultar indicadores em tempo real, independentemente de dispor de um *browser* disponível.

Conformidade com a Faturação Eletrónica Estruturada

A legislação portuguesa prevê a adoção progressiva da faturação eletrónica estruturada em formato CIUS-PT, compatível com a norma europeia EN 16931. A adaptação do módulo de faturação do Mini-Formiga a este formato, quando obrigatório para o segmento de empresas em que o Sr. Formiga se insere, garantirá a continuidade da conformidade fiscal sem necessidade de redesenho arquitetural.

6.3 Reflexão Final

O projeto Mini-Formiga representou, para a equipa, uma oportunidade singular de percorrer de forma integrada todas as etapas do ciclo de vida do desenvolvimento de *software*, desde a escuta ativa das necessidades de um cliente até à definição de uma arquitetura pronta a implementar. A complexidade inerente a um sistema distribuído com operação *offline*, sincro-

nização de dados e múltiplos perfis de utilizador exigiu um nível de rigor e consistência que elevou significativamente a maturidade da equipa de desenvolvimento.

A integração de agentes LLM ao longo de todo o processo **revelou-se simultaneamente uma vantagem e um desafio**. A vantagem reside na celeridade de geração de artefactos estruturados e na capacidade dos modelos de identificar ambiguidades e inconsistências com uma perspetiva externa. O desafio reside precisamente na necessidade de manter um pensamento crítico constante sobre os resultados produzidos, resistindo à tentação de aceitar acríticamente o *output* que, embora bem estruturado e plausível, pode não refletir com rigor as necessidades reais do negócio ou as restrições técnicas do sistema.

É esta tensão produtiva entre a capacidade generativa dos modelos e o julgamento humano da equipa que define, em última análise, a qualidade do trabalho realizado.

Consideramos portanto que o sistema Mini-Formiga está, assim, pronto para a sua próxima etapa. O caderno azul do Sr. Formiga pode, finalmente, ser reformado.

Referências

- [1] IEEE/ISO/IEC, *ISO/IEC/IEEE 29148:2018 — Systems and Software Engineering — Life Cycle Processes — Requirements Engineering*. IEEE, 2018.
- [2] IEEE, *IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications*. IEEE, 1998.
- [3] I. Sommerville, *Software Engineering*, 10.^a ed. Pearson Education, 2016.
- [4] R. S. Pressman e B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9.^a ed. McGraw-Hill Education, 2020.
- [5] ISO/IEC, *ISO/IEC 25010:2011 — Systems and Software Engineering — Systems and Software Quality Requirements and Evaluation (SQuaRE) — System and Software Quality Models*. ISO, 2011.
- [6] Object Management Group (OMG), *Unified Modeling Language Specification, Version 2.5.1*. OMG, 2017. Disponível em: <https://www.omg.org/spec/UML/2.5.1>
- [7] M. Cohn, *User Stories Applied: For Agile Software Development*. Addison-Wesley, 2004.
- [8] E. Evans, *Domain-Driven Design: Tackling Complexity in the Heart of Software*. Addison-Wesley, 2003.
- [9] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," tese de doutoramento, University of California, Irvine, 2000. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- [10] Parlamento Europeu e Conselho da União Europeia, *Regulamento (UE) 2016/679 — Regulamento Geral sobre a Proteção de Dados (RGPD)*, Jornal Oficial da União Europeia, abril de 2016.
- [11] D. Hou, S. Zhao, H. Jin et al., "Large Language Models for Software Engineering: A Systematic Literature Review," *ACM Transactions on Software Engineering and Methodology*, 2024. DOI: 10.1145/3695988

Lista de Siglas e Acrónimos

«Apresentar uma lista com todas as siglas e acrónimos utilizados durante a realização do trabalho. O formato base para esta lista deverá ser da forma como abaixo se apresenta.»

BD Base de Dados

DW Data Warehouse

OLTP On-Line Analytical Processing

Sr Senhor

SRS Especificação de Requisitos de *Software*

PDV Ponto de Venda

Resp Responsável

LLM Large Language Model

Anexos

Anexo 1 — Entrevistas com os *Stakeholders*

O presente anexo reúne as transcrições das entrevistas simuladas realizadas com os principais *stakeholders* do sistema Mini-Formiga. As entrevistas foram conduzidas pela equipa de desenvolvimento com o objetivo de levantar requisitos funcionais e não funcionais, e serviram de base à construção das *User Stories* apresentadas no Capítulo de Definição de Requisitos.

Legenda:

E — Entrevistador (equipa de desenvolvimento)

S — *Stakeholder*

Entrevista 1 — Proprietário / Gestor da Cadeia (Sr. Formiga)

Data: 16 de fevereiro de 2026

Duração: aproximadamente 45 minutos

Formato: presencial

Local: escritório da loja Mini-Formiga

E: Bom dia, Sr. Formiga. Antes de mais, obrigado por nos receber. Gostaríamos de perceber melhor como gere atualmente a sua loja e quais as principais dificuldades que enfrenta, sobretudo tendo em conta os planos de expansão que nos mencionou.

S: Bom dia. Com todo o gosto. Olhe, enquanto era só esta loja, ia gerindo à base da experiência e da memória. Mas agora que estou a planear abrir mais estabelecimentos, começo a perceber que o que tenho não chega. Os registos de vendas estão espalhados por vários ficheiros *Excel*, sem nenhuma organização consistente, o *stock* está anotado em documentos separados por mês, e os contactos dos fornecedores estão distribuídos entre e-mails e uma agenda de papel. No fundo, tudo muito disperso.

E: E como é feito o controlo diário das vendas?

S: Tenho um caderno onde registo o fecho de caixa todas as noites. Funciona para uma loja, mas se tiver dois ou três estabelecimentos, não consigo estar em todo o lado. Precisava de conseguir ver, a partir de um único ponto, o que está a acontecer em cada loja, sem depender de telefonemas ou de compilar ficheiros manualmente.

E: Quando imagina esse ponto de acesso centralizado, que informação considera mais importante ter disponível?

S: Quero uma pagina com as vendas de cada loja, a margem de lucro por produto, e perceber quais os estabelecimentos mais rentáveis. Idealmente com gráficos e filtros por período. E que me avise automaticamente quando o *stock* de alguma loja estiver a baixar — já perdi vendas por falta de produto e só me apercebi tarde demais.

E: E relativamente a fornecedores, como gere as encomendas atualmente?

S: É tudo por telefone ou e-mail, sem registo centralizado. Às vezes nem sei qual é o contacto certo de um determinado fornecedor. Se o sistema tivesse uma lista de fornecedores com os produtos de cada um, e me permitisse fazer encomendas consolidadas para todas as lojas de uma só vez, era uma enorme ajuda.

E: Tem também necessidade de exportar dados para o seu contabilista?

S: Sim, todos os meses é uma chatice. Ele pede-me um ficheiro com as vendas, as compras e as faturas, e eu tenho de juntar tudo manualmente a partir de sítios diferentes. Se o sistema gerasse isso automaticamente em CSV ou *Excel*, poupava-me muito tempo.

E: Uma última questão: utiliza habitualmente dispositivos móveis no trabalho?

S: Uso bastante o telemóvel. Quando estiver fora da loja, queria conseguir consultar como estão as vendas ou se há algum alerta de *stock*. Não é a minha prioridade principal, mas seria uma mais-valia.

E: Muito obrigado pela sua disponibilidade, Sr. Formiga. As informações que partilhou são muito valiosas para o projeto.

S: Obrigado a vocês. Qualquer coisa estão à vontade para voltar a contactar-me.

Entrevista 2 — Gerente de Loja

Data: 16 de fevereiro de 2026

Duração: aproximadamente 35 minutos

Formato: presencial

Local: loja Mini-Formiga

E: Boa tarde. Obrigado por nos receber. Gostaríamos de perceber como é o seu dia a dia como gerente e onde sente que um sistema informático poderia ajudar mais.

S: Boa tarde. Claro. O meu dia começa logo com a verificação do *stock* e a abertura de caixa. Faço tudo a olho e em papel. É demorado e sujeito a erros, principalmente quando há muita mercadoria para conferir.

E: Em termos de *stock*, que informação precisa de ter disponível?

S: Preciso de saber o que temos em armazém, o que está a acabar, e quais os produtos que os clientes pedem e não temos. Gostava de ter uma lista organizada, com alertas quando um produto estivesse abaixo de um mínimo que eu próprio pudesse definir.

E: E quando chega mercadoria nova, como é feito o registo?

S: O responsável de armazém preenche uma folha à mão com o que chegou, e depois eu passo isso para um ficheiro *Excel*. É um processo duplicado e muito propenso a erros. O ideal era que, quando a mercadoria chegasse, se registasse diretamente no sistema e o *stock* atualizasse de forma automática.

E: E o fecho de caixa, como é tratado?

S: No fim do dia conto o dinheiro, comparo com o que está registado nas vendas e vejo se bate certo. Registo isso num bloco de notas e no final da semana entrego ao Sr. Formiga juntamente com os ficheiros de vendas. Se houvesse uma totalização automática das vendas do dia, poupava-me tempo e havia mais segurança nos números.

E: Tem funcionários a trabalhar consigo. Como gere os seus acessos ao sistema informático?

S: Neste momento toda a gente usa o mesmo computador com as mesmas credenciais. É um problema de segurança que reconheço. Precisava de poder definir o que cada funcionário pode ou não fazer no sistema — quem pode só registar vendas, quem pode ver relatórios, etc.

E: E em relação a relatórios de vendas, que análise considera mais útil?

S: Gostava de ver as vendas por produto, para perceber o que vende mais. Também por turno, para saber as alturas do dia com mais movimento. E por período — semana, mês —, para comparar com períodos anteriores.

E: Por fim, já alguma vez ficou sem acesso ao sistema informático? Como foi gerido?

S: Já aconteceu a internet falhar e ficámos sem conseguir fazer nada durante horas. Foi um caos. O sistema tem mesmo de funcionar sem ligação à internet — isso é indispensável.

E: Muito obrigado pela sua colaboração. As suas respostas foram extremamente úteis.

Entrevista 3 — Funcionário (Operador de PDV)

Data: 17 de fevereiro de 2026

Duração: aproximadamente 20 minutos

Formato: presencial

Local: balcão de atendimento, loja Mini-Formiga

E: Boa tarde. Sabemos que está a trabalhar, por isso vamos ser breves. Queríamos perceber como é feito o registo de vendas atualmente.

S: Boa tarde. Sim, claro. Neste momento uso a caixa registadora para as vendas normais e, quando há alguma transação pouco habitual, o gerente regista à parte num ficheiro. É lento, especialmente quando há fila de espera.

E: O que considera mais importante numa interface de ponto de venda?

S: Que seja rápida e fácil de usar. Não tenho muito tempo para andar à procura de botões. Se conseguir registar um produto com um código de barras ou com uma pesquisa rápida, é o ideal. E que não bloqueie — nada pior do que o sistema travar quando há clientes à espera.

E: Em relação aos meios de pagamento, quais são aqueles que os clientes utilizam habitualmente?

S: A maioria paga em numerário ou em cartão. Mas há cada vez mais clientes que querem pagar com MB Way. Por isso o sistema tem mesmo de aceitar os três.

E: E se um cliente pedir fatura ou recibo, como é feito esse processo?

S: É muito manual — escrevemos à mão num livro de recibos que temos no balcão. Era muito melhor se o sistema emitisse automaticamente um recibo ou uma fatura simplificada com os dados da venda.

E: Alguma vez se enganou ao registar um produto e precisou de corrigir antes de fechar a venda?

S: Muitas vezes. Por exemplo, leio o código errado ou o cliente muda de ideias. Preciso de conseguir apagar um artigo da lista antes de fechar a venda, sem ter de anular tudo e começar do início.

E: E devoluções? Acontecem com frequência?

S: Não muito, mas acontecem. Normalmente é o gerente que trata disso, mas seria útil que eu também conseguisse registar uma devolução no sistema de forma simples.

E: Obrigado pelo seu tempo. Foi muito prestável.

Entrevista 4 — Responsável de Armazém

Data: 17 de fevereiro de 2026

Duração: aproximadamente 30 minutos

Formato: presencial

Local: armazém da loja Mini-Formiga

E: Bom dia. Obrigado por nos receber. Pode descrever-nos como é o seu processo de trabalho diário no armazém?

S: Bom dia. Claro. Recebo a mercadoria dos fornecedores, confirmo se o que chegou corresponde ao que foi encomendado, e registo tudo numa folha de papel que depois entrego ao gerente. Quando há alguma diferença entre o encomendado e o recebido, anoto à parte.

E: Qual é a principal dificuldade nesse processo?

S: É mesmo o registo manual. Às vezes as guias de remessa têm dezenas de linhas e confirmar produto a produto é muito demoroso. Depois ainda tenho de escrever tudo à mão para entregar ao gerente, que passa para o *Excel*. Um sistema onde eu pudesse registar as entradas diretamente, associando à guia de remessa, seria uma enorme ajuda para todos.

E: Em relação ao inventário físico, com que frequência é feito e como se processa?

S: Fazemos um inventário completo uma vez por mês. Contamos tudo fisicamente e comparamos com o que está no ficheiro *Excel* do gerente. Quando há diferenças — e há quase sempre —, é difícil perceber onde aconteceu o erro. Precisava de poder aceder ao inventário onde pudesse registar as contagens e o sistema identificasse automaticamente as discrepâncias face ao *stock* registado.

E: Tem noção de onde estão todos os produtos no armazém? Existe algum registo de localização?

S: Sei de cor, mas se estiver de férias ou faltar um dia, os outros não sabem onde está cada coisa. Era útil ter uma forma de registar a localização de cada produto no armazém — por exemplo, por corredor e prateleira —, para facilitar a reposição na loja.

E: Muito obrigado pela sua disponibilidade. As informações que partilhou são muito importantes para o sistema que estamos a desenvolver.

S: Obrigado. Espero que o sistema facilite mesmo o trabalho, porque há muito para melhorar.

Entrevista 5 — Contabilista (Responsável Financeiro Externo)

Data: 18 de fevereiro de 2026

Duração: aproximadamente 25 minutos

Formato: videoconferência

E: Boa tarde. Obrigado por aceitar esta reunião por videoconferência. A sua perspetiva como responsável financeiro externo é muito importante para garantirmos que o sistema responde às necessidades de contabilidade.

S: Boa tarde. Com prazer. Digam lá o que precisam.

E: Que tipo de dados precisa habitualmente do Sr. Formiga para fechar a contabilidade mensal?

S: Preciso dos totais de vendas por período, dos registos de compras a fornecedores e das faturas emitidas. Neste momento o Sr. Formiga envia-me um ficheiro *Excel* que ele próprio compila manualmente a partir de vários sítios — o caderno de caixa, os ficheiros de vendas, as faturas em papel. Demora sempre tempo e nem sempre os dados estão todos lá.

E: Que formato de exportação seria mais conveniente para si?

S: CSV ou *Excel* são os que utilizo habitualmente. Desde que os campos estejam bem organizados — data, descrição, valor, IVA — qualquer um dos dois serve. O importante é que eu não precise de reformatar nada depois de receber.

E: Quando o negócio expandir para várias lojas, essa necessidade altera-se?

S: Sim, passa a ser ainda mais crítico. Vou precisar que os dados venham discriminados por loja, para conseguir fazer a contabilidade de cada estabelecimento separadamente. Se o sistema conseguir gerar um relatório consolidado e também por loja, é exatamente o que preciso.

E: Tem necessidade de aceder diretamente ao sistema, ou prefere receber os dados por exportação?

S: Prefiro exportação. Não preciso de aceder regularmente ao sistema — apenas uma vez por mês para recolher os dados. Gerar relatórios poderia ser útil pontualmente, mas não é essencial.

E: Muito obrigado pelo seu contributo. Isto ajuda-nos bastante com a parte financeira do sistema.

S: De nada. Qualquer dúvida estou disponível.

Entrevista 6 — Representante de Fornecedor

Data: 19 de fevereiro de 2026

Duração: aproximadamente 20 minutos

Formato: telefónico

E: Boa tarde. Obrigado por aceitar esta breve conversa. Estamos a desenvolver um sistema de gestão para a cadeia Mini-Formiga e gostaríamos de perceber como funciona atualmente a comunicação entre a loja e os fornecedores.

S: Boa tarde. Claro. Neste momento a comunicação é toda por telefone ou e-mail. O Sr. Formiga contacta-nos quando precisa de alguma coisa, e nós respondemos com uma proposta ou confirmamos a encomenda.

E: Existem dificuldades nesse processo?

S: Às vezes as encomendas chegam-nos com informação incompleta — falta a referência do produto ou a quantidade não está clara. Isso gera trocas de mensagens desnecessárias. Se houvesse um sistema onde as encomendas chegassem com toda a informação estruturada, seria muito mais eficiente para ambas as partes.

E: Da vossa parte, costumam enviar catálogos ou listas de preços atualizadas?

S: Sim, enviamos por e-mail, geralmente em PDF ou *Excel*. Mas sei que os clientes têm dificuldade em manter esses dados atualizados. Se existisse uma forma de o Sr. Formiga importar as nossas listas de preços diretamente para o sistema dele, poupávamos trabalho dos dois lados.

E: Com a expansão planeada para várias lojas, como vê o impacto nas encomendas?

S: Seria mais fácil se as encomendas viessem consolidadas. Em vez de receber pedidos separados de cada loja, preferia receber uma encomenda única com as quantidades totais. Para nós é mais simples de processar e normalmente traduz-se em melhores condições para o cliente.

E: Muito obrigado pela disponibilidade. O seu contributo ajuda-nos a pensar na integração com fornecedores no sistema.

S: Com prazer. Boa sorte com o projeto.

Anexo 2 — Ata da reunião com o Sr. Formiga para Validação de requisitos

Ata - Sessão de Validação com o Sr. Formiga

A sessão de validação com o Sr. Formiga foi realizada após a consolidação das User Stories, dos requisitos funcionais, não funcionais e de domínio do sistema Mini-Formiga. O objetivo principal desta reunião consistiu em confirmar o alinhamento entre os requisitos especificados e as necessidades operacionais reais do negócio, bem como identificar formulações ambíguas ou insuficientemente detalhadas que pudessem comprometer a futura implementação e validação do sistema.

Durante a análise do requisito **RF-05**, relativo à monitorização dos níveis de stock e emissão de alertas automáticos, o Sr. Formiga manifestou preocupação com a utilização do termo “*continuamente*”. Na sua perspetiva, esta formulação não clarificava se a verificação seria realizada em tempo real, no momento de uma venda ou ajuste de inventário, ou apenas através de verificações periódicas. Neste contexto, salientou que o comportamento esperado deveria ser imediato, afirmando que o sistema deveria avisar logo no instante em que um produto atingisse o nível mínimo, evitando situações de rutura não detetadas a tempo.

No requisito **RF-17**, relacionado com a sincronização automática após restabelecimento da ligação de rede, o Sr. Formiga levantou uma questão relativa à consistência dos dados em caso de conflito entre alterações realizadas localmente na loja e dados já existentes no servidor central. Em particular, referiu que o requisito não deixava claro qual das versões da informação deveria prevalecer, observando que o sistema não poderia deixar esta decisão indefinida ou dependente do acaso. Esta observação levou à necessidade de explicitar a regra de resolução de conflitos a aplicar.

Relativamente ao requisito **RNF-10**, referente à usabilidade do terminal de ponto de venda, o Sr. Formiga considerou que a formulação inicial, baseada numa sessão de treino de 30 minutos, permanecia demasiado subjetiva. Na sua perspetiva, o requisito deveria definir de forma mais objetiva o que se entende por domínio suficiente da interface, sugerindo que a validação se focasse nas operações nucleares do PDV, nomeadamente registo de venda, processamento de pagamento e emissão de recibo, executadas sem assistência por um funcionário sem formação técnica prévia.

No final da sessão, as observações apresentadas pelo Sr. Formiga foram consideradas pertinentes pela equipa e deram origem às reformulações introduzidas no registo de alterações decorrentes do processo de validação. Estas clarificações contribuíram para reforçar a verificabilidade e a precisão dos requisitos, reduzindo ambiguidades antes da fase de arquitetura e implementação.

Figura 6.1: Ata da reunião com o Sr. Formiga

Anexo 3 — Relacionamentos do Modelo de Domínio do Mini-Formiga

Entidade A	Card.A	Relacionamento	Card.B	Entidade B	Notas	Origem
Utilizador	(1,N)	tem	(1,1)	Perfil	Cada utilizador tem exatamente um perfil; um perfil pode ser atribuído a vários utilizadores.	RNF-04; US-11
Utilizador	(1,N)	está associado a	(1,1)	Loja	Cada utilizador opera numa loja; uma loja tem pelo menos um utilizador.	RNF-04; US-11
Produto	(1,N)	pertence a	(1,1)	Categoria	Cada produto pertence a uma categoria; uma categoria agrupa vários produtos.	RF-02; US-03
Produto	(1,N)	tem	(1,1)	TaxaIVA	Cada produto tem uma taxa de IVA; uma taxa pode aplicar-se a vários produtos.	RF-14; RNF-08
Fornecedor	(1,N)	fornece	(1,N)	Produto	Um fornecedor fornece vários produtos; um produto pode ser fornecido por vários fornecedores.	RF-03; US-04
Condicao-Comercial	(1,1)	define	(1,1)	Fornecedor	Cada condição comercial está associada a um fornecedor específico.	RF-03; US-04

(continua na página seguinte)

Entidade A	Card. A	Relacionamento	Card. B	Entidade B	Notas	Origem
Condicao-Comercial	(1,1)	aplica-se a	(1,N)	Produto	Uma condição comercial aplica-se a pelo menos um produto; um produto pode ter condições distintas por fornecedor.	RF-03; US-04
Loja	(1,1)	possui	(1,N)	Stock	Cada loja possui <i>stock</i> de pelo menos um produto; cada registo de <i>stock</i> pertence a uma loja.	RF-07; US-08
Produto	(1,1)	tem	(0,N)	Stock	Um produto pode ter <i>stock</i> em várias lojas; um <i>stock</i> refere-se a um produto.	RF-07; US-08
Stock	(1,1)	tem definido	(0,1)	NivelMi-nimo	Um registo de <i>stock</i> pode ter um nível mínimo configurado; o nível mínimo pertence a um <i>stock</i> .	RF-05; US-02
NivelMi-nimo	(0,N)	desencadeia	(1,1)	AlertaStock	Um nível mínimo pode desencadear vários alertas ao longo do tempo; cada alerta resulta de um nível mínimo.	RF-05; US-02
AlertaStock	(1,N)	é recebido por	(1,N)	Utilizador	Um alerta é recebido por um ou mais utilizadores com perfil relevante; um utilizador pode receber vários alertas.	RF-05; US-02

(continua na página seguinte)

Entidade A	Card. A	Relacionamento	Card. B	Entidade B	Notas	Origem
Produto	(1,1)	tem	(0,1)	Localizacao-Produto	Um produto pode ter uma localização em armazém (opcional); uma localização refere-se a um produto.	RF-18; US-21
Loja	(1,1)	realiza	(0,N)	Venda	Cada venda ocorre numa loja; uma loja pode ainda não ter vendas (ex.: recém-aberta).	RF-12; US-13
Utilizador	(1,1)	registra	(0,N)	Venda	Cada venda é registada por um utilizador; um utilizador pode não ter vendas registadas.	RF-12; US-13
Venda	(1,1)	contém	(1,N)	LinhaVenda	Cada venda tem pelo menos uma linha; cada linha pertence a uma venda.	RF-12; RF-15; US-16
LinhaVenda	(1,N)	referencia	(1,1)	Produto	Cada linha de venda refere um produto; um produto pode aparecer em várias linhas de venda.	RF-12; US-13
Venda	(1,N)	é liquidada por	(1,1)	MeioPagamento	Cada venda é liquidada por um meio de pagamento; um meio de pagamento pode ser usado em várias vendas.	RF-13; US-14

(continua na página seguinte)

Entidade A	Card. A	Relacionamento	Card. B	Entidade B	Notas	Origem
Venda	(1,1)	origina	(0,1)	Fatura	Uma venda pode originar uma fatura (quando solicitada); uma fatura está associada a exatamente uma venda.	RF-14; RNF-08; US-15
FechoCaixa	(1,1)	consolida	(1,N)	Venda	Cada fecho de caixa agrega as vendas do dia; cada venda é consolidada num fecho de caixa.	RF-08; US-09
Loja	(1,1)	registra	(0,N)	FechoCaixa	Cada fecho pertence a uma loja; uma loja pode ainda não ter fechos registados.	RF-08; US-09
Utilizador	(1,1)	efetua	(0,N)	FechoCaixa	Cada fecho é efetuado por um utilizador; um utilizador pode nunca ter efetuado um fecho.	RF-08; US-09
Loja	(1,1)	efetua	(0,N)	Encomenda	Cada encomenda é efetuada por uma loja; uma loja pode ainda não ter encomendas.	RF-03; US-04
Encomenda	(1,N)	é dirigida a	(1,1)	Fornecedor	Cada encomenda é dirigida a um fornecedor; um fornecedor pode receber várias encomendas.	RF-03; US-04

(continua na página seguinte)

Entidade A	Card. A	Relacionamento	Card. B	Entidade B	Notas	Origem
Encomenda	(1,N)	tem	(1,1)	EstadoEncomenda	Cada encomenda tem um estado corrente; um estado pode corresponder a várias encomendas.	RF-03; US-04
Encomenda	(1,1)	contém	(1,N)	LinhaEncomenda	Cada encomenda tem pelo menos uma linha; cada linha pertence a uma encomenda.	RF-03; US-04
LinhaEncomenda	(1,N)	referencia	(1,1)	Produto	Cada linha de encomenda refere um produto; um produto pode figurar em várias linhas de encomenda.	RF-03; US-04
GuiaRemessa	(1,N)	está associada a	(1,1)	Encomenda	Cada guia de remessa refere uma encomenda; uma encomenda pode ter zero ou mais guias (entregas parciais).	RF-06; US-19
EntradaMercadoria	(1,N)	referencia	(1,1)	GuiaRemessa	Cada entrada de mercadoria referencia uma guia de remessa; uma guia pode corresponder a várias entradas.	RF-06; US-07; US-19
EntradaMercadoria	(1,N)	atualiza	(1,1)	Stock	Cada entrada atualiza o <i>stock</i> de pelo menos um produto; um <i>stock</i> pode ser atualizado por várias entradas.	RF-06; US-07

(continua na página seguinte)

Entidade A	Card. A	Relacionamento	Card. B	Entidade B	Notas	Origem
Entrada-Mercadoria	(1,N)	é registada em	(1,1)	Loja	Cada entrada de mercadoria ocorre numa loja; uma loja pode ter várias entradas registadas.	RF-06; US-07
Utilizador	(1,1)	registra	(0,N)	Entrada-Mercadoria	Cada entrada é registada por um utilizador; um utilizador pode não ter entradas registadas.	RF-06; US-19
AjusteInventario	(1,N)	afeta	(1,1)	Stock	Cada ajuste afeta o <i>stock</i> de um produto; um <i>stock</i> pode sofrer vários ajustes.	RF-11; US-12
AjusteInventario	(1,N)	tem	(1,1)	MotivoAjuste	Cada ajuste tem um motivo; um motivo pode justificar vários ajustes.	RF-11; US-12
Utilizador	(1,1)	efetua	(0,N)	AjusteInventario	Cada ajuste é efetuado por um utilizador; um utilizador pode não ter ajustes realizados.	RF-11; US-12
Loja	(1,1)	realiza	(0,N)	Inventario-Fisico	Cada inventário físico é realizado numa loja; uma loja pode ainda não ter inventários.	RF-19; US-20
Inventario-Fisico	(1,1)	contém	(1,N)	LinhaInventario	Cada inventário tem pelo menos uma linha; cada linha pertence a um inventário.	RF-19; US-20

(continua na página seguinte)

Entidade A	Card. A	Relacionamento	Card. B	Entidade B	Notas	Origem
LinhaInventario	(1,N)	referencia	(1,1)	Produto	Cada linha de inventário refere um produto; um produto pode figurar em vários inventários.	RF-19; US-20
Utilizador	(1,1)	conduz	(0,N)	Inventario-Fisico	Cada inventário é conduzido por um utilizador; um utilizador pode não ter conduzido inventários.	RF-19; US-20
LogAuditoria	(1,N)	classifica-se em	(1,1)	TipoOperacao	Cada entrada de <i>log</i> tem um tipo de operação; um tipo pode classificar vários registos.	RNF-05; US-11
Utilizador	(1,1)	gera	(0,N)	LogAuditoria	Cada entrada de <i>log</i> é gerada por um utilizador; um utilizador pode não ter entradas de <i>log</i> .	RNF-05; US-11
Loja	(1,1)	executa	(0,N)	Sincronizacao	Cada sincronização é executada por uma loja; uma loja pode ainda não ter sincronizações.	RNF-03; RF-17; US-18
Sincronizacao	(1,N)	tem	(1,1)	EstadoSincronizacao	Cada sincronização tem um estado; um estado pode corresponder a várias sincronizações.	RNF-03; RF-17

(continua na página seguinte)

Entidade A	Card. A	Relacionamento	Card. B	Entidade B	Notas	Origem
Venda	(1,1)	origina	(0,N)	Devolucao	Uma venda pode originar zero ou mais devoluções; cada devolução está associada a uma única venda.	RF-16; US-17
Devolucao	(1,N)	referencia	(1,1)	Produto	Cada devolução refere um produto; um produto pode figurar em várias devoluções.	RF-16; US-17

Tabela 6.1: Relacionamentos do Modelo de Domínio do Mini-Formiga

Anexo 4 — Mockups das Interfaces do Sistema

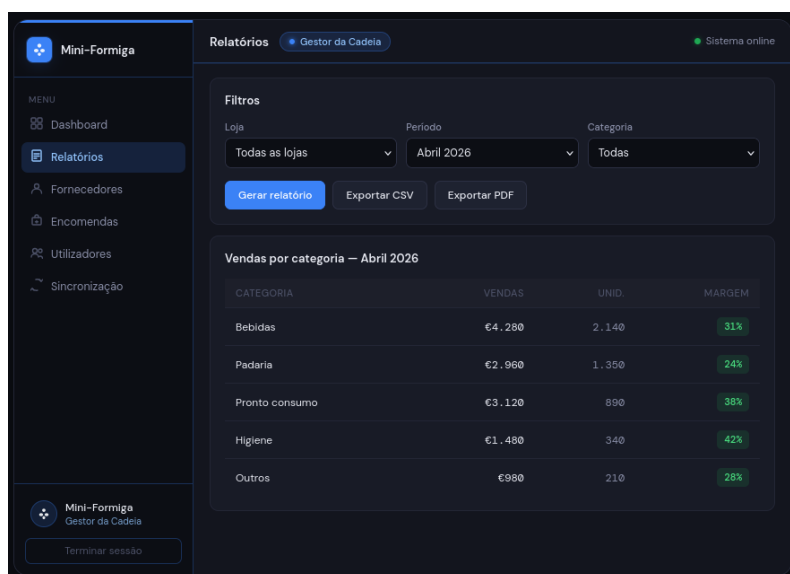


Figura 6.2: Mockup da interface de relatórios do Gestor da Cadeia

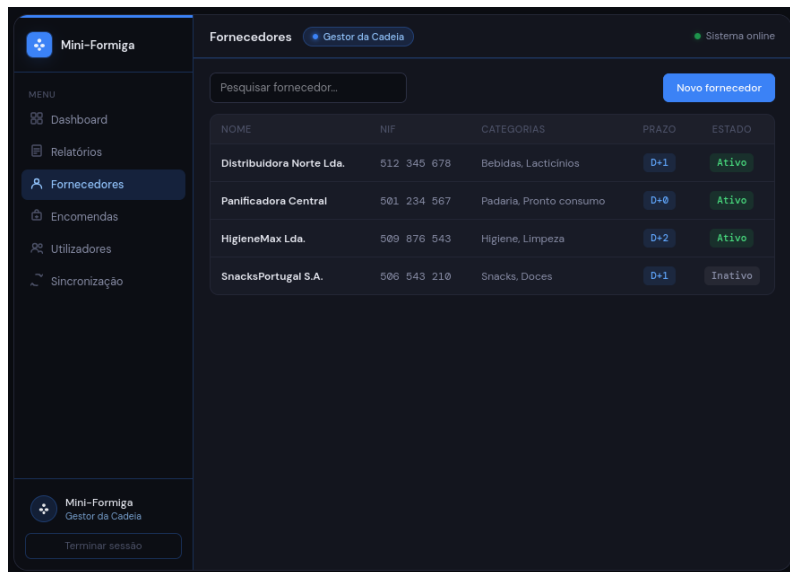


Figura 6.3: Mockup da interface de gestão de fornecedores

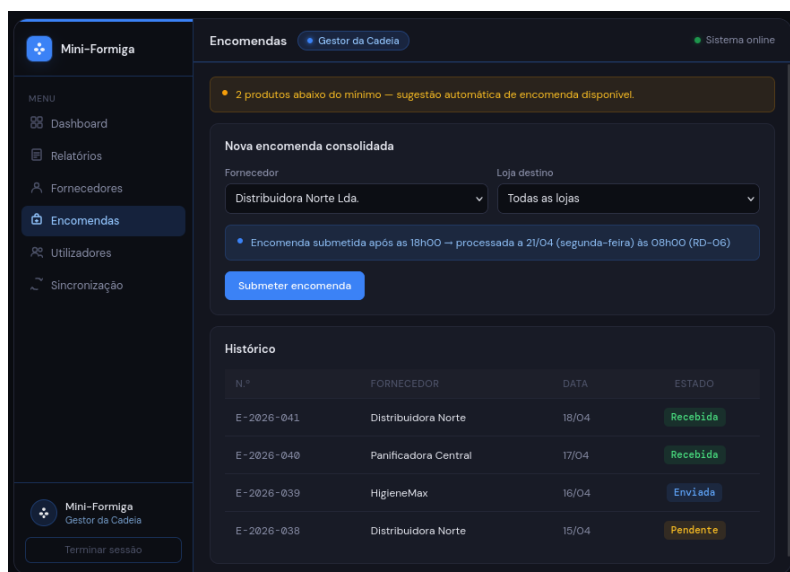


Figura 6.4: Mockup da interface de encomendas

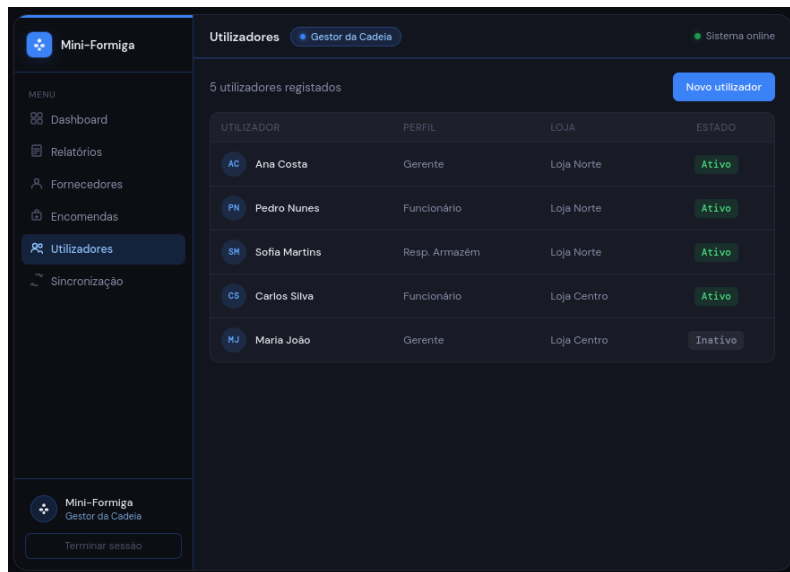


Figura 6.5: Mockup da interface de gestão de utilizadores

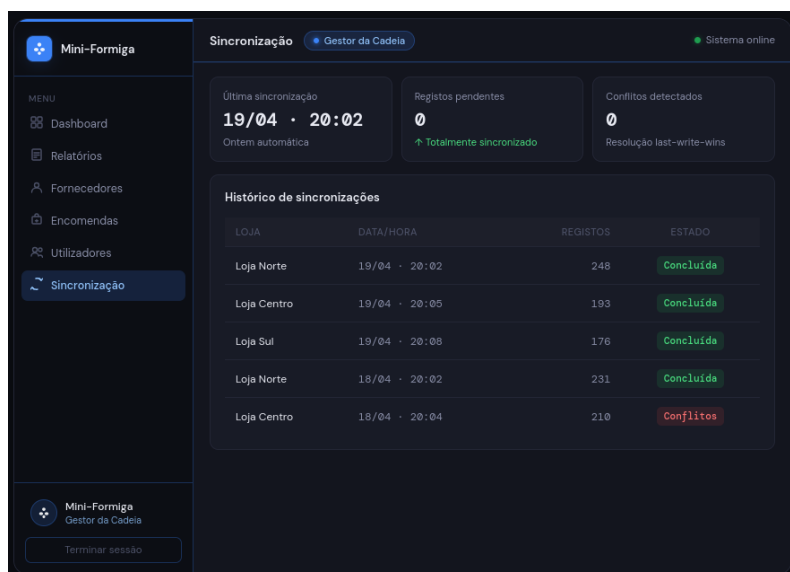


Figura 6.6: Mockup da interface de sincronização

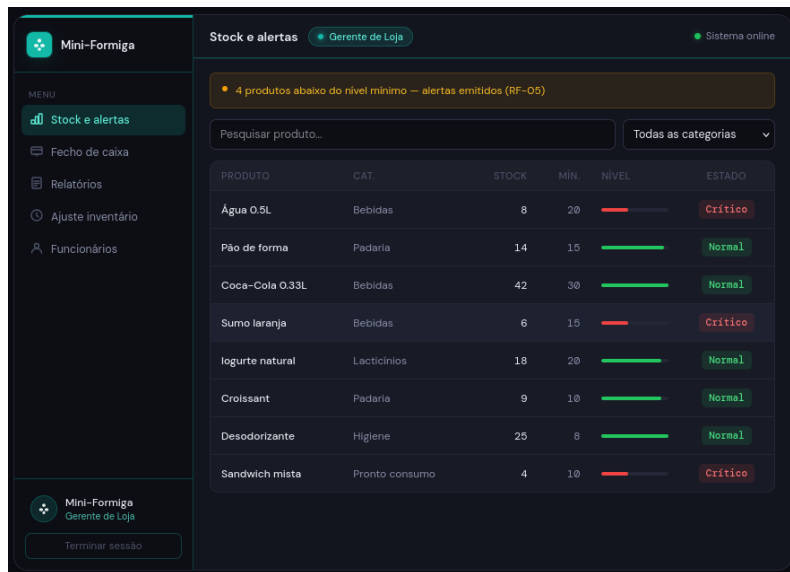


Figura 6.7: Mockup da interface de consulta de stock e alertas

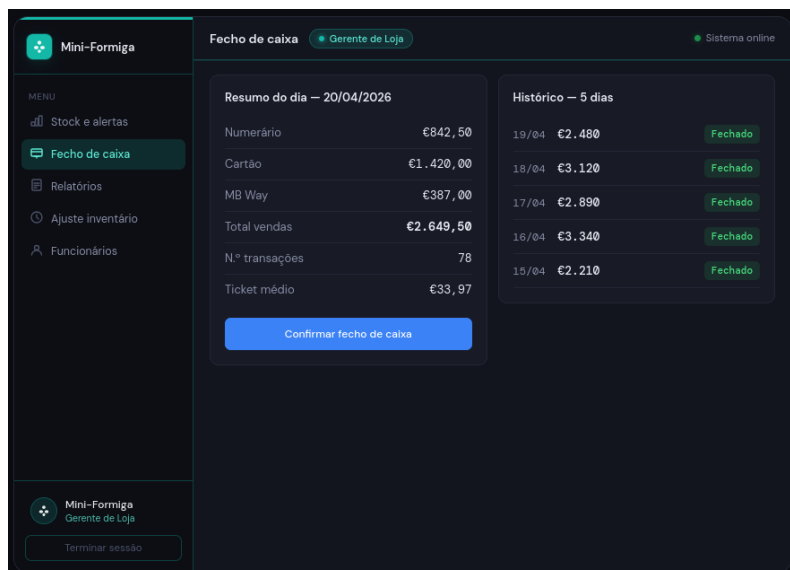


Figura 6.8: Mockup da interface de fecho de caixa

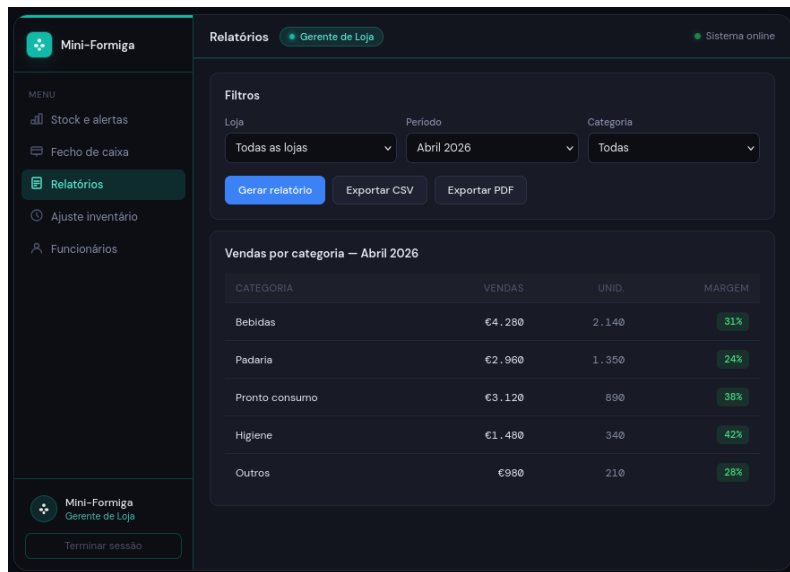


Figura 6.9: Mockup da interface de relatórios do Gerente de Loja

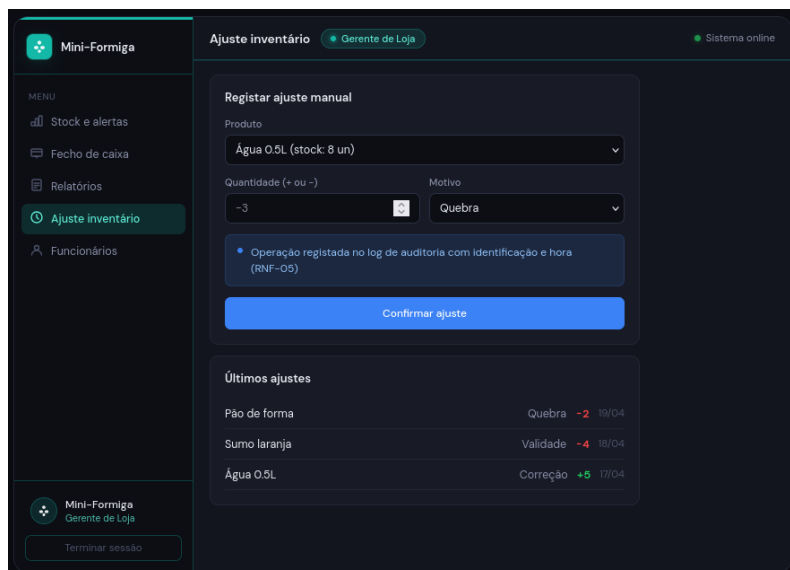


Figura 6.10: Mockup da interface de ajuste manual de inventário

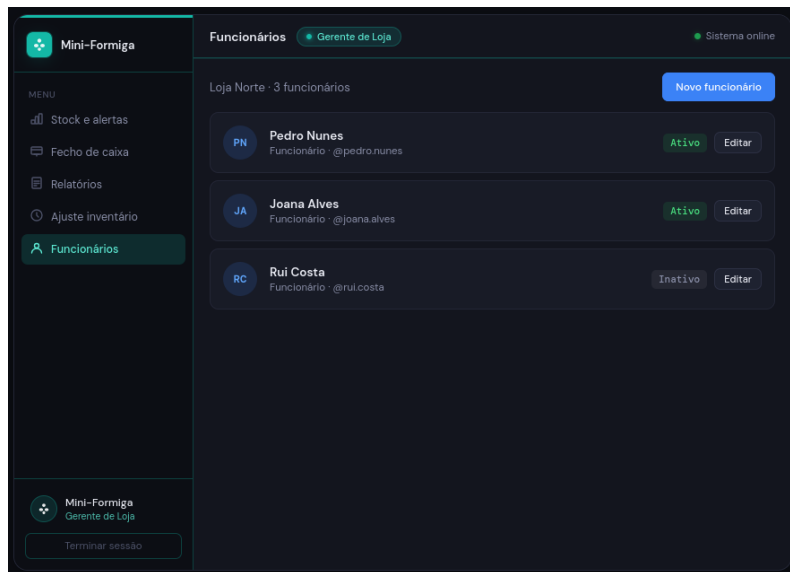


Figura 6.11: Mockup da interface de gestão de funcionários

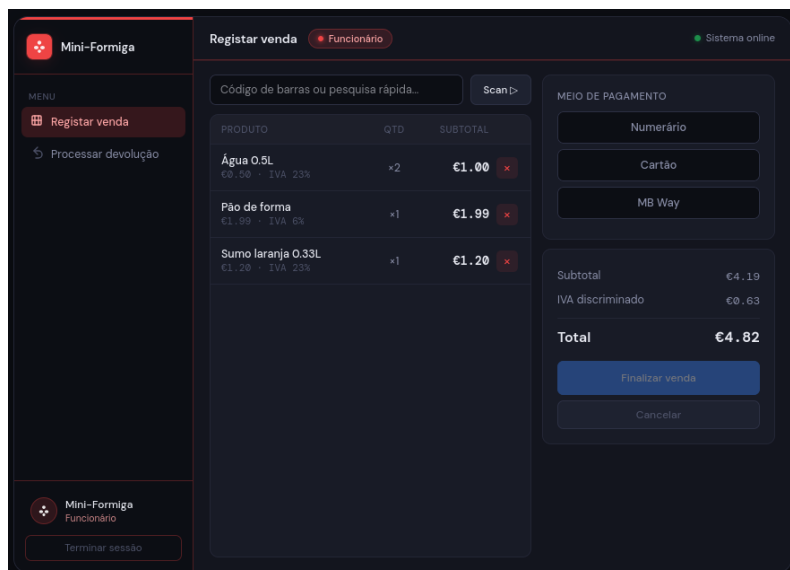


Figura 6.12: Mockup da interface de registo de venda no PDV

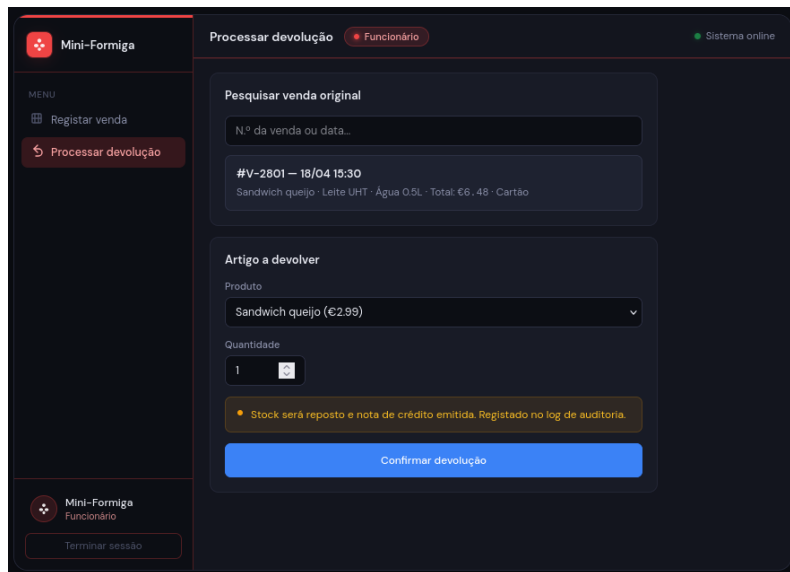


Figura 6.13: Mockup da interface de processamento de devolução

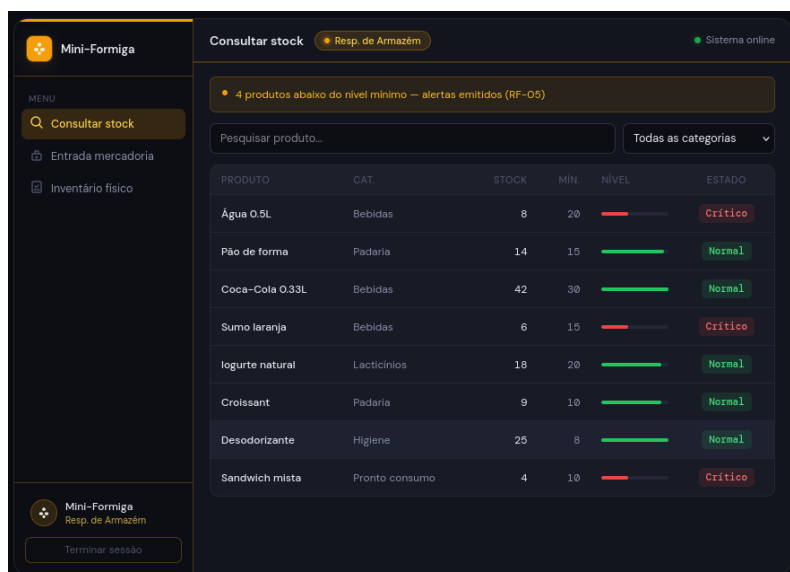


Figura 6.14: Mockup da interface de consulta de stock do armazém

Mini-Formiga

Consultar stock

Entrada mercadoria

Inventário físico

Mini-Formiga

Resp. de Armazém

Terminar sessão

Entrada mercadoria

Resp. de Armazém

Sistema online

Registar entrada de mercadoria

Encomenda associada

E-2026-041 — Distribuidora Norte

N.º guia de remessa

GR-2026-0584

PRODUTO	ENCOM.	RECEBIDO	DIF.
Água 0.5L	48	48	—
Coca-Cola 0.33L	24	24	—
Sumo laranja	36	30	-6
Iogurte natural	24	24	—

Discrepância: Sumo laranja — 6 unidades em falta. Confirme a quantidade recebida.

Confirmar receção e atualizar stock

Figura 6.15: Mockup da interface de registo de entrada de mercadoria

Mini-Formiga

Consultar stock

Entrada mercadoria

Inventário físico

Mini-Formiga

Resp. de Armazém

Terminar sessão

Inventário físico

Resp. de Armazém

Sistema online

Inventário físico — 20/04/2026

Fechar inventário

PRODUTO	SISTEMA	CONTADO	DIFERENÇA
Água 0.5L	8	8	OK
Pão de forma	14	12	-2
Coca-Cola 0.33L	42	42	OK
Sumo laranja	6	8	+2
Iogurte natural	18	20	+2
Croissant	9	9	OK
Desodorizante	25	25	OK
Sandwich mista	4	4	OK

Figura 6.16: Mockup da interface de inventário físico

Anexo 5 — Ata da reunião com o Sr. Formiga para Validação e Refinamento de requisitos

Ata — Sessão de Verificação da Satisfação dos Requisitos do SRS

A sessão de verificação da satisfação dos requisitos com o Sr. Formiga foi realizada após a conclusão da fase de implementação e execução dos testes do sistema Mini-Formiga. O objetivo principal desta reunião consistiu em demonstrar ao cliente que os requisitos funcionais, não funcionais e de domínio especificados se encontram efetivamente implementados e validados, confirmando que o sistema entregue corresponde às necessidades operacionais previamente acordadas.

A equipa começou por apresentar ao Sr. Formiga o terminal de Ponto de Venda, demonstrando o registo de uma venda completa com leitura de código de barras, adição de múltiplos produtos e finalização com diferentes meios de pagamento. O Sr. Formiga questionou se o sistema continuava a funcionar sem ligação à internet, recordando que esta havia sido uma das suas exigências fundamentais. A equipa procedeu à desativação da ligação de rede em tempo real e demonstrou que o registo de vendas prosseguiu sem qualquer interrupção, sendo os dados sincronizados automaticamente com o servidor central após o restabelecimento da ligação, aplicando a regra de resolução de conflitos por sugestão do próprio Sr. Formiga na sessão de validação de requisitos.

De seguida, a equipa apresentou o módulo de controlo de stock e alertas automáticos. O Sr. Formiga pediu para verificar pessoalmente o comportamento do alerta, tendo a equipa simulado uma venda que reduziu o stock de um produto para um valor inferior ao nível mínimo configurado. O alerta foi emitido de forma imediata no painel do gestor, sem qualquer atraso perceptível. O Sr. Formiga confirmou que este era exatamente o comportamento que havia solicitado na reunião anterior, quando pediu que o sistema avisasse logo no instante em que o produto atingisse o mínimo.

Na demonstração do fecho de caixa diário, o Sr. Formiga questionou se a totalização incluía a discriminação por meio de pagamento utilizado. A equipa confirmou que o resumo apresenta os totais por numerário, cartão e MB Way, bem como o número de transações realizadas no dia. O Sr. Formiga considerou esta informação suficiente para o controlo financeiro diário e para a comunicação com o contabilista.

Relativamente aos relatórios e dashboard, a equipa demonstrou a consulta de indicadores de vendas por loja e por período, com representação gráfica e filtros interativos. O Sr. Formiga perguntou se era possível exportar os dados diretamente para o contabilista. A equipa demonstrou a exportação em formato CSV, confirmando que os ficheiros gerados seguem a estrutura de campos acordada. O Sr. Formiga manifestou satisfação com esta funcionalidade, referindo que eliminaria a necessidade de compilar manualmente os dados mensais.

No que respeita à emissão de faturas, o Sr. Formiga questionou se o sistema distinguia corretamente entre fatura simplificada e fatura completa. A equipa demonstrou que, para transações até 1000€ com consumidores finais, é emitida fatura simplificada, e que o

sistema solicita automaticamente o NIF do cliente quando este o requer ou quando o valor excede esse limiar, em conformidade com as normas da Autoridade Tributária. O Sr. Formiga verificou ainda que a numeração dos documentos fiscais é sequencial e ininterrupta.

Por fim, o Sr. Formiga perguntou se o sistema impedia a venda de produtos sem stock disponível. A equipa demonstrou que, ao tentar registar uma venda de um produto com stock zero, o sistema bloqueia a transação e apresenta uma mensagem de alerta ao operador, impedindo a criação de stock negativo.

No final da sessão, o Sr. Formiga declarou que o sistema implementado corresponde às necessidades que havia comunicado ao longo do projeto, considerando que as funcionalidades nucleares do negócio se encontram operacionais e alinhadas com os requisitos acordados. As observações levantadas durante a demonstração foram esclarecidas pela equipa em tempo real, não tendo resultado em alterações adicionais à implementação.

Anexo 6 — Guia de Utilização e Manutenção do Trabalho

Objetivo do guia

Este anexo reúne as instruções necessárias para executar, utilizar, validar e manter o sistema Mini-Formiga no contexto do trabalho desenvolvido. O objetivo é tornar explícito o procedimento operacional associado à implementação entregue, incluindo os comandos de arranque, os perfis de execução, os acessos de demonstração, os testes de validação e os cuidados de manutenção mais relevantes.

Pré-requisitos

Para executar o projeto em ambiente local são necessários os seguintes componentes:

- Java 21;
- Maven 3.9 ou superior;
- Node.js 20 ou superior;
- npm;
- Docker e Docker Compose, caso seja usado o arranque containerizado ou o PostgreSQL central em container.

Antes de iniciar a aplicação, devem ser confirmadas as versões ativas:

```
java -version
javac -version
node -v
npm -v
```

O *backend* foi desenvolvido para Java 21. Caso seja usado um JDK anterior, o Maven poderá falhar durante a compilação com erro relacionado com a versão `release 21`. O *frontend* pressupõe uma versão recente de Node.js compatível com as dependências de TypeScript, Vite e ESLint.

Estrutura do projeto

O repositório encontra-se organizado em dois blocos principais:

- `backend/`: aplicação Spring Boot, executável em perfil `local` para a loja e em perfil `central` para consolidação;
- `frontend/`: aplicação React com Vite e TypeScript.

A raiz do projeto inclui ainda o ficheiro `docker-compose.yml`, responsável por orquestrar a execução integrada da base de dados central, dos dois perfis de *backend* e do *frontend*.

Arranque rápido com Docker Compose

A forma recomendada de arrancar a solução completa para demonstração é através de Docker Compose, a partir da raiz do projeto:

```
docker compose up --build
```

Este comando inicia os seguintes serviços:

- PostgreSQL central;
- *backend* central, no perfil `central`, disponível na porta 8081;
- *backend* local da loja, no perfil `local`, disponível na porta 8080;
- *frontend* React/Vite, disponível na porta 3000.

Após o arranque, os principais endereços são:

- *Frontend*: `http://localhost:3000`;
- *Backend* local: `http://localhost:8080`;
- *Backend* central: `http://localhost:8081`;
- Swagger local: `http://localhost:8080/api/swagger-ui.html`;
- Swagger central: `http://localhost:8081/api/swagger-ui.html`.

O PostgreSQL é exposto no *host* através da porta 5433, evitando conflito com instalações locais que utilizem a porta padrão 5432.

Arranque manual

Em alternativa ao Docker Compose completo, os componentes podem ser arrancados manualmente em consolas separadas.

Base de dados central

```
docker compose up -d postgres
```

Backend central

```
cd backend
SPRING_PROFILES_ACTIVE=central \
SERVER_PORT=8081 \
DB_URL=jdbc:postgresql://localhost:5433/miniFormiga_central \
DB_USERNAME=miniFormiga \
DB_PASSWORD=miniFormiga_dev \
mvn spring-boot:run
```

Backend local da loja

```
cd backend
SPRING_PROFILES_ACTIVE=local \
SERVER_PORT=8080 \
DB_URL=jdbc:sqlite:./mini-formiga-local.db \
CENTRAL_SYNC_URL=http://localhost:8081/api/v1/central/sincronizacao/receber \
mvn spring-boot:run
```

Frontend

```
cd frontend
npm ci
VITE_API_BASE_URL=http://localhost:8080/api/v1 \
VITE_CENTRAL_API_BASE_URL=http://localhost:8081/api/v1 \
npm run dev -- --host 0.0.0.0 --port 3000
```

Depois do arranque, a interface deve ser aberta em <http://localhost:3000>.

Utilização da aplicação

O sistema disponibiliza dados de demonstração por omissão, através da propriedade `DEMO_DATA_ENABLED=true`. Todos os utilizadores de demonstração usam a mesma palavra-passe:

MiniFormiga2026!

As contas disponíveis são:

- `gestor.formiga`;
- `gerente.braga`, `operador.braga` e `armazem.braga`;
- `gestor.porto`, `gerente.porto`, `operador.porto` e `armazem.porto`;
- `gestor.lisboa`, `gerente.lisboa`, `operador.lisboa` e `armazem.lisboa`.

O *frontend* escolhe a API de acordo com o perfil funcional:

- o Gestor da Cadeia consulta o servidor central, em `http://localhost:8081/api/v1`;
- o Gerente de Loja, o Funcionário e o Responsável de Armazém operam sobre o servidor local da loja, em `http://localhost:8080/api/v1`.

Esta separação reproduz a arquitetura implementada: cada loja mantém operação local com SQLite, enquanto o servidor central consolida informação recebida por sincronização.

Validação rápida

Para confirmar que o *backend* local está acessível e que a autenticação funciona, pode ser efetuado um pedido de login:

```
curl -s -X POST http://localhost:8080/api/v1/auth/login \  
-H 'Content-Type: application/json' \  
-d '{"username":"gestor.formiga","password":"MiniFormiga2026!"}'
```

A documentação OpenAPI pode ser validada através dos seguintes endereços:

```
curl -I http://localhost:8080/api/swagger-ui.html  
curl -s http://localhost:8080/api/docs
```

A validação automatizada do *backend* deve ser feita com:

```
cd backend  
mvn clean verify
```

A validação do *frontend* deve ser feita com:

```
cd frontend  
npm run lint  
npm run build
```

Configuração e manutenção

As principais variáveis de configuração são:

- `SPRING_PROFILES_ACTIVE`: define o perfil ativo, `local` ou `central`;
- `SERVER_PORT`: define a porta HTTP de cada instância;
- `DB_URL`, `DB_USERNAME` e `DB_PASSWORD`: configuram a ligação à base de dados;
- `JWT_SECRET` e `JWT_EXPIRATION_MINUTES`: configuram a autenticação JWT;
- `SYNC_TOKEN` e `CENTRAL_SYNC_URL`: configuram a comunicação de sincronização;
- `DEMO_DATA_ENABLED`: ativa ou desativa o carregamento de dados de demonstração;
- `CORS_ALLOWED_ORIGINS`: define as origens autorizadas para o *frontend*.

Em ambiente de desenvolvimento, os ficheiros SQLite locais, os diretórios de *logs* e os artefactos de construção são considerados artefactos gerados. Estes elementos não devem ser tratados como fonte principal do projeto e podem ser recriados a partir do código e dos dados de arranque.

Cuidados operacionais

Para manter o sistema em boas condições de execução, devem ser observados os seguintes cuidados:

- executar os testes automatizados antes de integrar alterações relevantes;
- confirmar que os perfis `local` e `central` usam bases de dados distintas;
- alterar os segredos de desenvolvimento, como `JWT_SECRET` e `SYNC_TOKEN`, antes de qualquer utilização fora de ambiente local;
- verificar periodicamente os ficheiros de auditoria em `logs/`, uma vez que registam operações sensíveis;
- preservar cópias de segurança da base de dados central PostgreSQL quando houver dados reais;
- garantir que a comunicação externa em contexto de *deployment* é protegida por HTTPS/TLS, conforme assumido na implementação.

Para parar os serviços containerizados, pode ser usado:

```
docker compose down
```

Para remover apenas artefactos locais de construção, mantendo o código-fonte, podem ser usados os comandos de limpeza próprios de cada módulo, como `mvn clean` no *backend* e remoção/reinstalação de `node_modules` no *frontend*, quando necessário.